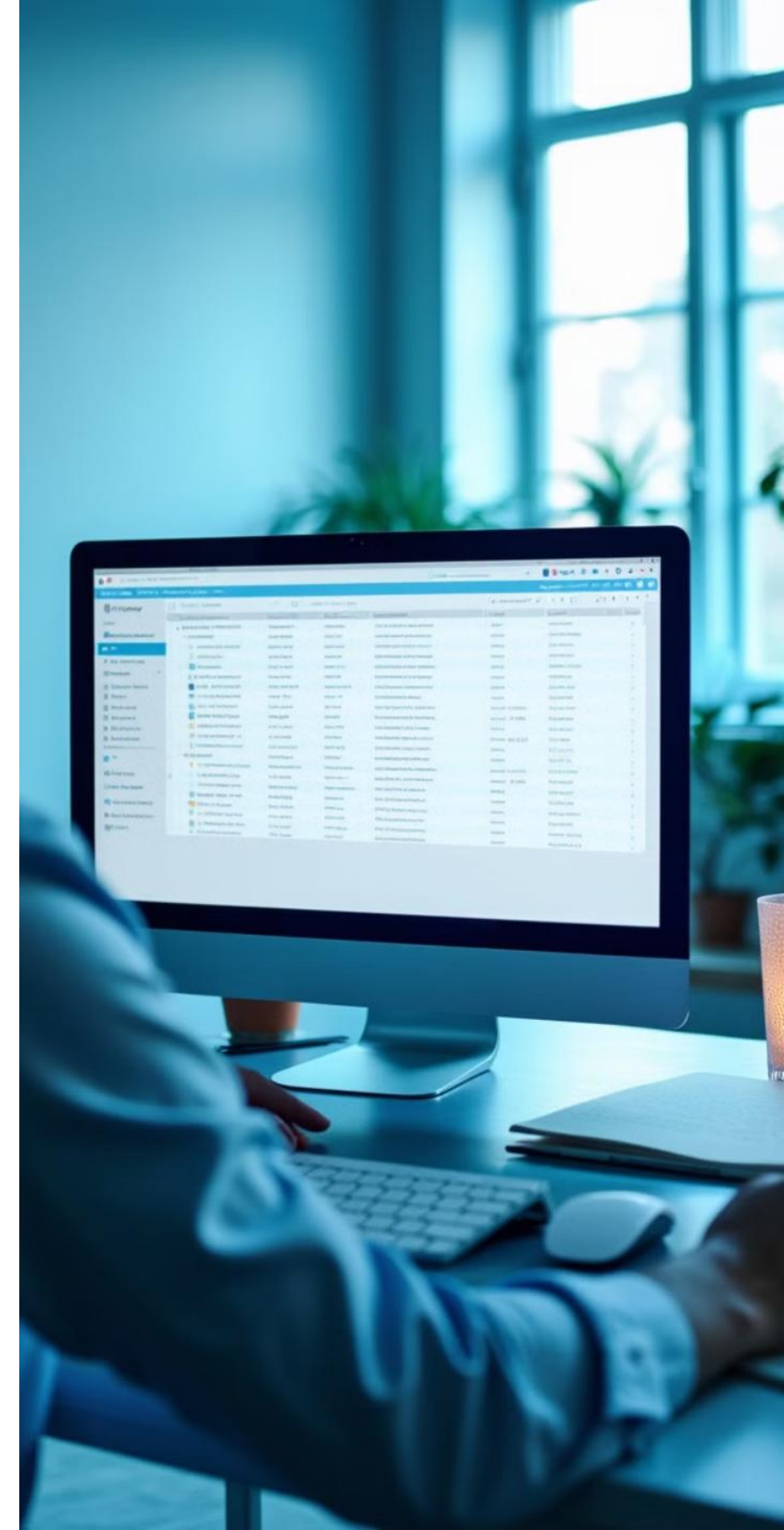


MÓDULO: ACCESO A DATOS



UT1: Manejo de ficheros





RESULTADO DE APRENDIZAJE: RA1

RA1.- Desarrolla aplicaciones que gestionan información almacenada en ficheros identificando el campo de aplicación de los mismos y utilizando clases específicas.

CRITERIOS DE EVALUACIÓN

- | | |
|----|---|
| a) | Se han utilizado clases para la gestión de ficheros y directorios. |
| b) | Se han valorado las ventajas y los inconvenientes de las distintas formas de acceso. |
| c) | Se han utilizado clases para recuperar información almacenada en ficheros. |
| d) | Se han utilizado clases para almacenar información en ficheros. |
| e) | Se han utilizado clases para realizar conversiones entre diferentes formatos de ficheros. |
| f) | Se han previsto y gestionado las excepciones. |
| g) | Se han probado y documentado las aplicaciones desarrolladas. |



Contenidos de la UT1

- 1 **Introducción**
Conceptos básicos sobre ficheros y su estructura
- 2 **Clases para gestión de ficheros y directorios**
Operaciones fundamentales con la clase File
- 3 **Operaciones generales sobre ficheros**
Lectura y escritura
- 4 **Formas de acceso a ficheros y flujos de datos**
Acceso secuencial y aleatorio. Basados en bytes y caracteres.
Clases para gestión de flujos de datos
- 5 **Trabajo con ficheros XML**
Análisis y conversión de datos. Parsing y Binding.
- 6 **Manejo de excepciones**
Gestión de errores en operaciones con ficheros
- 7 **Documentación de aplicaciones**

1. INTRODUCCIÓN

- MEMORIA FRENTE A DISCO
- PERSISTIR LOS DATOS



2. FICHEROS Y DIRECTORIOS

- CREAR
- BORRAR
- COPIAR
- MOVER



3. LEER Y ESCRIBIR

- LECTURA
- ESCRITURA



4. FORMAS DE ACCESO

- SECUENCIAL
- ALEATORIO



MANEJO DE FICHEROS Y FLUJOS DE DATOS UT1 · ACCESO A DATOS

7. DOCUMENTACIÓN Y PRUEBAS

- JAVADOC
- CASOS DE PRUEBA



6. EXCEPCIONES

- CAPTURAR
- DIFERENCIAR TIPOS
- DECLARAR



5. FICHEROS XML

- CONCEPTOS CLAVE
- ANALIZADOR (PARSER)
- VINCULACIÓN (JAXB)
- CONVERSIÓN A OTROS FORMATOS



1. INTRODUCCIÓN

1.1. Persistencia de datos en ficheros

- **Ficheros** = forma más básica de almacenamiento.
- Al final, todos los sistemas escriben datos en ficheros.
- **Analogía histórica:** archivadores con fichas de papel (biblioteca: título, autor, tema).



- **Estructura:** ficheros con registros y campos de longitud fija.
- **Búsquedas más rápidas:** ficheros índice → ISAM (IBM).
- **COBOL (1959):** gran soporte para ficheros indexados (muy usado en banca).
- **Evolución:** las BBDD relacionales sustituyen a los sistemas basados en ficheros para grandes volúmenes/estructuras regulares, pero internamente también gestionan ficheros.
- **Qué haremos en la UT:** organizaciones sencillas + programas en Java para consultar, añadir, borrar y modificar datos.

1. INTRODUCCIÓN

1.1. Persistencia de datos en ficheros



- **Consultas complejas:** difíciles de programar.
- **Muchos datos o cambios frecuentes:** bajo rendimiento (especialmente en borrados/modificaciones).
- **Acceso concurrente:** riesgo de inconsistencias o daños; controlar accesos y transacciones complica el sistema.
- **Integridad:** difícil imponer y verificar restricciones.
- **Redundancia:** duplicar datos → desperdicio y valores incoherentes.

¿Se siguen utilizando?

- **Ficheros XML para almacenar datos.**
- **Correo electrónico:** formatos de fichero sencillos.
- **Procesos masivos (periódicos o puntuales) basados en texto.**
- **Intercambio de datos:** exportación/importación.
- **Copias de seguridad y múltiples formatos propios de aplicaciones.**

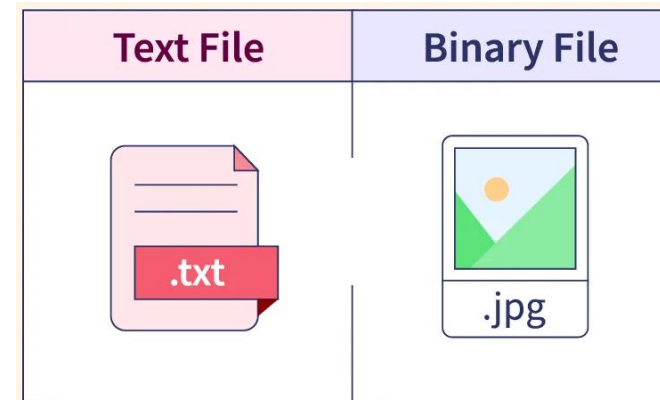
1. INTRODUCCIÓN

1.2. Estructura de los ficheros

CLASIFICACIÓN

Ficheros de caracteres

Almacenan texto legible por cualquier editor de texto.



Ficheros binarios

Contienen datos no legibles directamente (imágenes, vídeos, etc.), y requieren programas específicos para su lectura.

CLASIFICACIÓN POR TIPO DE ACCESO

Acceso secuencial

Se leen los datos de manera ordenada desde el principio hasta el final.



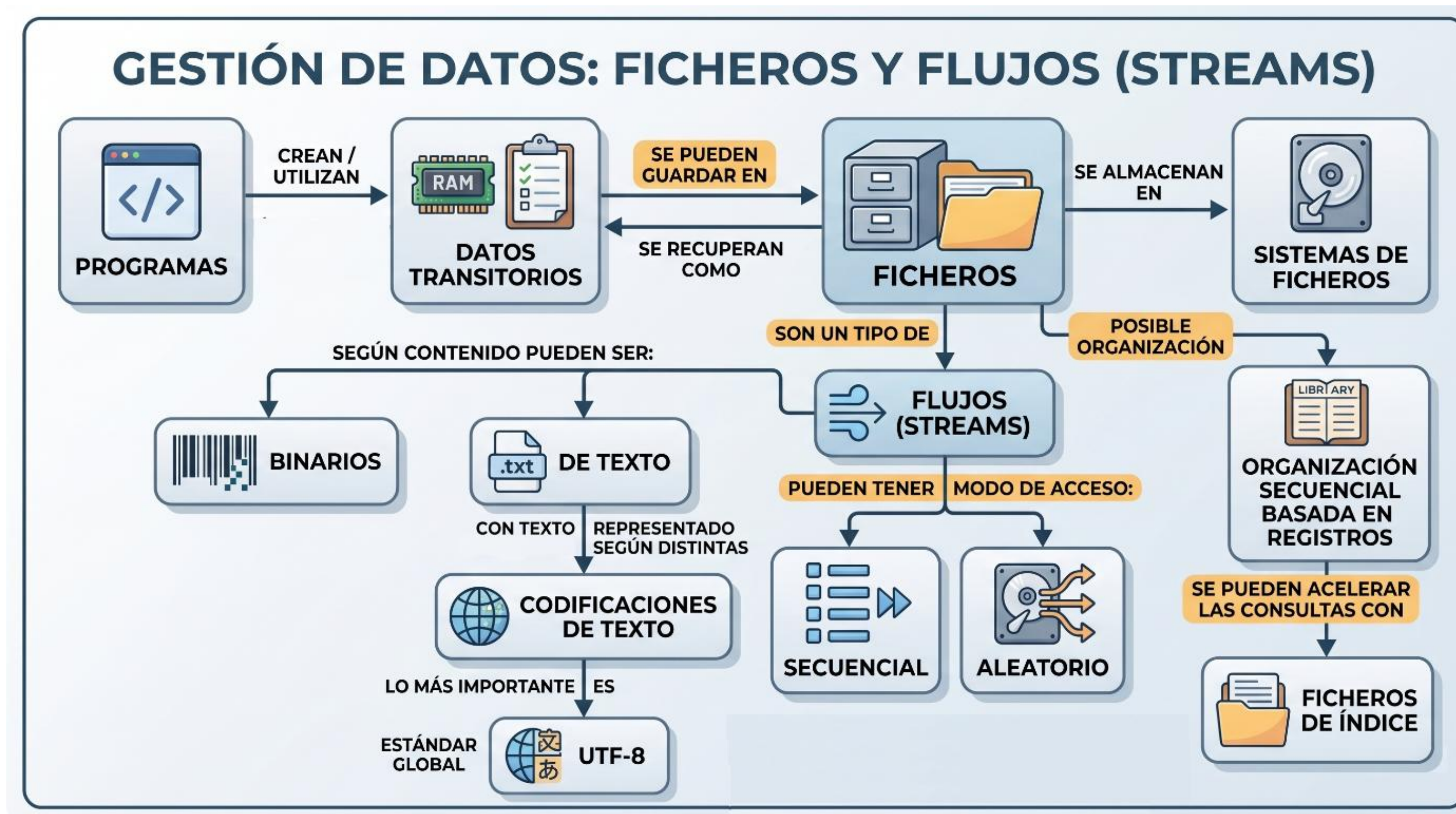
Acceso aleatorio

Permite acceder directamente a una posición específica sin necesidad de leer los datos anteriores.

uental File

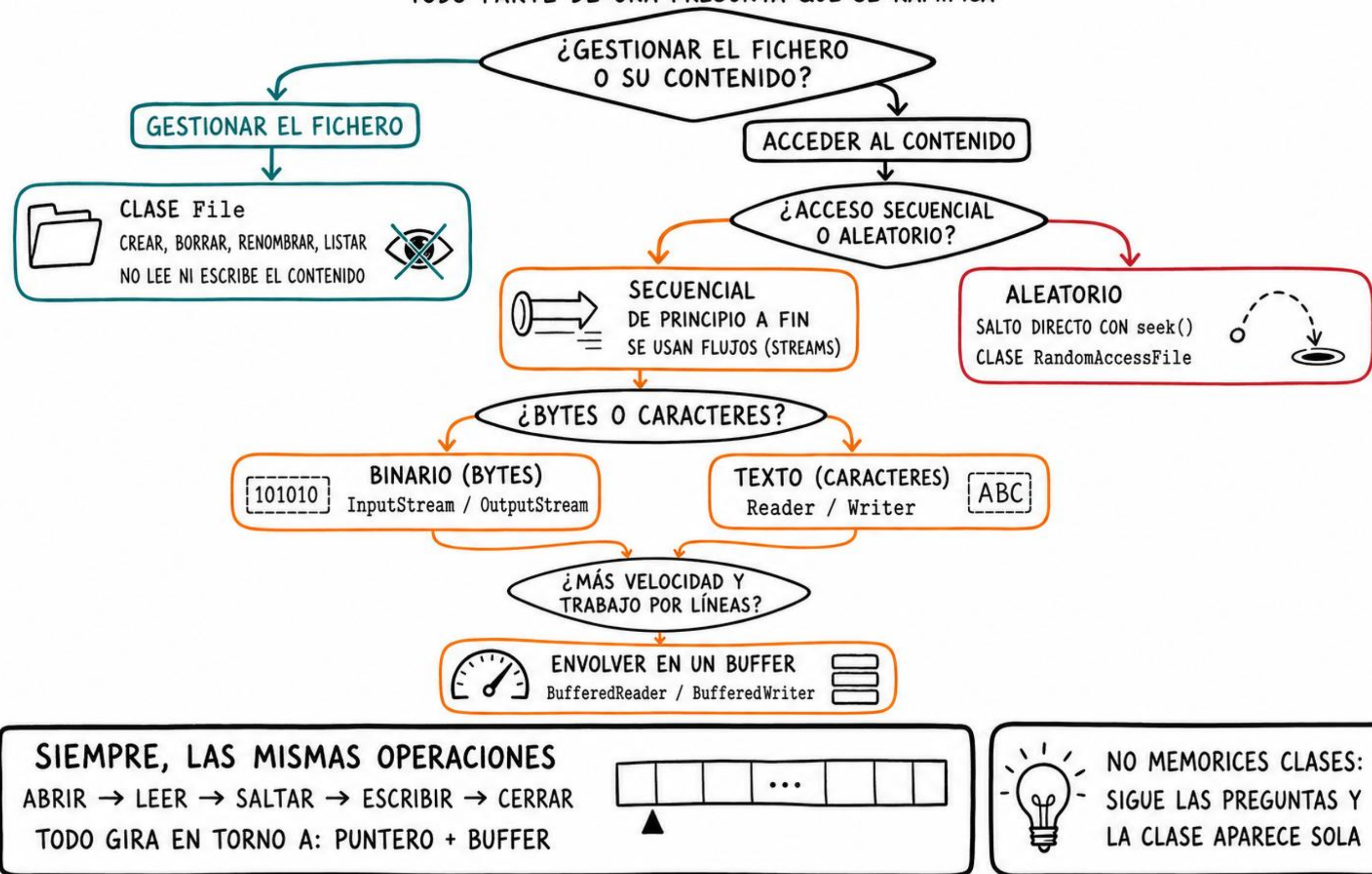
andom File Access

MAPA CONCEPTUAL



MANEJO DE FICHEROS EN JAVA

TODO PARTE DE UNA PREGUNTA QUE SE RAMIFICA

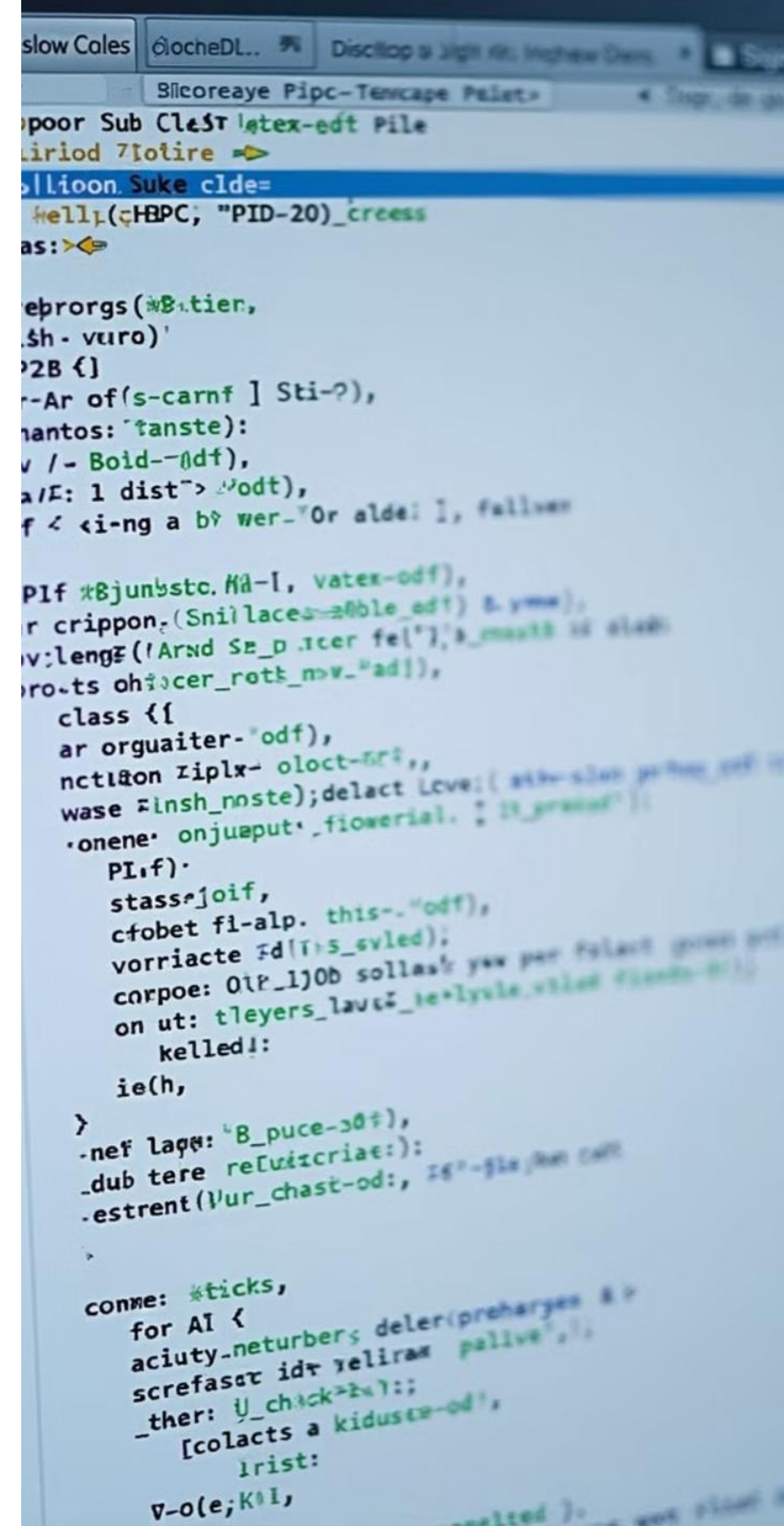


2. Clase File: Gestión de ficheros y directorios

La **clase File**, perteneciente al **paquete java.io**, es fundamental para la gestión de ficheros y directorios. Representa nombres de archivos o directorios y permite operaciones como la creación, renombrado, eliminación y consulta de información.



<https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/io/File.html>



2. Clase File: métodos destacados

1

Creación

- **createNewFile()**: Crea un nuevo fichero vacío.
- **mkdir()**: Crea un nuevo directorio.
- **makedirs()**: Crea directorios intermedios que no existan.

2

Información

- **exists()**: Verifica si un fichero o directorio existe.
- **isFile()**: Verifica si es un fichero.
- **isDirectory()**: Verifica si es un directorio.
- **getName()**: Devuelve el nombre del fichero o directorio.
- **getAbsolutePath()**: Devuelve la ruta absoluta del fichero o directorio.

3

Modificación

- **renameTo(File dest)**: Renombra o mueve un fichero.
- **deleteOnExit()**: Marca el fichero para ser eliminado cuando finalice la ejecución.



¿QUÉ PASA CON EL ACCESO A LOS FICHEROS (write, read...)?

El acceso a los ficheros se trata como un flujo (stream) de información entre el programa y el fichero.

Lo veremos en el siguiente punto

Aplicaciones reales

La gestión de ficheros es esencial para diversas aplicaciones modernas, como:

- Servicios como **Dropbox y Google Drive**: Dependen de la creación y sincronización de ficheros.
- Aplicaciones multimedia, como editores de audio y vídeo, que requieren acceso secuencial o aleatorio a los datos.



1 CONSTRUCTOR

`File(String ruta)`

CREA EL OBJETO PARA UN FICHERO O DIRECTORIO



**File REPRESENTA RUTAS;
NO LEE NI ESCRIBE
EL CONTENIDO DEL FICHERO**

2 CONSULTAR PROPIEDADES

- `exists()`
- `isFile()`
- `isDirectory()`
- `canRead()`
- `canWrite()`
- `length()`
- `getName()`
- `getParent()`



4 CREAR, BORRAR Y RENOMBRAR

- `createNewFile()`
- `mkdir()`
- `delete()`
- `renameTo()`



5 PUENTE HACIA java.nio

`toPath()`

`Path`

DA ACCESO A FUNCIONALIDAD ADICIONAL (COPIAR, MOVER...)



3 LISTAR CONTENIDO

- `list()`
- `listFiles()`

DEVUELVEN LO QUE HAY DENTRO DE UN DIRECTORIO



**File GESTIONA EL FICHERO DESDE FUERA;
PARA LEER O ESCRIBIR SU CONTENIDO SE USAN FLUJOS**



ACTIVIDAD PRÁCTICA



Actividad 2 RA1_A2

Gestión básica de ficheros y directorios

Realizar una serie de operaciones básicas sobre ficheros y directorios utilizando la clase File de Java. Estas operaciones incluyen la creación de un fichero, la verificación de su existencia, el renombrado del fichero y, finalmente, su eliminación.

[Ver Actividad 2 en Aula Virtual]

Ejemplo: Mostrar información de ficheros y directorios

```
// Uso de la clase File para mostrar información de ficheros y directorios
import java.io.File;
import java.io.IOException;

public class ListadoDirectorio {
    public static void main(String[] args) {
        String ruta = ".";
        if (args.length >= 1) {
            ruta = args;
        }

        File fich = new File(ruta);
        if (!fich.exists()) {
            System.out.println("No existe el fichero o directorio (" + ruta + ").");
        } else {
            if (fich.isFile()) {
                System.out.println(ruta + " es un fichero.");
            } else {
                System.out.println(ruta + " es un directorio. Contenidos: ");
                File[] ficheros = fich.listFiles(); // Ojo, ficheros o directorios
                for (File f : ficheros) {
                    String textoDescr = f.isDirectory() ? "/" : "";
                    if (f.isFile()) {
                        textoDescr += " ";
                    }
                    System.out.println(textoDescr + f.getName());
                }
            }
        }
    }
}
```

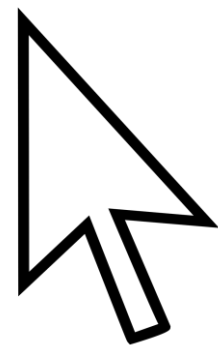
3. Operaciones sobre ficheros

Mecanismo de acceso a un fichero

¿CÓMO SE ACCEDE A UN FICHERO?

El mecanismo de acceso a un fichero está basado en un **puntero** y en una zona de memoria que se suele llamar **buffer**. El puntero siempre apunta a un lugar del fichero, o bien a una posición especial de fin de fichero, que a veces se denomina EOF (del inglés end of file). Esta se puede entender que está situada inmediatamente a continuación del último byte del fichero.

Puntero



EOF

Buffer

CABDEFGAGDADRAP ►►

Operaciones básicas con ficheros

¿QUÉ SE PUEDE HACER CON LOS FICHEROS?

Todas las clases para operar con ficheros disponen de las siguientes operaciones básicas:

- ☐ **Apertura**

Antes de hacer nada con un fichero, hay que abrirlo. Esto se hace al crear una instancia de una clase que se utilizará para operar con él.

- ☐ **Lectura**

Mediante el método **read()**. Consiste en leer contenidos del fichero para volcarlos a memoria y poder trabajar con ellos. El puntero se sitúa justo después del último carácter leído.



■ Salto

Mediante el método **skip()**.

Consiste en hacer avanzar el puntero un número determinado de bytes o caracteres hacia delante.

■ Escritura

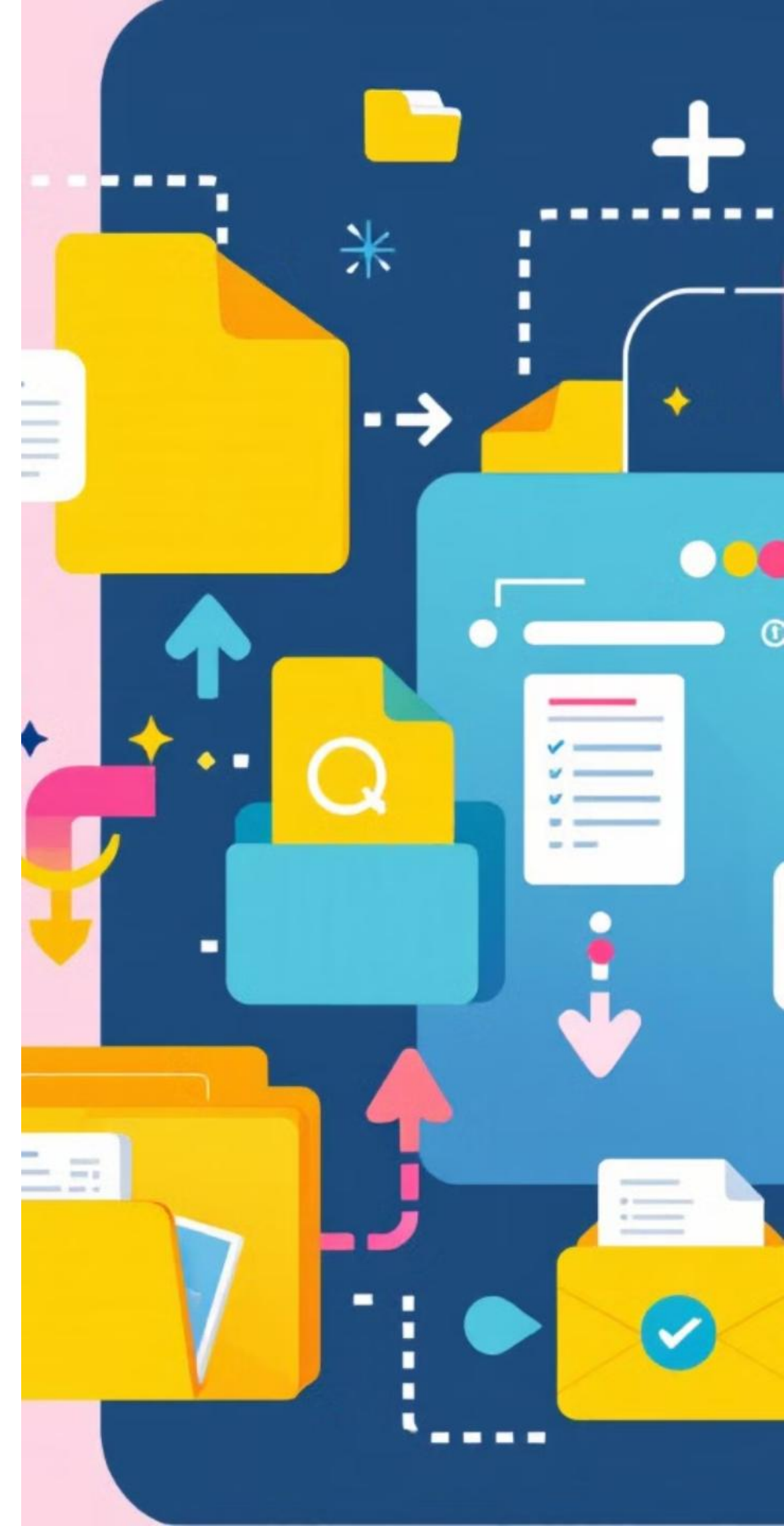
Mediante el método **write()**.

Consiste en escribir contenidos de memoria en un lugar determinado del fichero. El puntero se sitúa justo después del último carácter escrito.

■ Cierre

Mediante el método **close()**.

Para terminar, hay que cerrar el fichero.



3.1 Operaciones de lectura

¿Qué ocurre al leer?



Cuando se lee desde el fichero, hay que indicar el buffer, que recibirá los datos que se lean desde él. Si no se indica el número de bytes o de caracteres para leer, se leerá hasta llenar el buffer. Lógicamente, no se leerá nada si el puntero apunta a la posición EOF (final del fichero).

Buffer

El buffer es una zona de memoria que se utiliza para almacenar los datos que se leen desde el fichero.

Lectura

La operación de lectura consiste en transferir datos del fichero al buffer.

EOF

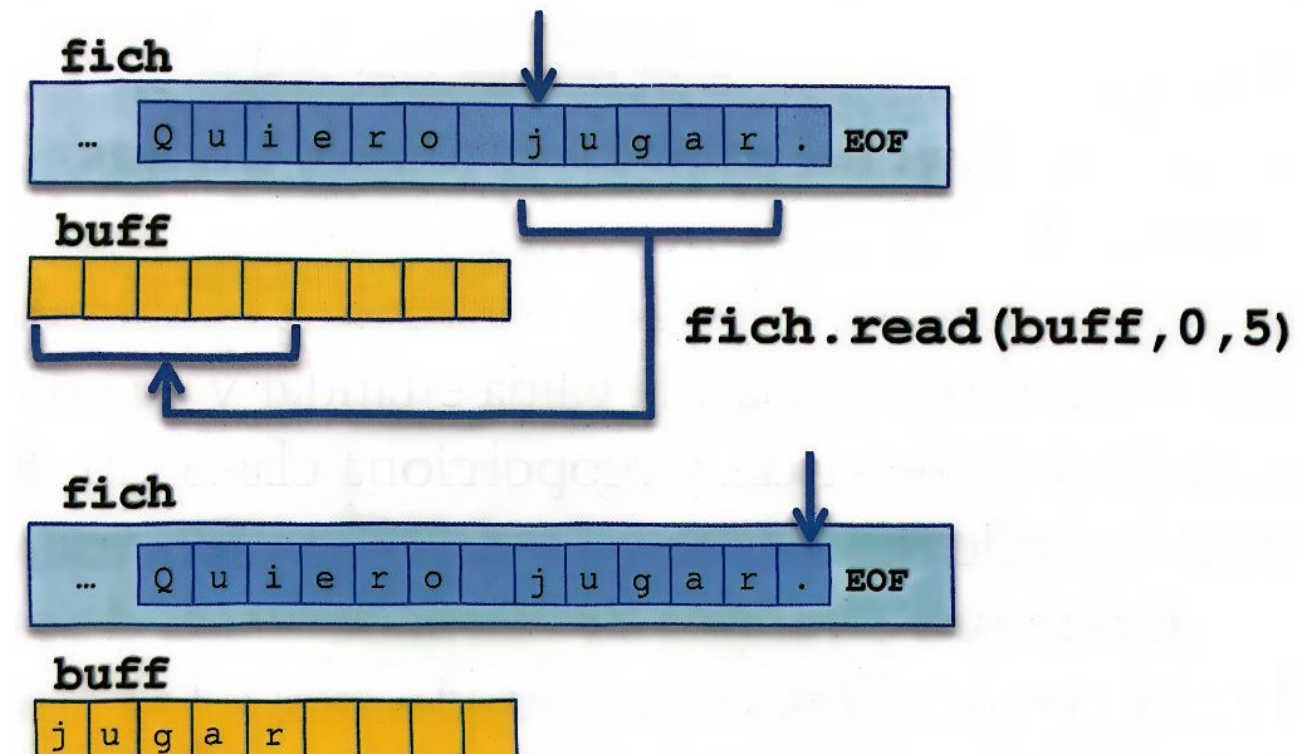
EOF (final del fichero) es una posición especial que indica el final del fichero. Si el puntero apunta a EOF, no se leerán más datos.

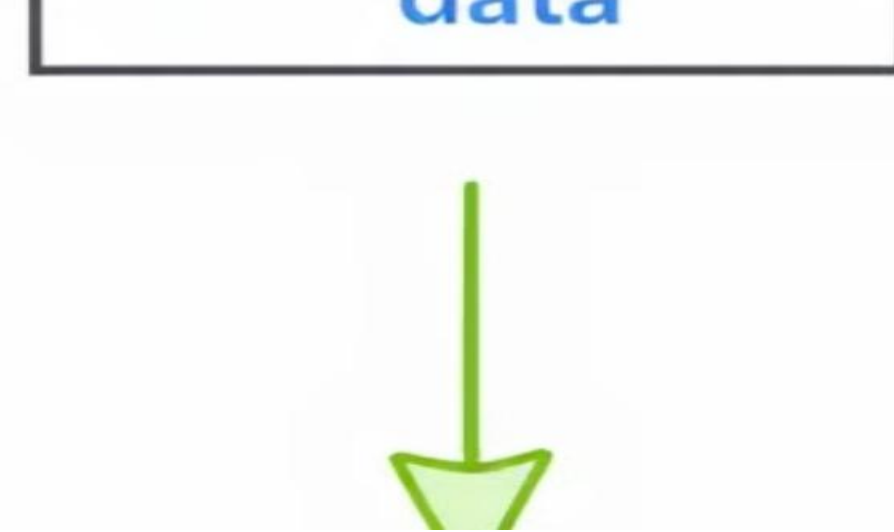


Ejemplo de operación de lectura

Ejemplo para un fichero de texto en el que cada carácter ocupa un byte.

Ten en cuenta que la operación de lectura podría realizarse para un fichero tanto binario como de texto. En el primer caso, se leerían bytes, y en el segundo, caracteres.



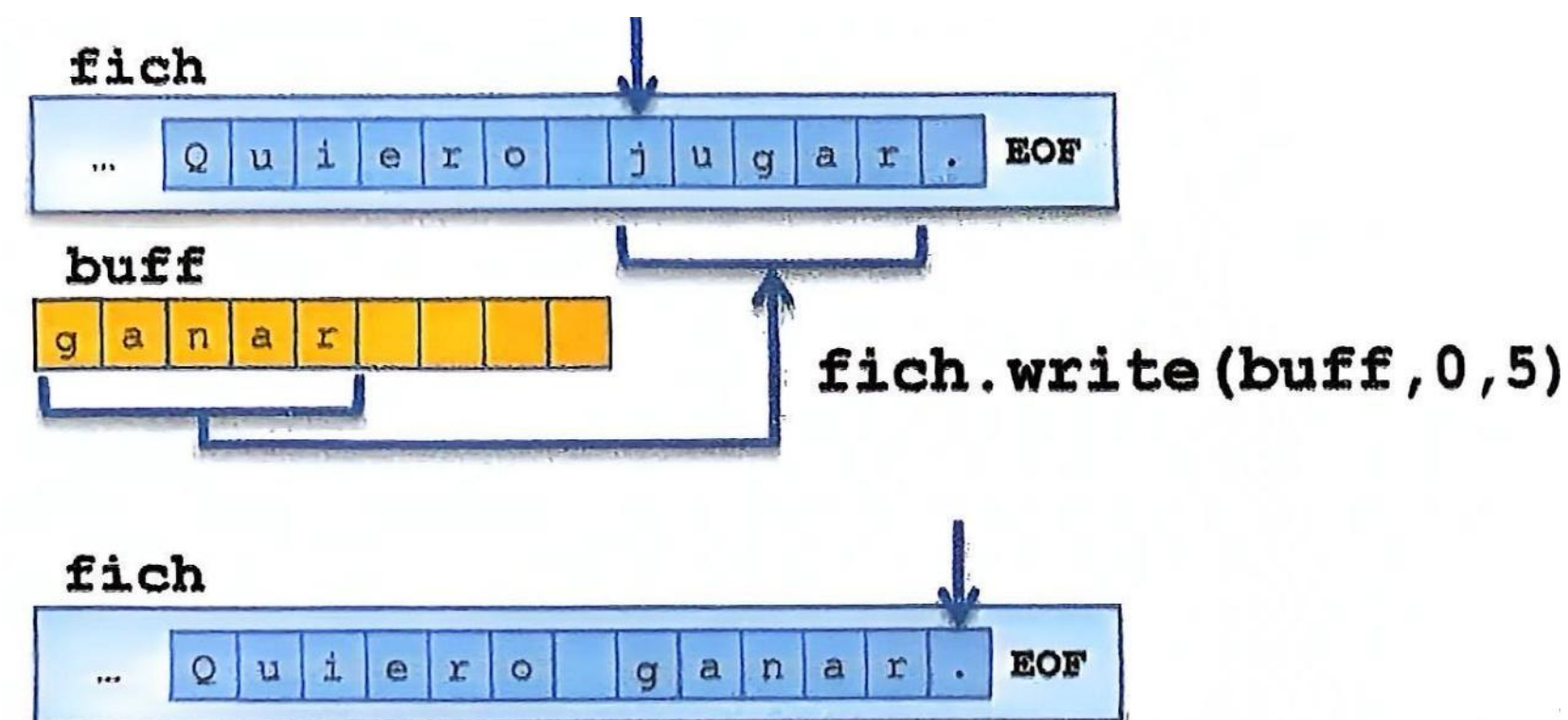


3.2 Operaciones de escritura

¿Qué ocurre al escribir?

Cuando se escribe en el fichero, hay que indicar el buffer y el número de bytes para leer. Desde el buffer se transfieren los datos a la posición a la que apunta el puntero.

- Si el puntero apunta al final del fichero, se añaden los datos al final de este.



Conclusión operaciones básicas

Poco más se puede hacer que lo visto hasta ahora.

Insertar datos

No existe ninguna forma directa de insertar datos en medio de un fichero

Eliminar fragmento

No existe ninguna forma directa de eliminar un fragmento de un fichero

Sustituir fragmento

No existe ninguna forma directa de sustituir los contenidos de un fragmento de un fichero por otros, como no sea que tengan la misma longitud.

Esto es así en Java y en prácticamente todos los lenguajes de programación, y se debe a que las implementaciones de los sistemas de ficheros más habituales no proporcionan ninguna manera directa para hacer operaciones de este tipo. Pero se puede hacer utilizando **ficheros auxiliares**.

OPERACIONES SOBRE FICHEROS EN JAVA

LAS MISMAS OPERACIONES PARA TEXTO O BINARIO,
Y PARA ACCESO SECUENCIAL O ALEATORIO

LAS 5 OPERACIONES COMUNES

1. APERTURA: AL CREAR EL OBJETO DE E/S
2. LECTURA (read): TRAE DATOS AL BUFFER Y AVANZA EL PUNTERO
3. SALTO (skip): ADELANTA EL PUNTERO N BYTES O CARACTERES
4. ESCRITURA (write): ENVÍA DATOS DEL BUFFER AL FICHERO Y AVANZA EL PUNTERO
5. CIERRE (close): LIBERA RECURSOS

PUNTERO

MARCA LA POSICIÓN ACTUAL EN EL FICHERO (O EL EOF, JUSTO DETRÁS DEL ÚLTIMO BYTE)



BUFFER

ZONA DE MEMORIA QUE MUEVE DATOS ENTRE EL PROGRAMA Y EL FICHERO

EOF

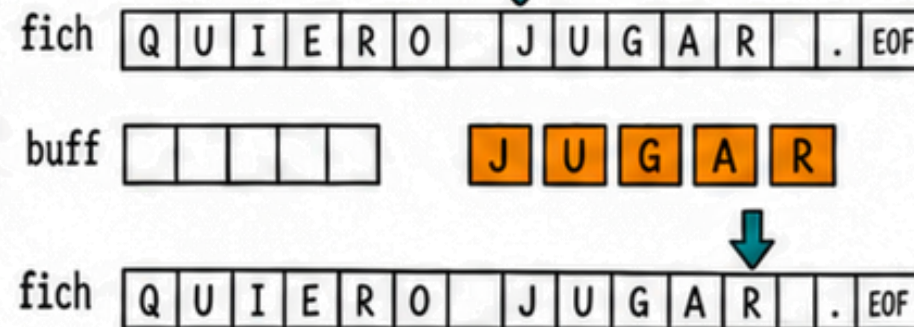
EOF

FIN DE FICHERO (END OF FILE)

LECTURA

- SI NO SE INDICA CUÁNTO LEER, SE LEE HASTA LLENAR EL BUFFER
- SI EL PUNTERO ESTÁ EN EOF, NO SE LEE NADA
- TRAS LEER, EL PUNTERO QUEDA DETRÁS DEL ÚLTIMO DATO
- TEXTO POR LÍNEAS: `readLine()` DE UN `BufferedReader` SOBRE UN `FileReader`

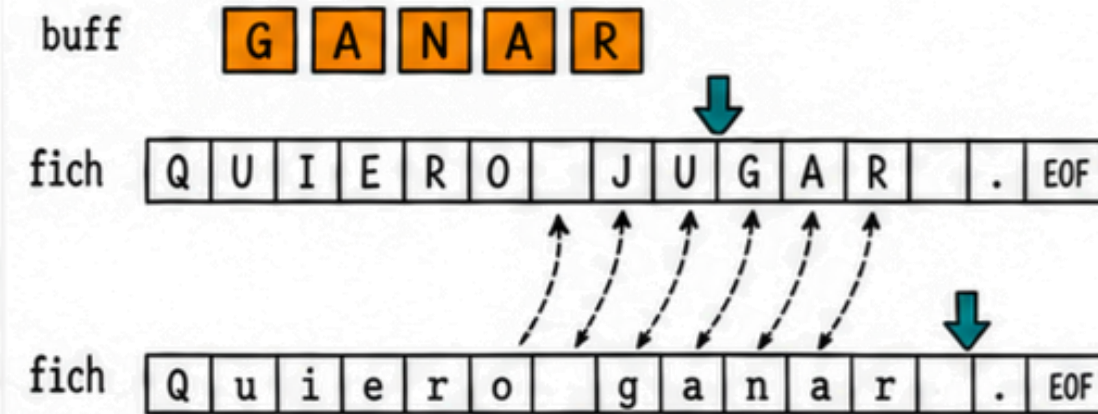
`read(buff, 0, 5)`



ESCRITURA

- LOS DATOS DEL BUFFER SE VUELCAN EN LA POSICIÓN DEL PUNTERO
- SI EL PUNTERO ESTÁ AL FINAL, LA ESCRITURA AÑADE (append)

`write(buff, 0, 5)`



LÍMITES DE LA ESCRITURA

- NO SE PUEDE INSERTAR EN MEDIO NI BORRAR UN FRAGMENTO
- SALVO QUE EL REEMPLAZO TENGA LA MISMA LONGITUD
- PARA ELLO SE USAN FICHEROS AUXILIARES



TODO GIRA EN TORNO A DOS IDEAS: EL PUNTERO Y EL BUFFER

4. FORMAS DE ACCESO A UN FICHERO

UN PUNTERO QUE AVANZA O QUE SALTA

4.1. ACCESO SECUENCIAL (FLUJOS DE DATOS)

DEL PRINCIPIO AL FINAL, EN ORDEN

CLASES DE FLUJOS

BYTES: InputStream / OutputStream
TEXTO: Reader / Writer
FileReader · FileWriter



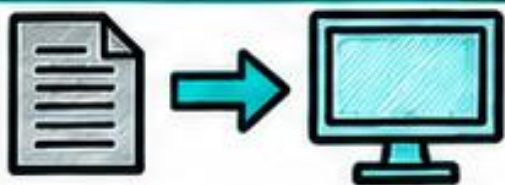
BUFFERING

BufferedReader · BufferedWriter
readLine() · newLine()



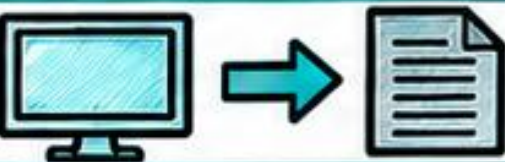
LECTURA

read()
DEVUELVE -1 EN EOF



ESCRITURA

write() · append()



RECODIFICACIÓN

InputStreamReader · OutputStreamWriter
UTF-8 / UTF-16



TUBERÍAS (PIPES)

PipedInputStream · PipedOutputStream



4.2. ACCESO ALEATORIO

SALTO DIRECTO A CUALQUIER POSICIÓN

LA CLASE RandomAccessFile

MODOS: "r" / "rw"

seek() · getFilePointer() · length()



REGISTROS DE LONGITUD FIJA

POSICIÓN = $i * \text{TAMAÑO}$



SECUENCIAL O ALEATORIO

SECUENCIAL

RECORRE TODO LO ANTERIOR
IDEAL PARA TEXTO Y LOGS



ALEATORIO

SALTA A LA POSICIÓN EXACTA
IDEAL PARA REGISTROS IGUALES



4.3. APLICACIONES REALES

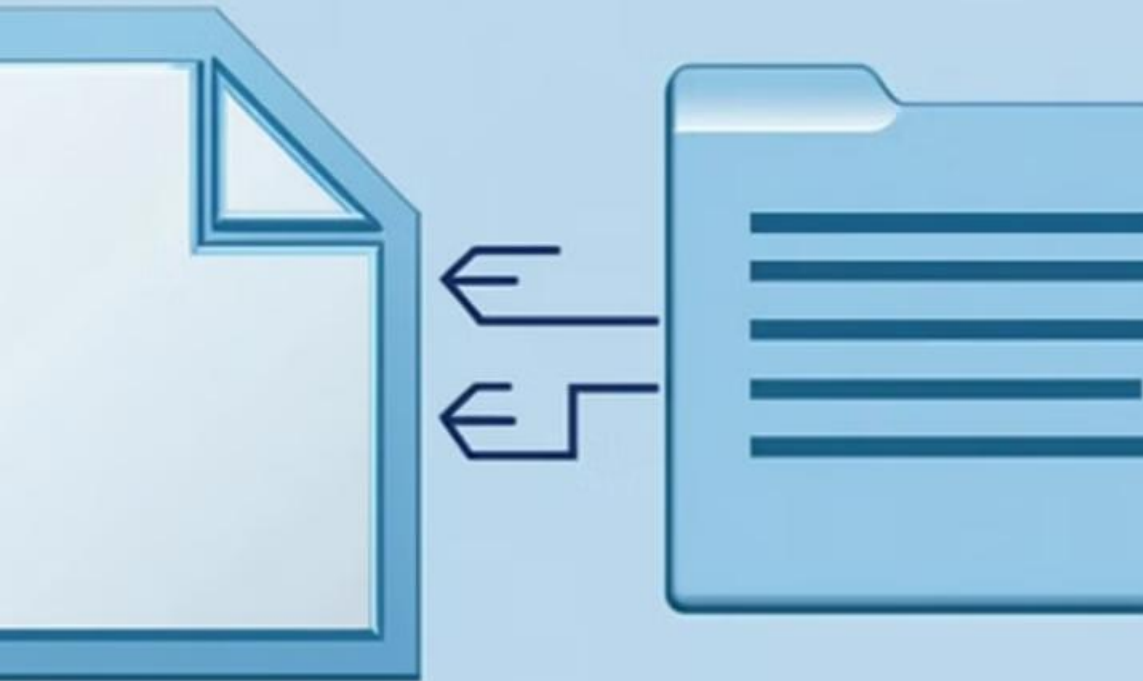


LOGS · BASES DE DATOS SENCILLAS · FICHEROS MULTIMEDIA

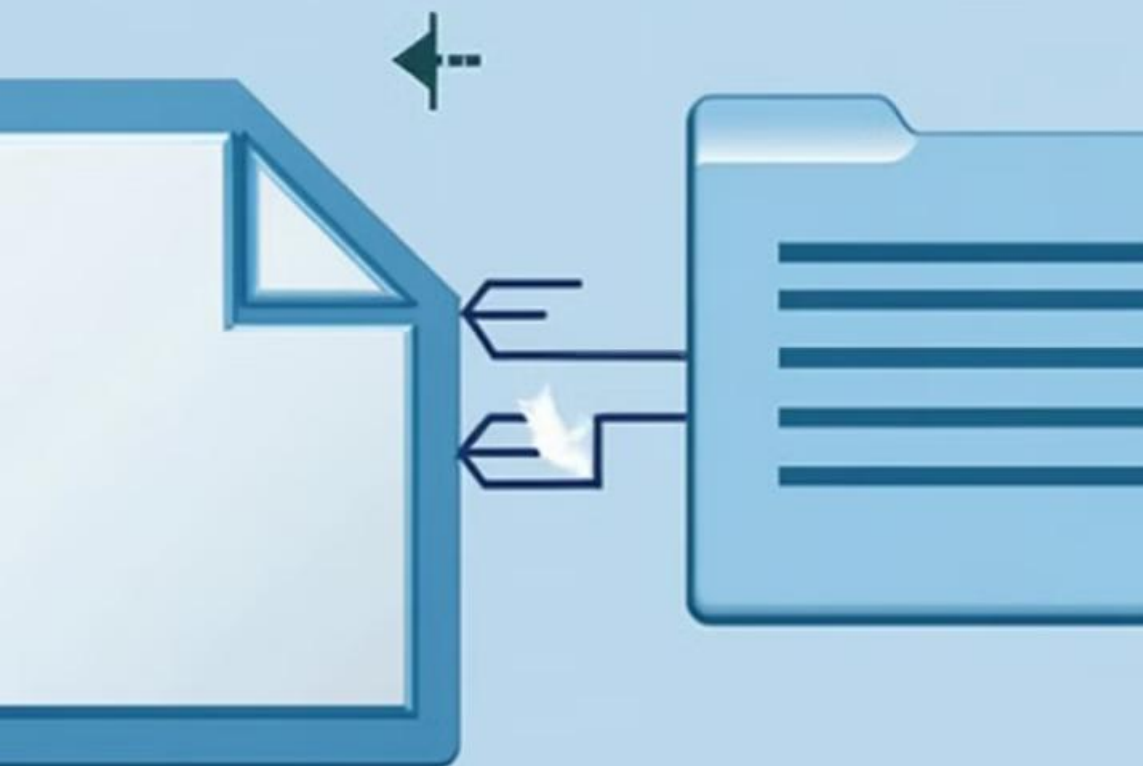


NO CONFUNDIR: BINARIO O TEXTO
ES EL CONTENIDO, NO LA FORMA
DE ACCESO

Sequential File



Random File Access



4. Formas de acceso a ficheros y flujos de datos

En el manejo de ficheros, es fundamental entender los diferentes tipos de acceso que podemos utilizar según el formato y la estructura de los datos almacenados.

En Java, las clases del paquete `java.io` nos permiten trabajar tanto con ficheros de **acceso secuencial** como de **acceso aleatorio**. Esta distinción es clave para optimizar la eficiencia y la adecuación de la solución a las necesidades de la aplicación.

Formas de acceso a ficheros

Acceso
secuencial

Acceso
aleatorio

Otros



4.1. Acceso secuencial (flujos de datos)

1

Definición de 'Acceso secuencial'

Forma básica de trabajar con ficheros. Los datos se almacenan y acceden secuencialmente, es decir, para llegar a un dato específico, es necesario leer todos los datos anteriores.

2

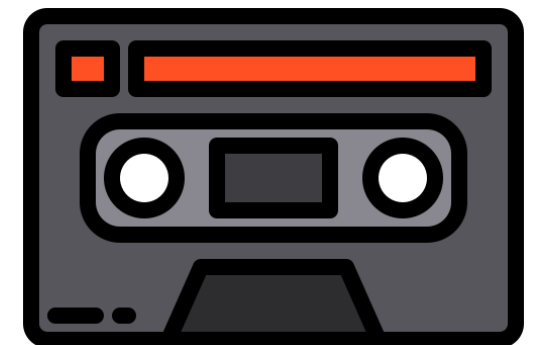
Ejemplo

Similar a una cinta de música. Para llegar a un dato, se deben leer todos los anteriores.

3

Uso común

Ficheros de texto donde los caracteres se almacenan uno tras otro.



Ficheros de texto secuenciales

1 Formato

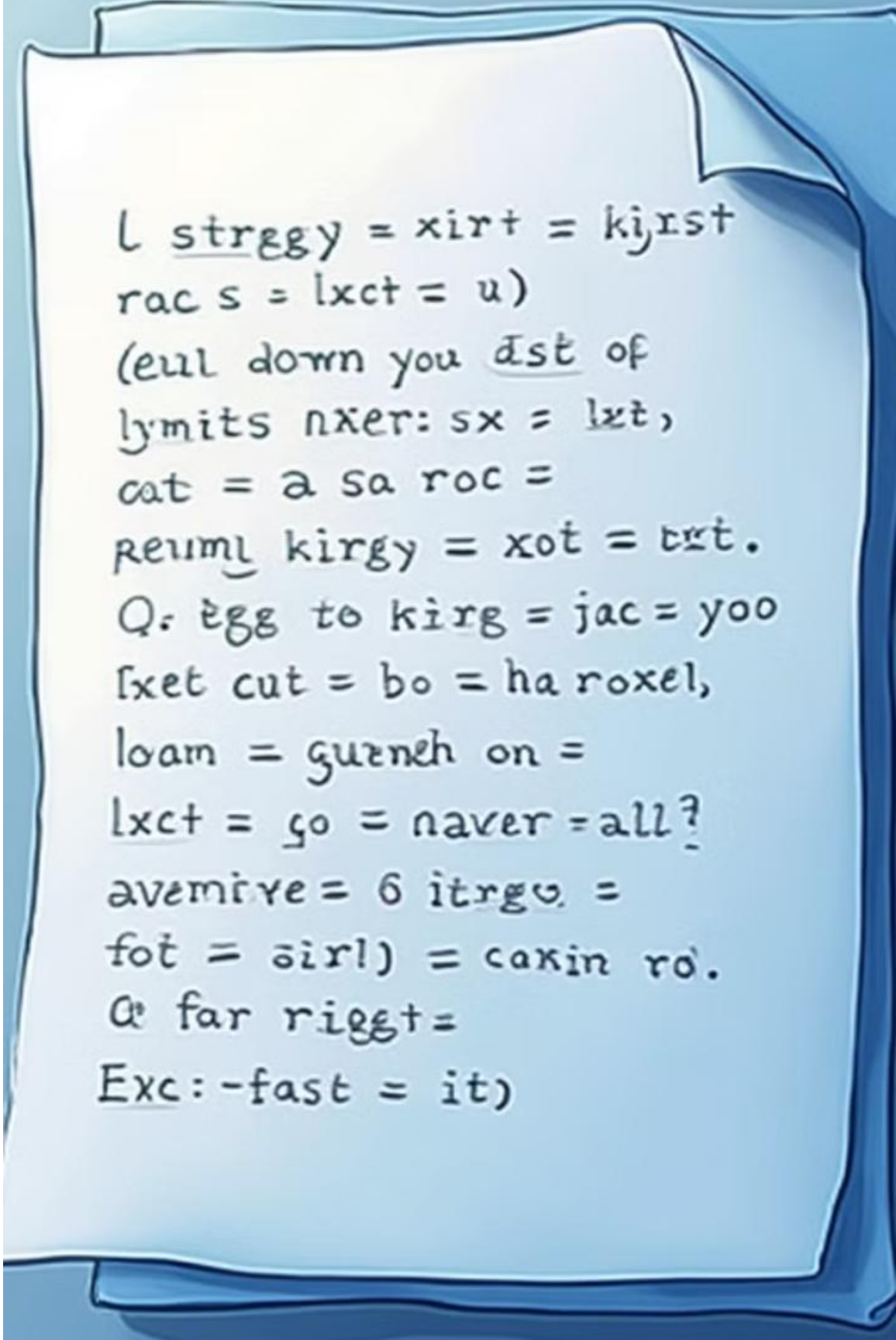
Datos almacenados en texto legible, como secuencia de caracteres.

2 Acceso

Para leer o escribir, el programa comienza desde el principio del fichero.

3 Adición de datos

Posible añadir datos al final de un fichero secuencial, datos al final de un fichero secuencial, aunque esto implica recorrer el fichero hasta el final, lo que puede ser ineficiente en ficheros grandes.



l straggy = xirt = kirst
rac s = lxct = u)
(eul down you ast of
lymits nxer: sx = lxt,
cat = a sa roc =
reumy kirgy = xot = txt.
Q: egg to kirg = jac = yoo
[xet cut = bo = ha roxel,
loam = guzneh on =
lxct = go = naver = all?
avemive = 6 itrgo =
fot = sirl) = caxin ro.
Q: far riggt =
Exc: -fast = it)

4.1. Acceso secuencial Flujos de datos (STREAM)

Describiremos los flujos de datos en Java. Aprenderemos cómo Java maneja las operaciones de entrada/salida (E/S) de manera eficiente y versátil, utilizando un sistema independiente del dispositivo.



Flujos de datos (STREAM)

Las operaciones con ficheros secuenciales en Java se realizan utilizando **flujos** (**streams**, en inglés) mediante clases del paquete java.io.

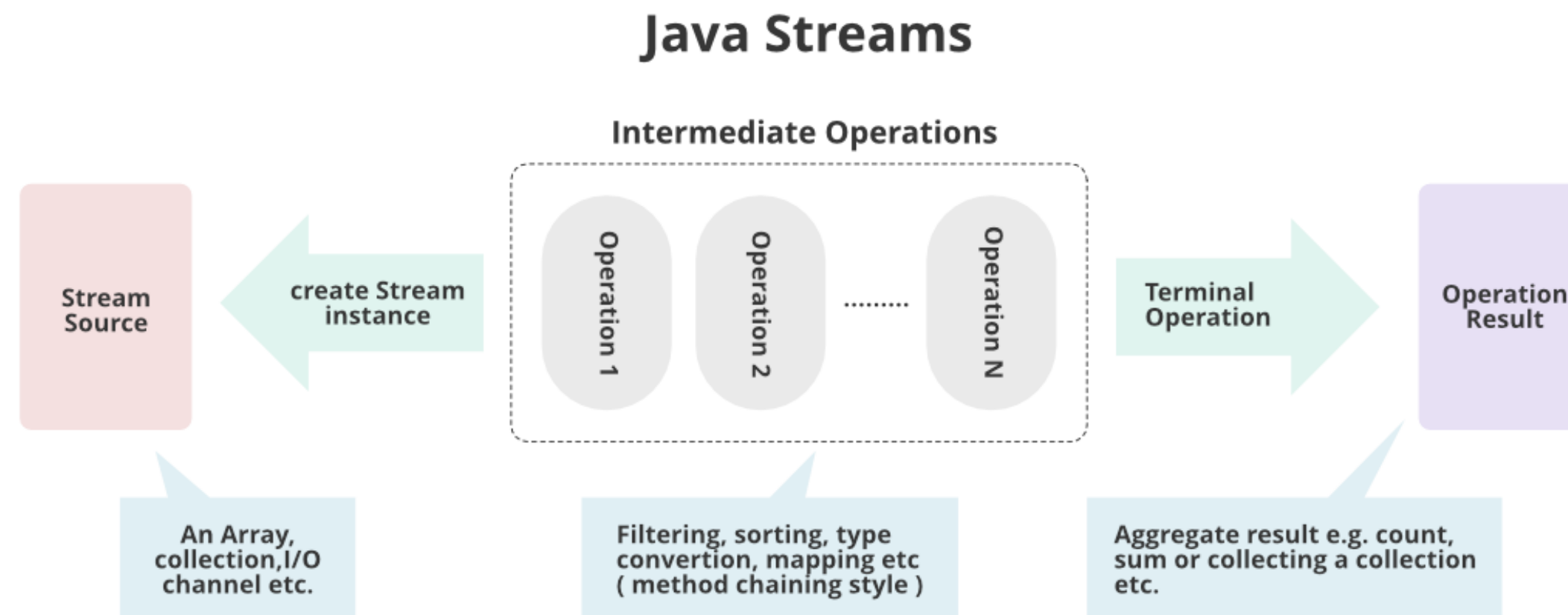
¿Qué es un flujo?

Un flujo es una abstracción de alto nivel para cualquier secuencia de datos.

Tipos de flujos

Los ficheros secuenciales son flujos, lo mismo que los datos enviados a través de una conexión de red, o la información contenida en una secuencia de posiciones consecutivas de memoria, o la entrada y la salida estándar y de error de un programa en ejecución.

Este paquete saca partido de la herencia, y proporciona clases con funcionalidades.



4.1.1 Clases relacionadas con flujos de datos

Para leer de un fichero o escribir en un fichero hay que **crear un flujo** (stream) asociado al mismo.

El tipo de flujo para crear será distinto según se trate de un **fichero binario o de texto**.

Flujos binarios (Byte streams)

No plantean mayor problema: se leen bytes de un flujo binario hacia una variable de tipo `byte[]`, o se escriben bytes de una variable de tipo `byte[]` a un flujo binario.

Byte streams (flujos de bytes)

Se utilizan para la lectura y escritura de datos binarios. Las clases principales que gestionan flujos de bytes son **InputStream** y **OutputStream**, junto con sus diversas subclases.

Flujos de texto (Character streams)

Más complejo por las codificaciones de texto.

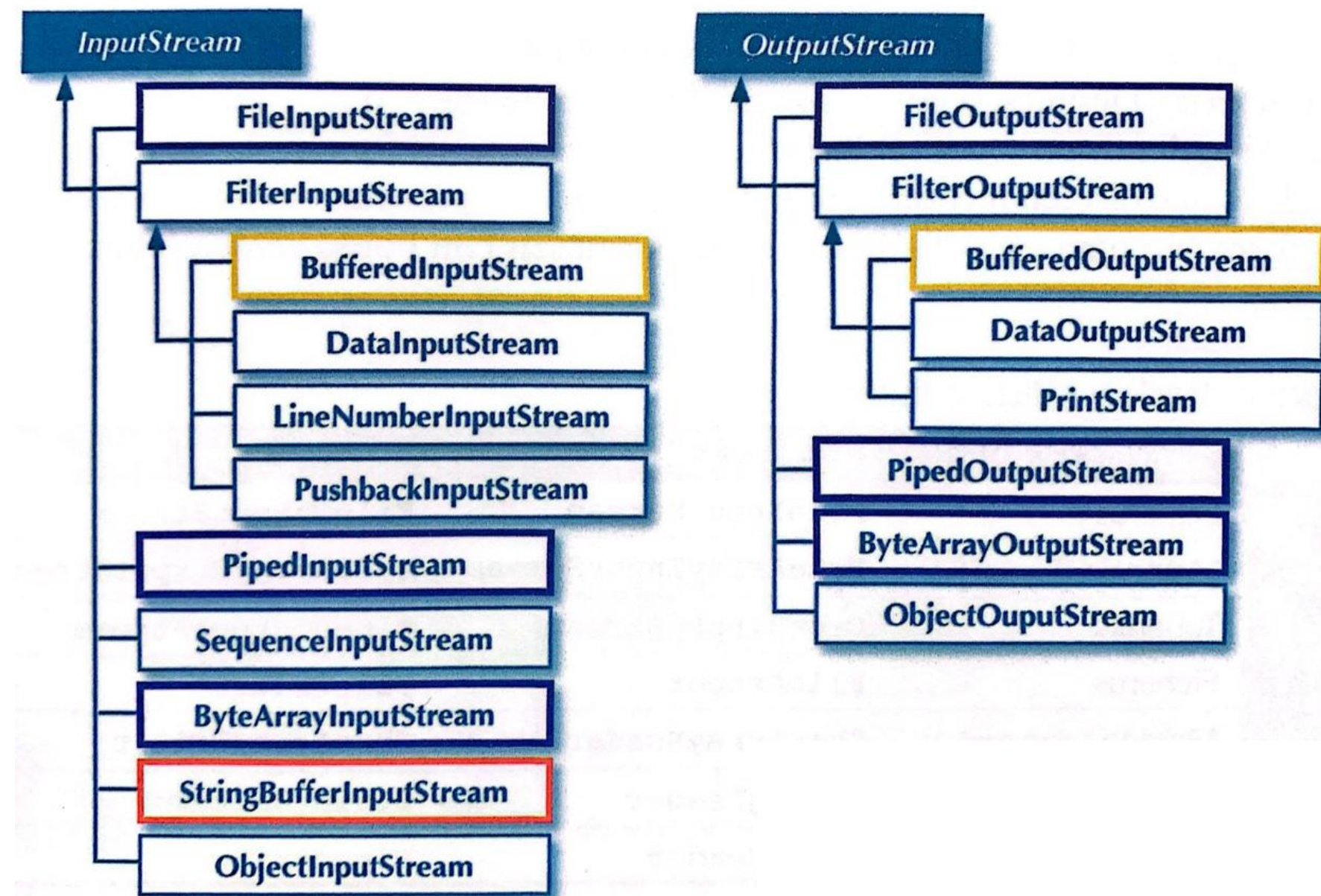
Todo funcionará bien mientras se trabaje con ficheros de texto con la codificación que Java asume por defecto. La más habitual **UTF-8**.

Character streams (flujos de caracteres)

Están orientados a la lectura y escritura de datos de texto. Los flujos de caracteres se gestionan a través de las clases abstractas **Reader** y **Writer**.

4.1.1 Clases relacionadas con flujos de datos

Jerarquía de clases para flujos binarios



4.1.1 Clases relacionadas con flujos de datos

Flujos binarios

Propósito y Uso

Los flujos de bytes en Java se utilizan para manejar datos binarios. Son esenciales para trabajar con archivos que no contienen texto plano, como imágenes, archivos de audio o cualquier otro tipo de datos que no se pueda representar directamente como texto.

Clases Principales

Las clases base para los flujos de bytes son **InputStream** y **OutputStream**.

- **InputStream**: se utiliza para leer datos de una Fuente.
- **OutputStream**: se utiliza para escribir datos a un destino.

Existen numerosas subclases que proporcionan funcionalidades específicas para diferentes tipos de operaciones.

4.1.1 Clases relacionadas con flujos de datos

Clases clave de Flujos binarios

FileOutputStream

Escribe bytes a un archivo. Incluye el modo de anexión para agregar datos al final de un archivo existente.

ObjectOutputStream

Serializa objetos en flujos de bytes. Esto permite guardar objetos complejos en archivos binarios.

FileInputStream

Lee bytes desde un archivo binario. Es fundamental para leer datos no textuales.

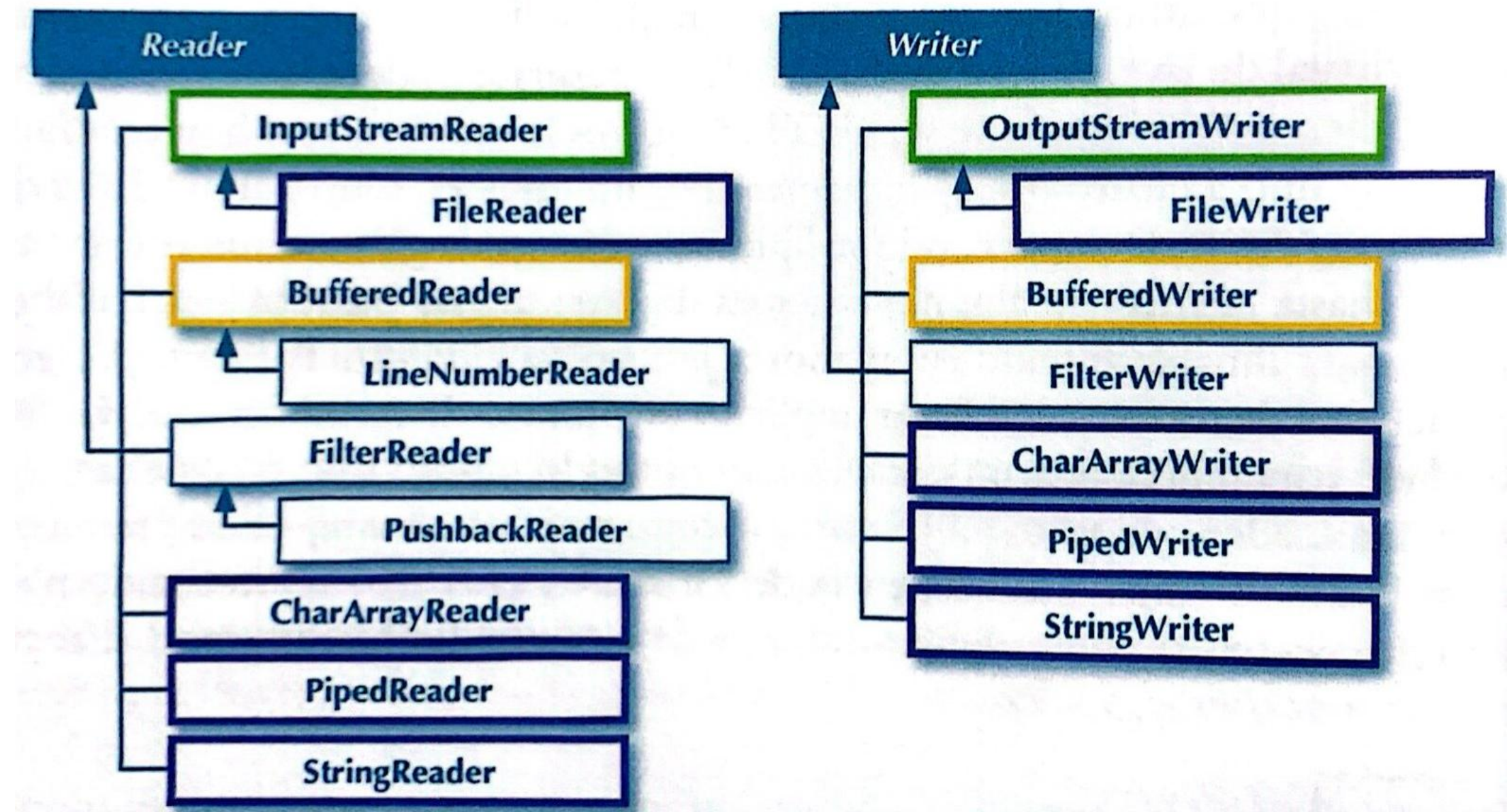
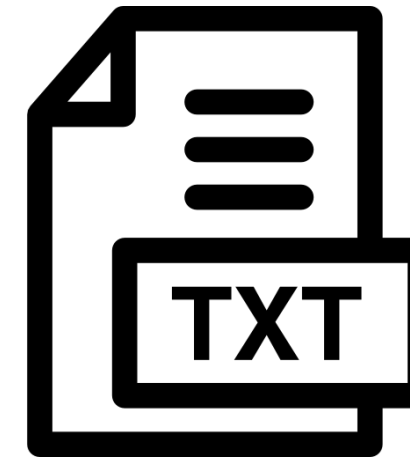
ObjectInputStream

Deserializa bytes de vuelta a objetos. Recupera los objetos complejos guardados previamente.



4.1.1 Clases relacionadas con flujos de datos

Jerarquía de clases para flujos de texto





4.1.1 Clases relacionadas con flujos de datos

Flujos basados en texto

■ Propósito y Uso

Los flujos de caracteres en Java se utilizan para manejar datos de texto. Son ideales para trabajar con archivos de texto plano, donde cada unidad de datos es un carácter Unicode.

■ Manejo de Unicode

Una característica clave de los flujos de caracteres es su capacidad para manejar texto Unicode. Esto permite trabajar con una amplia gama de caracteres de diferentes idiomas y alfabetos.

■ Clases Principales

Las clases base para los flujos de caracteres son `Reader` y `Writer`. `Reader` se utiliza para leer caracteres de una fuente, mientras que `Writer` se utiliza para escribir caracteres a un destino.

4.1.1 Clases relacionadas con flujos de datos

Clases clave de Flujos de texto

FileWriter

Escribe caracteres a un archivo. Ideal para trabajar con archivos de texto.

FileReader

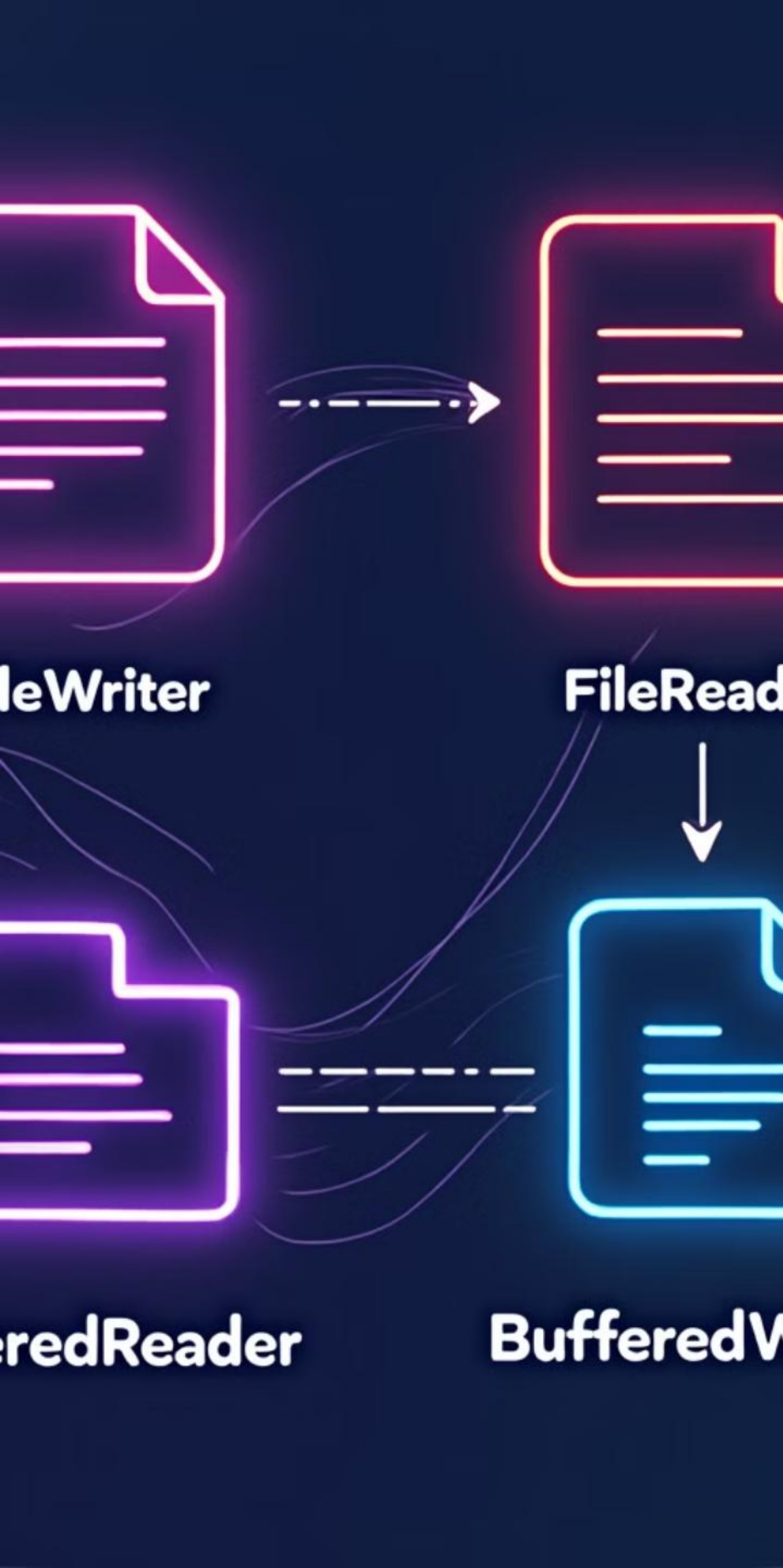
Lee caracteres desde un archivo. Permite leer datos de texto de manera eficiente.

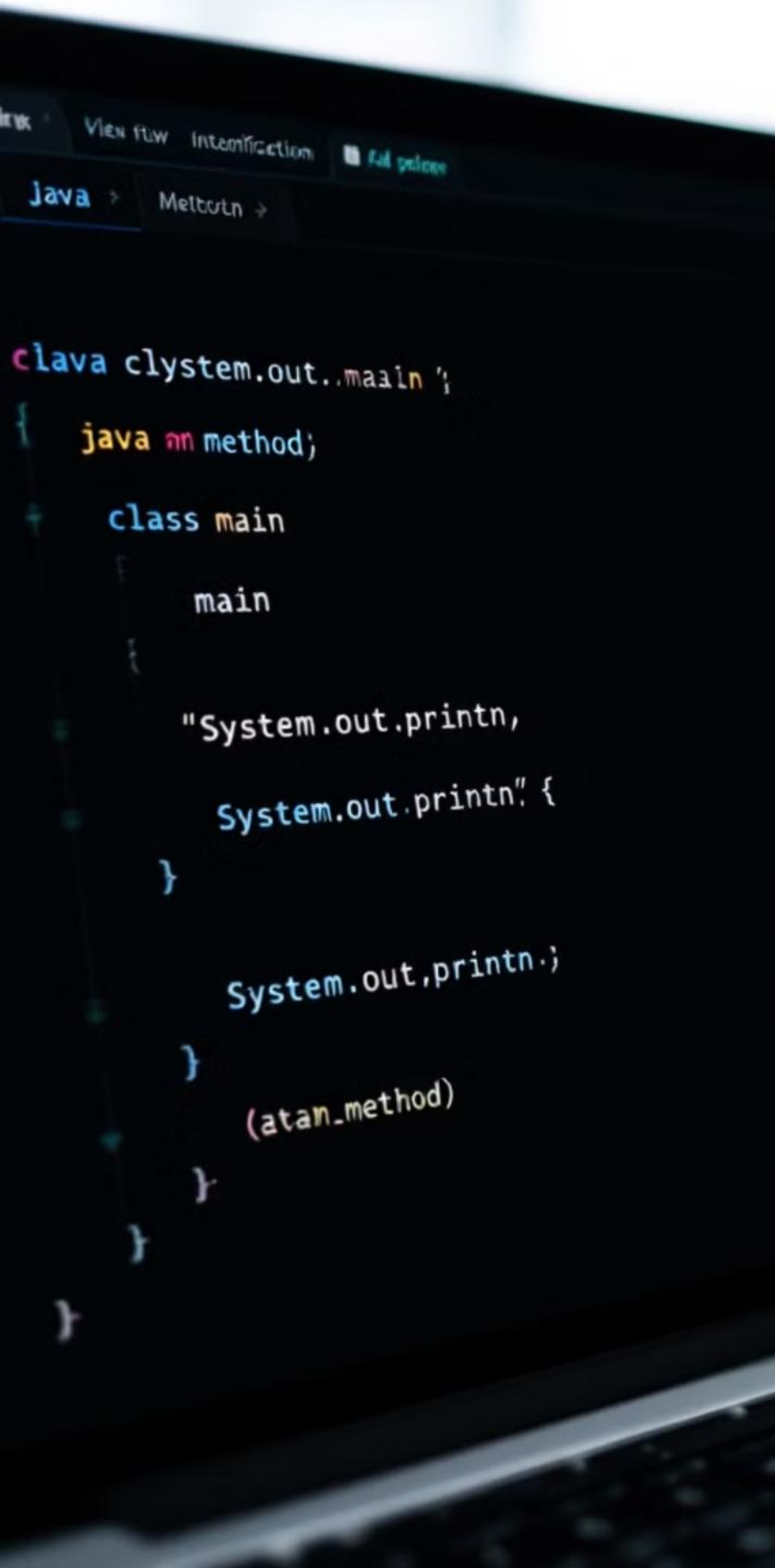
BufferedReader

Mejora el rendimiento de la lectura mediante el almacenamiento en búfer. Reduce el número de accesos al disco.

BufferedWriter

Mejora el rendimiento de la escritura mediante el almacenamiento en búfer. Optimiza la escritura a disco.





Resumen de funcionalidades de clases relacionadas con flujos de acceso secuencial

Funcionalidades de las jerarquías de clase en Java:

Lectura y escritura

Funcionalidad básica en flujos asociados a distintos tipos de fuentes de datos.

Recodificación de texto

Java codifica el texto en memoria en UTF-16, en variables de tipo String o char []. La mayoría de ficheros está codificado en **UTF-8**, y normalmente utiliza esta codificación por defecto para leer y escribir en ellos. Por lo tanto, las operaciones de lectura y escritura de texto en ficheros de texto lleva implícito, generalmente, un **proceso de recodificación**.

Buffering

Permite aumentar la velocidad de las operaciones de lectura y escritura, y hace posible leer y escribir líneas de texto para los ficheros de texto.

Resumen de nomenclatura para clases relacionadas con flujos de datos de acceso secuencial

Java usa una nomenclatura sistemática para las clases que ofrecen la funcionalidad básica de lectura y escritura.

Tipo de fuente

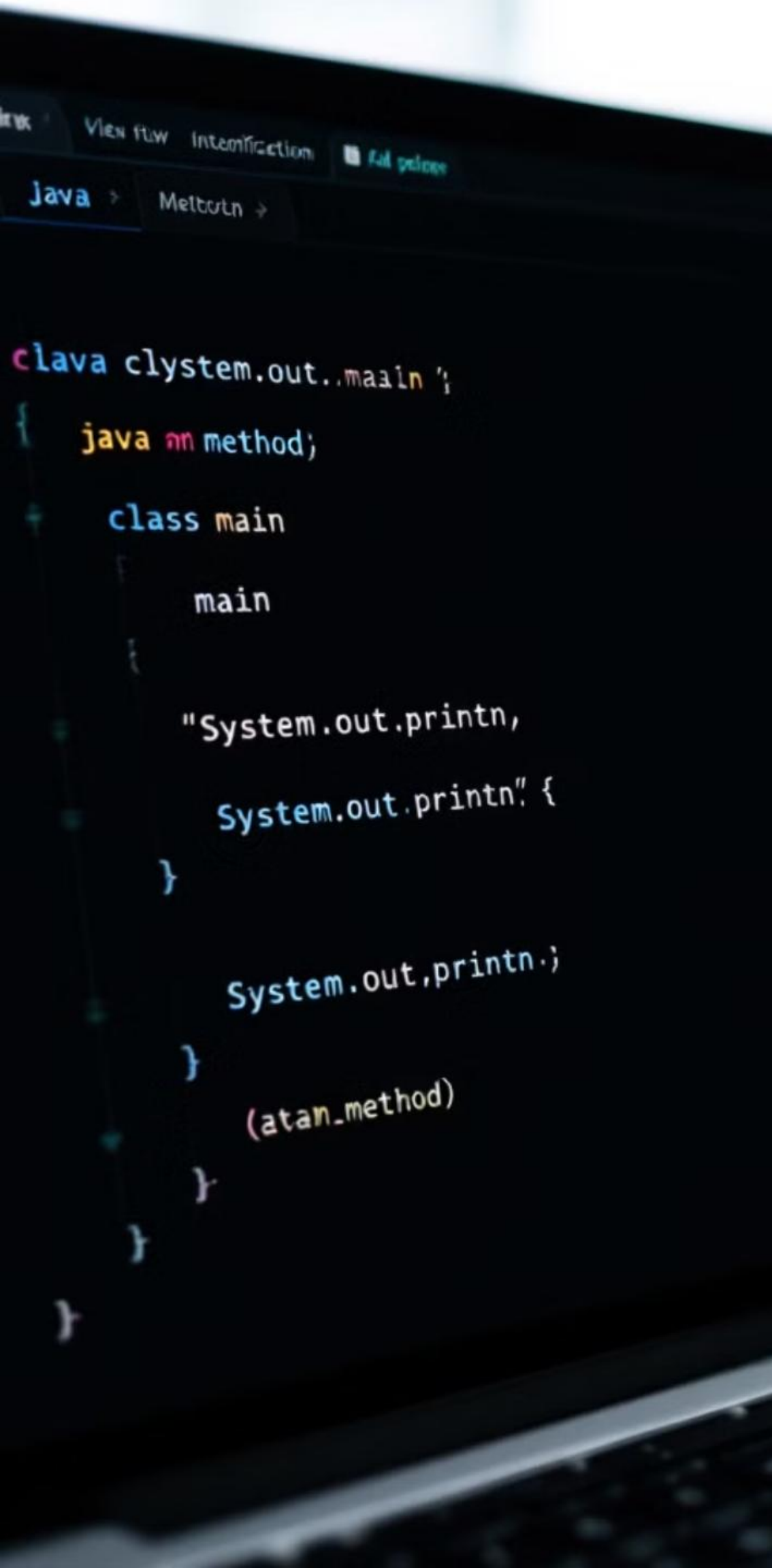
El nombre indica primero el tipo de fuente (**File** para fichero, **ByteArray** para memoria, **Piped** para tubería).

Entrada o salida

Si es de entrada (**Input o Reader**) o de salida (**Output o Writer**).

Flujo de bytes o texto

Si acaba con **Stream** es para flujos de bytes, y si no, para flujos de texto.



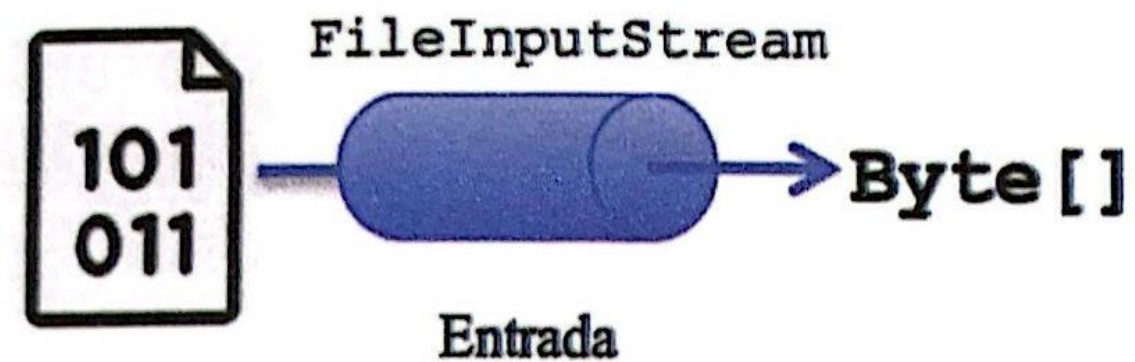


Clases básicas para entrada y salida de flujos

	Fuente de datos	Lectura	Escritura
Flujo binario	Ficheros	FileInputStream	FileOutputStream
	Memoria (byte[])	ByteArrayInputStream	ByteArrayOutputStream
	Tuberías	PipedInputStream	PipedOutputStream
Flujo de texto	Ficheros	FileReader	FileWriter
	Memoria (char[])	CharArrayReader	CharArrayWriter
	Memoria (String)	StringReader	StringWriter
	Tuberías	PipedReader	PipedWriter

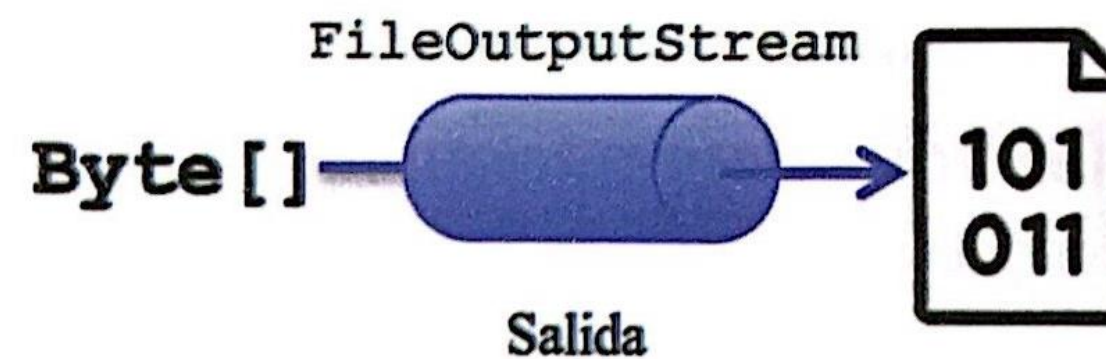
Ejemplos de creación de objetos de estas clases

Lectura fichero binario



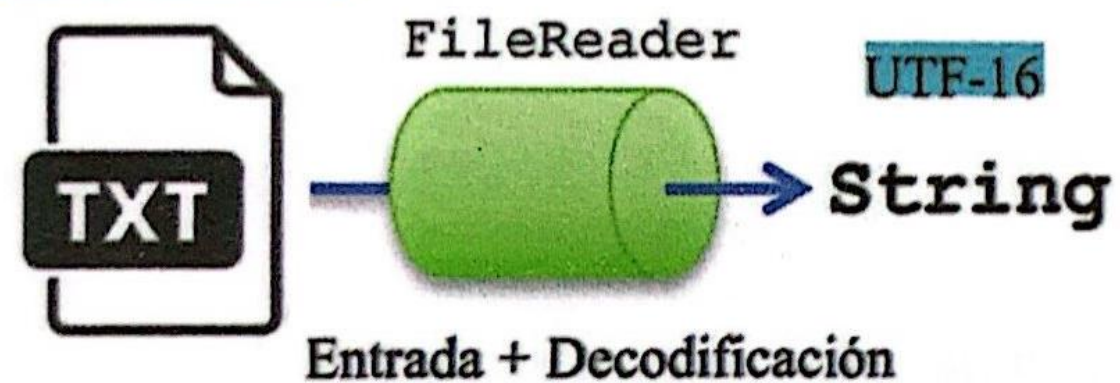
```
FileInputStream fis =  
new FileInputStream("f.bin ");
```

Escritura fichero binario



```
FileOutputStream fos =  
new FileOutputStream("f.bin ");
```

Cod. por defecto)



```
FileReader fr=  
new FileReader("f.txt ");
```

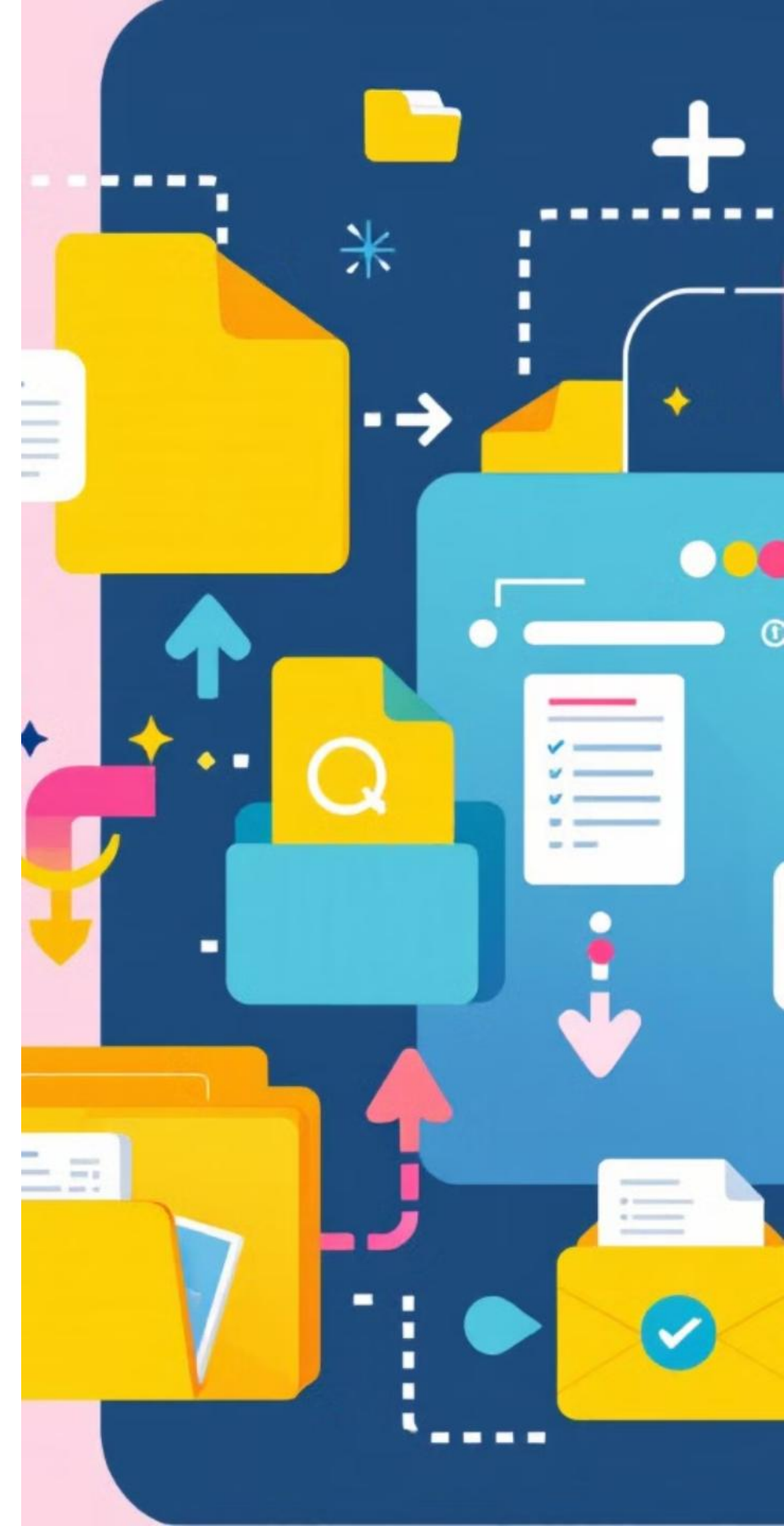
Lectura fichero texto

Cod. por defecto)



```
FileWriter fw2=  
new FileWriter("f.txt ");
```

Escritura fichero texto



4.1.2. Clases para buffering

¿Y cuándo se trata de textos muy largos?

BUFFERS

El acceso directo al disco puede ser lento si se leen o escriben pequeños fragmentos de datos de manera repetitiva.

Para optimizar el rendimiento, Java proporciona clases como **BufferedReader** y **BufferedWriter** para flujos de caracteres, y **BufferedInputStream** y **BufferedOutputStream** para flujos de bytes.

Estas clases almacenan temporalmente grandes cantidades de datos en memoria antes de realizar la operación en el disco, haciendo que los programas sean más eficientes.





4.1.2. Clases para buffering

Velocidad

Acelera las operaciones de lectura o escritura, utilizando una zona de intercambio en memoria llamada buffer.

Lectura y escritura de líneas

Permiten leer y escribir líneas de texto en ficheros de texto.

Recomendación

En general conviene utilizar las clases que proporcionan buffering. El rendimiento aumenta para ficheros grandes y cuando se realizan lecturas o escrituras consecutivas en posiciones contiguas, que es lo habitual.

	LECTURA	ESCRITURA
Flujo binario	BufferedInputStream	BufferedOutputStream
Flujo de texto	BufferedReader	BufferedWriter



4.1.2. Clases para buffering

Métodos para lectura y escritura de líneas en clases para buffering con ficheros de texto

Clase	Método	Funcionalidad
BufferedReader	String readLine()	Lee hasta el final de la línea actual.
BufferedWriter	void newLine()	Escribe un separador de líneas. El separador de líneas puede depender del sistema operativo, y suele ser distinto en Linux y en Windows. El método <code>readLine()</code> de <code>BufferedReader</code> tiene en cuenta estas particularidades.

4.1.3 Operaciones de lectura para flujos de entrada

Son similares para todas las clases que implementan flujos (streams) de entrada en Java.

- Las funciones **read** de clases que heredan de **InputStream** leen bytes.
- Las funciones read que heredan de **Reader** leen caracteres.

Función	Descripción
read()	Lee un byte o un carácter hasta llenar el buffer o el número de bytes o caracteres indicados, que se copiarán en la posición indicada (offset) dentro del buffer. Devuelven el número de bytes o caracteres leídos, o -1 si no se pudo leer nada porque el puntero estaba al final del fichero.
skip()	Saltan el número indicado de bytes o caracteres , aunque podrían ser menos si se alcanza el final del fichero. En cualquier caso, devuelven el número de bytes o de caracteres que se han saltado.
readLine()	Con ficheros de texto, para leer líneas se puede utilizar el método readLine() de un BufferedReader construido sobre un FileReader.

4.1.3 Operaciones de lectura para flujos de entrada

Métodos de lectura de las clases para la gestion de flujos de entrada

InputStream	Reader
<code>int read()</code> <code>int read(byte[] buffer)</code> <code>int read(byte[] buffer, int offset, int longitud)</code>	<code>int read()</code> <code>int read(char[] buffer)</code> <code>int read(char[] buffer, int offset, int longitud)</code> <code>int read(CharBuffer buffer)</code>
<code>long skip(long n)</code>	<code>long skip(long n)</code>
	BufferedReader
	<code>String readLine()</code>



4.1.4 Operaciones de escritura para flujos de salida

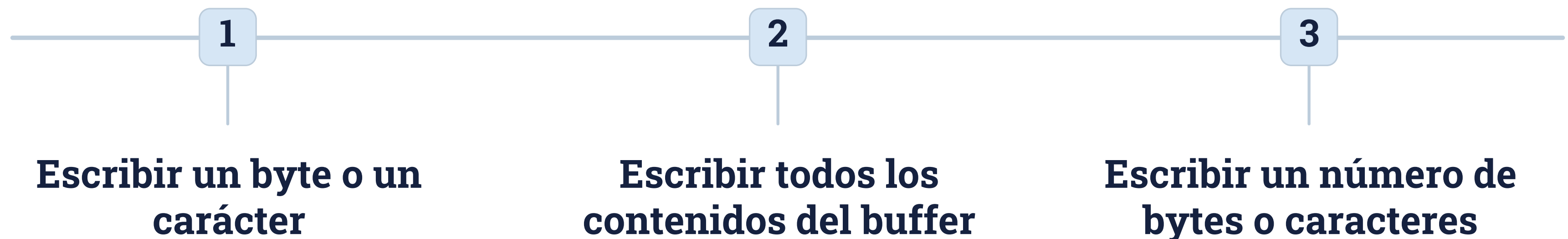
Son similares para todas las clases que implementan flujos (streams) de salida en Java.

Las funciones **write**:

- Clases que heredan de OutputStream escriben bytes.
- Clases que heredan de Writer escriben caracteres.

Según la variante, escriben un byte o un carácter, o todos los contenidos del buffer, o el número de bytes o caracteres indicados a partir de la posición indicada.

Si se alcanza el fin del fichero, siguen escribiendo. Las que heredan de Writer tienen métodos para escribir los contenidos de un String, y **métodos append** para añadir al final del fichero.



4.1.4 Operaciones de escritura para flujos de salida

Métodos para escritura de clases para gestión de flujos de salida

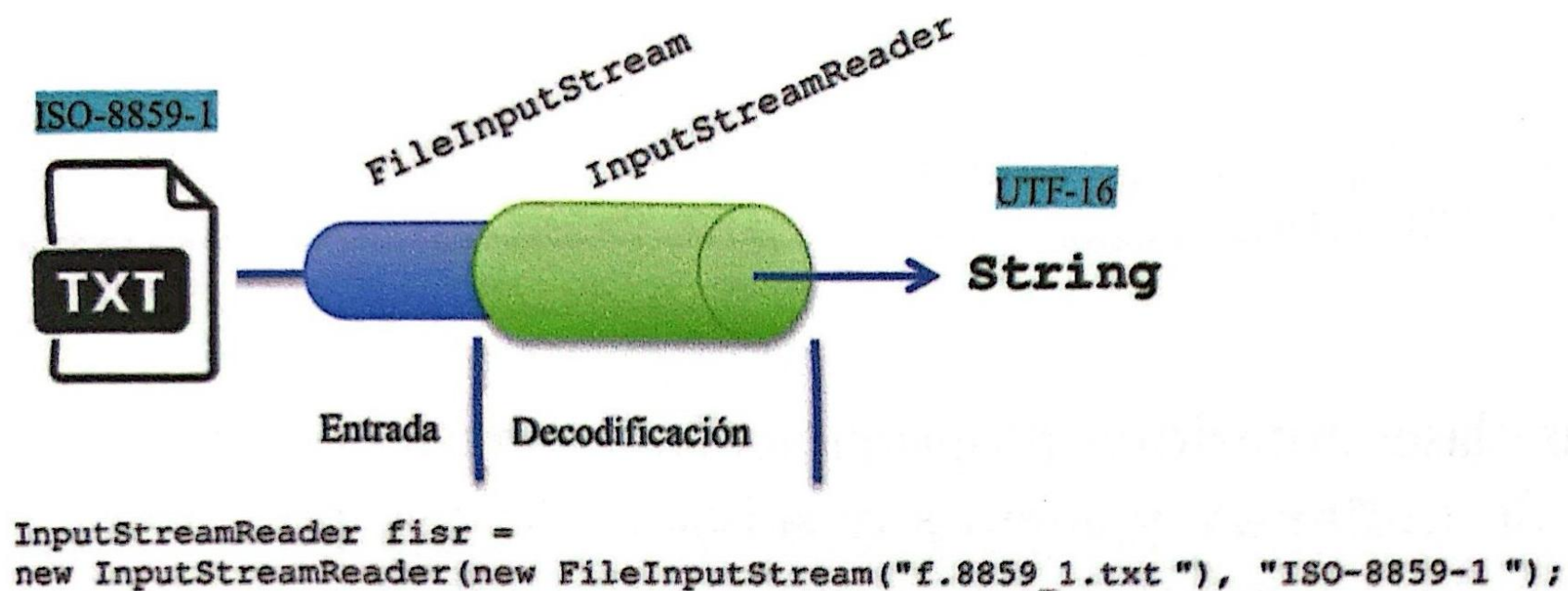
OutputStream	Writer
<code>void write(int b)</code> <code>void write(byte[] buffer)</code> <code>void write(byte[] buffer, int offset, int longitud)</code>	<code>void write(int c)</code> <code>void write(char[] buffer)</code> <code>void write(char[] buffer, int offset, int longitud)</code> <code>void write(String str)</code> <code>void write(String str, int offset, int longitud)</code>
	<code>Writer append(char c)</code> <code>Writer append(CharSequence csq)</code> <code>Writer append(CharSequence csq, int offset, int longitud)</code>
	BufferedWriter
	<code>void newLine()</code>



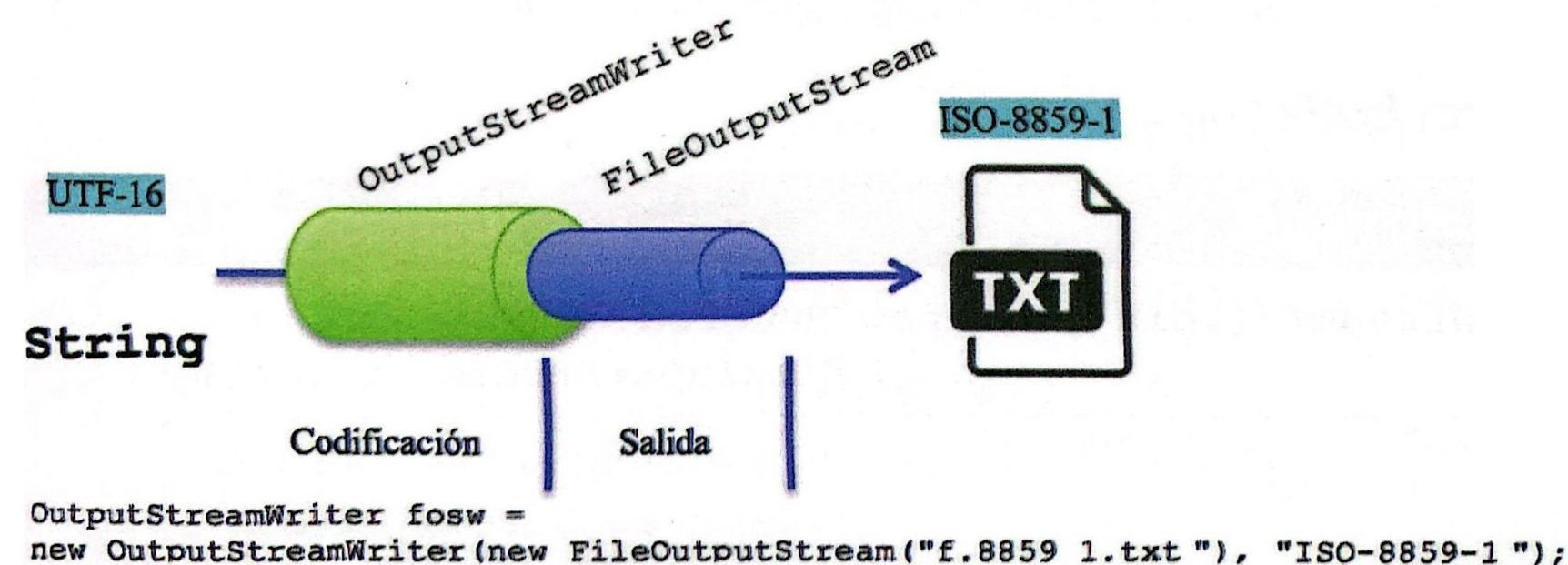
4.1.5. Clases para recodificación

Las clases **InputStreamReader** y **OutputStreamWriter** sirven de enlace entre ambas jerarquías: la de flujos binarios y la de flujos de texto.

- Convierten flujos binarios en flujos de texto y viceversa.
- Es necesario especificar una codificación a su constructor.



Lectura desde ficheros de texto con codificación distinta a la codificación por defecto



Escritura a ficheros de texto con codificación distinta a la codificación por defecto

ACTIVIDAD PRÁCTICA



Actividad 3 RA1_A3

Escritura y lectura de archivos

Escritura en Archivo

Lectura desde Archivo

Demuestra el uso básico de FileWriter para escribir datos en un archivo y FileReader para leerlos.

[Ver Actividad 3 en Aula Virtual]

ACTIVIDAD PRÁCTICA



Actividad 4 RA1_A4

Operaciones comunes sobre ficheros

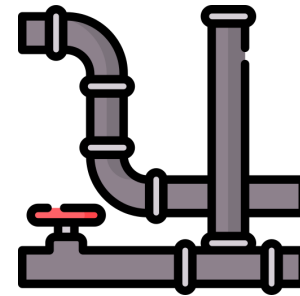
Realizar una serie de operaciones comunes sobre ficheros y directorios utilizando la clase File de Java. Estas operaciones incluyen la lectura de un fichero, la escritura en el fichero y la búsqueda de palabras determinadas en el fichero.

[Ver Actividad 4 en Aula Virtual]

4.1. Acceso secuencial

4.1.6. Tuberías o pipes

Conexión de programas

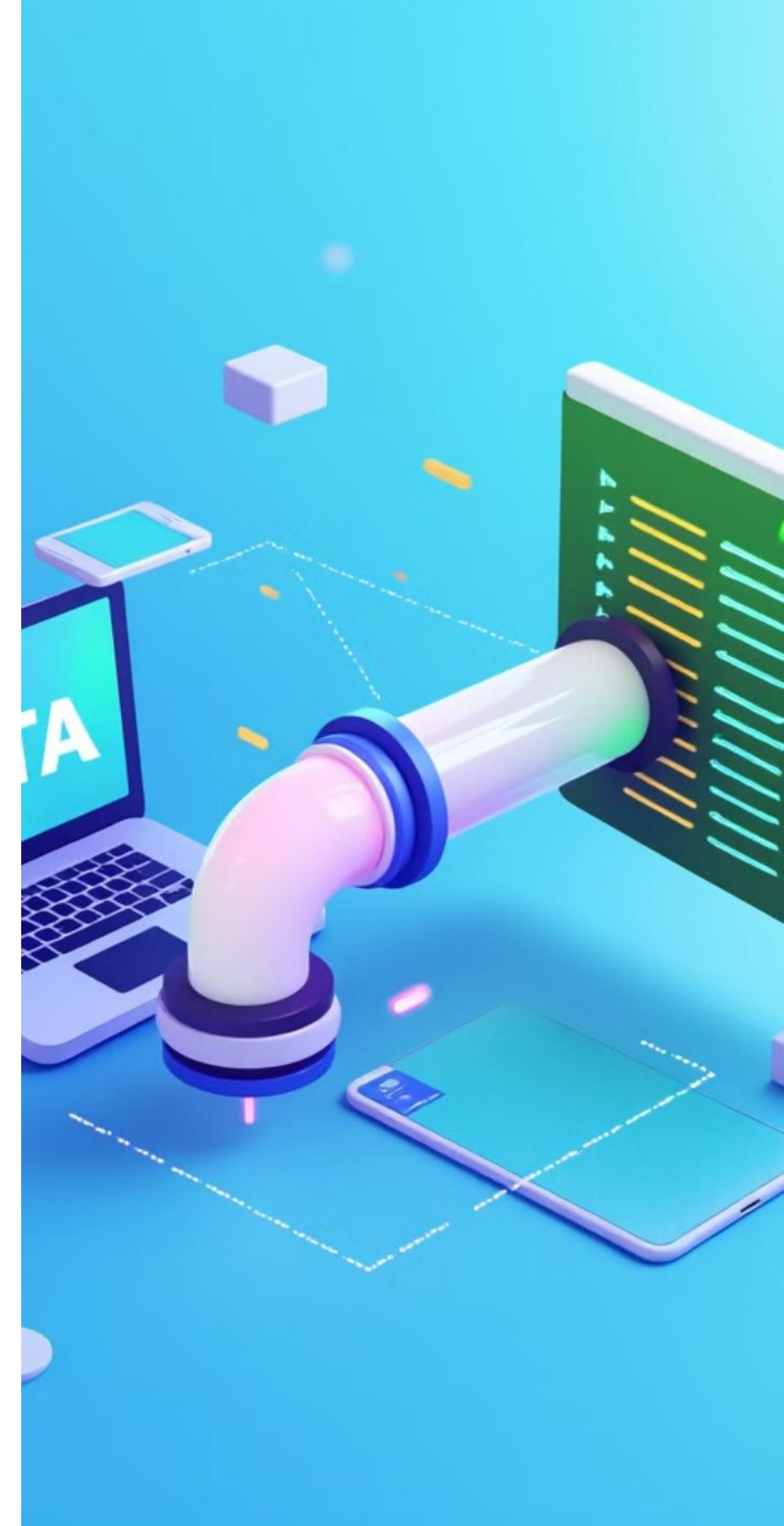


Otra forma avanzada de manejar el acceso a ficheros es mediante la concatenación o tuberías (pipes). Las tuberías son útiles para conectar programas que se ejecutan simultáneamente en la misma máquina, permitiendo que los datos producidos por un programa sean directamente consumidos por otro.

Implementación en Java



Aunque esta técnica es más común en sistemas operativos tipo Unix, Java permite trabajar con tuberías a través de clases como **PipedInputStream** y **PipedOutputStream**.



4.1.6. Tuberías o pipes

Intercambio de datos en tiempo real

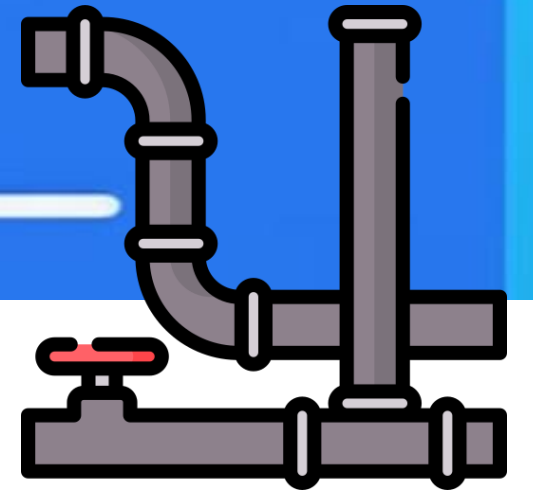
Las tuberías se utilizan en escenarios donde dos procesos independientes necesitan intercambiar datos en tiempo real, y el uso de ficheros intermedios no es necesario ni eficiente.

Las tuberías (pipes) son flujos, es decir, pertenecen al mundo secuencial. Encajan en la misma idea que `FileInputStream` o `FileReader`. La diferencia está en **de dónde salen y a dónde van los datos**, no en cómo se accede a ellos.

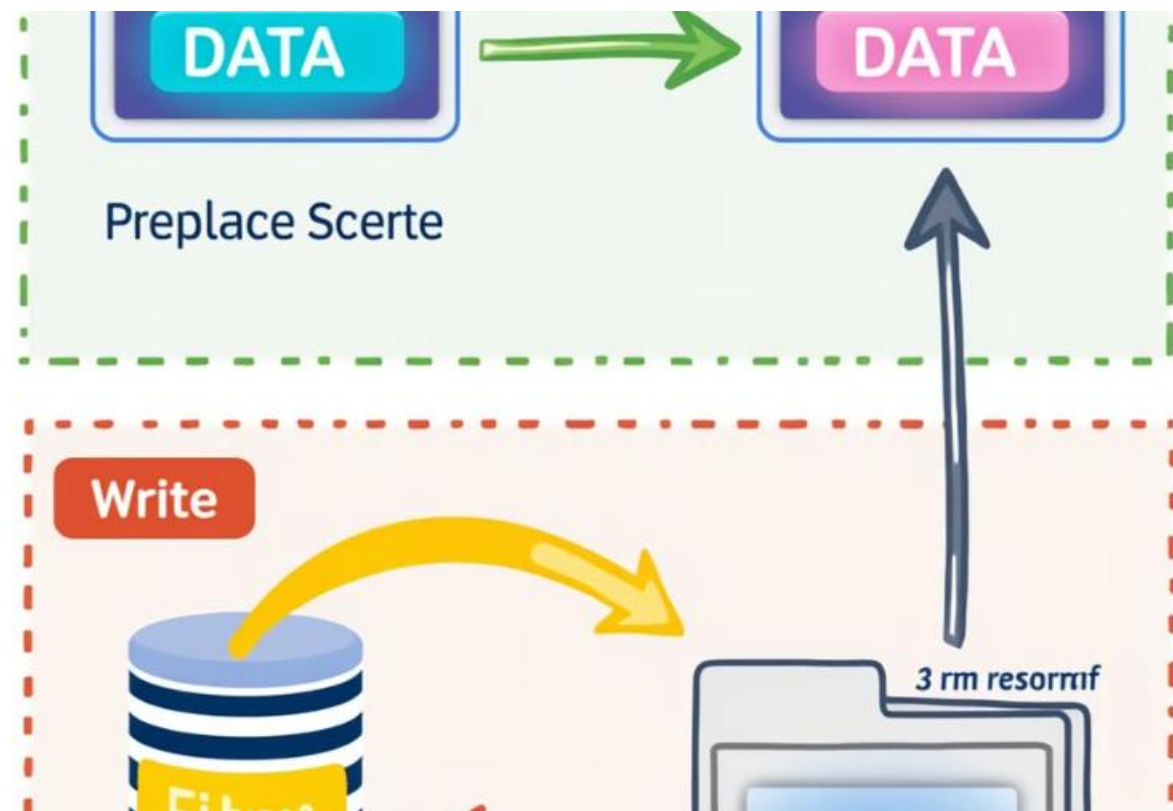
Un **fichero** es como escribir una carta y guardarla para leerla luego.

Una **tubería** es como un tubo neumático entre dos despachos: uno mete el mensaje por un lado y sale por el otro, en directo.

En ambos casos los datos van **en orden** (por eso son flujos), pero el fichero *persiste* en disco y la tubería *no*: es solo un canal de paso entre dos hilos.



Ejemplos reales de Flujos de datos



Sistemas de Bases de Datos Simples (basados en archivos)

Los flujos son cruciales para la gestión de datos en sistemas de bases de datos simples que almacenan información en archivos. Permiten leer y escribir datos de forma eficiente, facilitando la persistencia de la información.



Aplicaciones Multimedia (reproductores de música y vídeo)

En reproductores multimedia, los flujos con búfer son esenciales para minimizar la latencia. El búfer permite una reproducción fluida, evitando interrupciones causadas por la espera de datos desde el disco o la red.

FLUJOS DE DATOS (ACCESO SECUENCIAL)

UN FLUJO (STREAM) ES UNA ABSTRACCIÓN PARA CUALQUIER SECUENCIA DE DATOS: FICHEROS, RED, MEMORIA, ENTRADA/SALIDA ESTÁNDAR

LAS 4 COMBINACIONES POSIBLES

BINARIO (BYTES)

TEXTO (CARACTERES)

BINARIO + ENTRADA

InputStream
FileInputStream

101010

TEXTO + ENTRADA

Reader
FileReader

ABC

BINARIO + SALIDA

OutputStream
FileOutputStream

101010

TEXTO + SALIDA

Writer
FileWriter

ABC

PUENTE ENTRE BINARIO Y TEXTO

101010

ABC

UTF-8

UTF-16

InputStreamReader Y OutputStreamWriter
CONVIERTEN BYTES ↔ CARACTERES

HAY QUE INDICARLES LA CODIFICACIÓN

TRES FUNCIONALIDADES CLAVE



LECTURA Y ESCRITURA:
FUNCIÓN BÁSICA DEL FLUJO



RECODIFICACIÓN: EL TEXTO
SE GUARDA EN MEMORIA EN
UTF-16 Y EN EL FICHERO
SUELE ESTAR EN UTF-8



BUFFERING: ACELERA LA E/S
Y PERMITE TRABAJAR
POR LÍNEAS

ENTRADA
(LECTURA)



SALIDA
(ESCRITURA)



CON BUFFER (RECOMENDADO)



BufferedReader Y BufferedWriter

readLine() LEE UNA LÍNEA

newLine() ESCRIBE UN SALTO DE LÍNEA

PRIMERO DECIDE: ¿BYTES O CARACTERES? ¿ENTRADA O SALIDA?
ESO ELIGE TU CLASE

4.2. Acceso aleatorio a ficheros

Definición

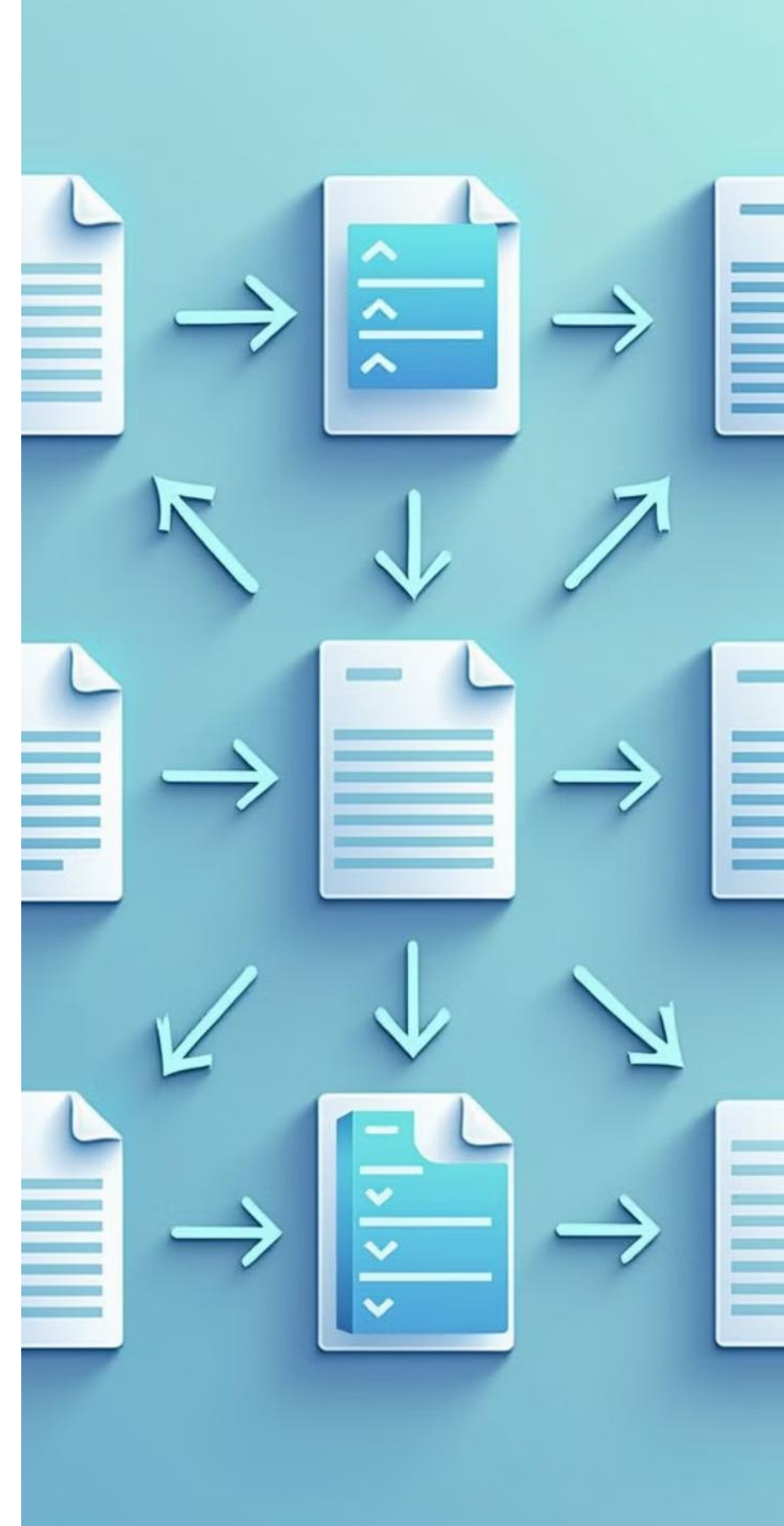
Permite acceder a cualquier parte del fichero directamente, sin necesidad de leer todo el contenido anterior.

¿Por qué se puede hacer esto? Organización

Fichero organizado en bloques o registros de longitud fija, lo que facilita el salto a cualquier posición dentro del mismo.

Eficiencia

Más eficiente para leer o modificar datos específicos en ficheros grandes.



4.2. Acceso aleatorio a ficheros

4.2.1. La clase `RandomAccessFile`

Para el acceso aleatorio a ficheros se utiliza la clase **`RandomAccessFile`**.

Novedades respecto a lo ya visto hasta ahora:

1. En todo momento el cursor se puede situar en cualquier posición mediante la función **`seek()`**. En eso consiste el acceso aleatorio.
2. Sobre el fichero se pueden realizar operaciones tanto de lectura como de escritura.

Limitaciones

Limitaciones que no vienen del lenguaje Java (ni del paquete `java.io`), sino de los sistemas de ficheros habituales:

Limitación 1

No es posible eliminar o insertar bloques de bytes o de caracteres en mitad de un fichero, ni reemplazar un bloque de un fichero por otro, a no ser que tengan exactamente el mismo tamaño en bytes.

Limitación 2

Habría que utilizar ficheros temporales auxiliares.

4.2. Acceso aleatorio a ficheros

4.2.1. La clase RandomAccessFile

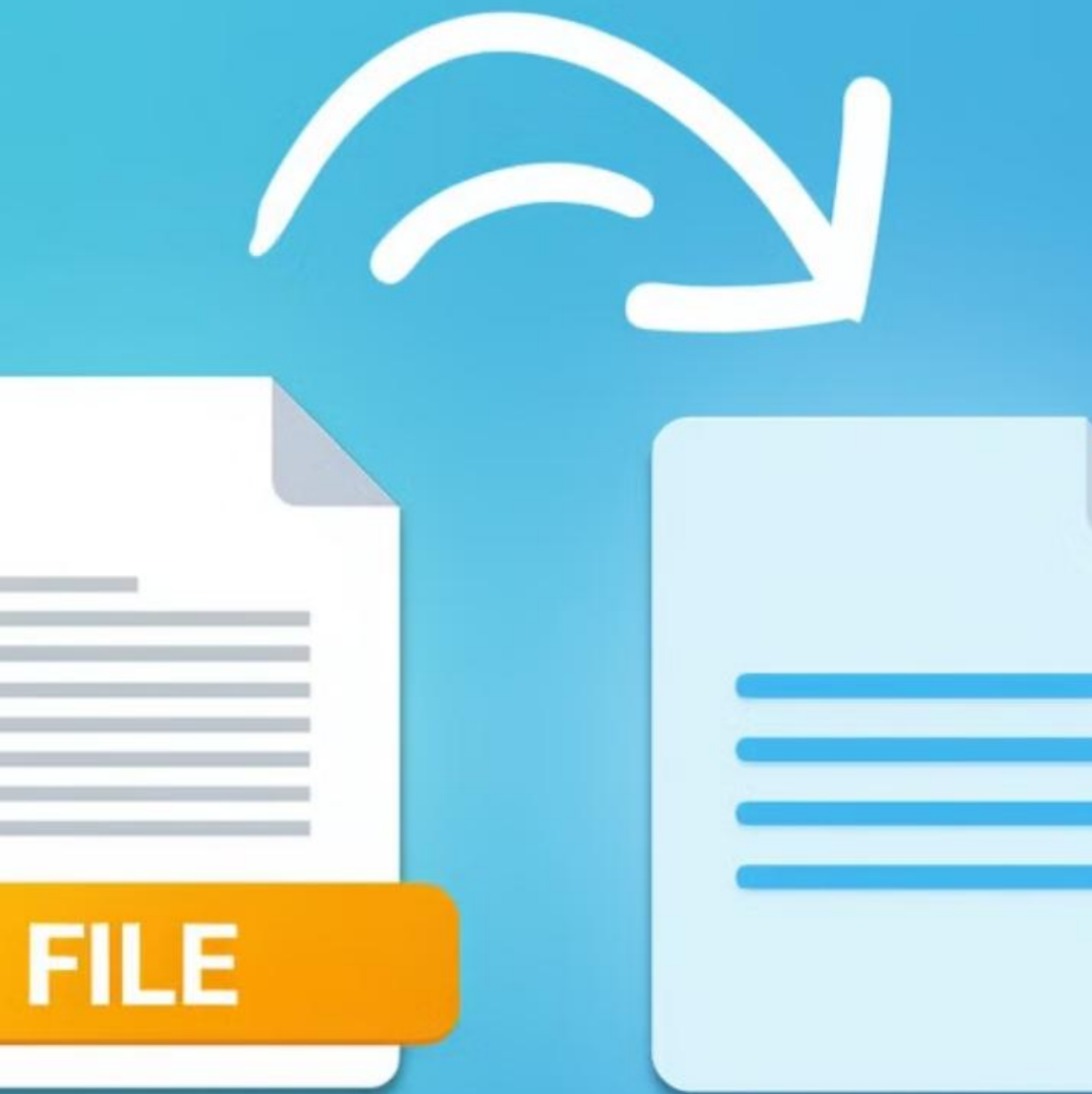
Constructores de la clase RandomAccessFile

La clase RandomAccessFile posee dos constructores:

```
RandomAccessFile(File file, String mode).
```

```
RandomAccessFile(String name, String mode).
```

- Un objeto File como primer parámetro, o un String.
- El modo es: "r" si se abre en modo lectura o "rw" si se abre en modo lectura y escritura.



Métodos de la clase `RandomAccessFile`

Método	Funcionalidad
<code>RandomAccessFile(File file, String mode)</code>	Constructor. Abre el fichero en el modo indicado, si se dispone de permisos suficientes: r: modo de solo lectura. rw: modo de lectura y escritura.
<code>RandomAccessFile(String name, String mode)</code>	rwd, rws: Como rw pero con escritura síncrona. Esto significa que todas las operaciones de escritura (de datos con rwd y de datos y metadatos con rws) deben haberse completado cuando termina la función. Esto puede hacer que la llamada a la función tarde más, pero asegura que no se pierde información crítica ante una caída del sistema.
<code>void close()</code>	Cierra el fichero. Es conveniente hacerlo siempre al final.
<code>void seek(long pos)</code>	Posiciona el puntero en la posición indicada.
<code>int skipBytes(int n)</code>	Intenta avanzar el puntero el número de bytes indicado. Se devuelve el número de bytes que se ha avanzado. Podría ser menor que el solicitado, si se alcanza el fin del fichero.



Métodos de la clase RandomAccessFile

Método	Funcionalidad
int read() int read(byte[] buffer) int read(byte[] buffer, int offset, int longitud)	Lee del fichero. Según la variante, un byte o hasta llenar el buffer, o el número de bytes indicados, que se copiarán en la posición indicada (offset) del buffer. Devuelve el número de bytes leídos, o -1 si no se pudo leer nada porque el puntero estaba al final del fichero.
void readFully(byte[] buffer) readFully(byte[] buffer, int offset, int longitud)	Como <code>int read(byte[] b)</code> , pero si no se puede leer hasta llenar el buffer, o el número de bytes indicado, porque se llega al final del fichero, se lanza la excepción <code>IOException</code> . Útil cuando se sabe que se podrá leer hasta llenar el buffer o el número de bytes indicados. En cualquier caso, la eventualidad de que no se pueda completar la lectura se puede gestionar capturando la excepción.
String readLine()	Lee hasta el final de la línea de texto actual.
void write(int b) void write(byte[]buffer) void write(byte[] buffer, int offset, int longitud)	Escribe en el fichero. Según la variante, un byte, o todos los contenidos del buffer, o el número de bytes indicados a partir de la posición indicada (offset). Si alcanza el fin del fichero, siguen escribiendo.





Principales métodos para el acceso aleatorio

- [seek\(long posicion\)](#)
Permite mover el punter de lectura/escritura a una posición específica de un objeto RandomAccessFile.
- [getFilePointer\(\)](#)
Devuelve la posición actual del puntero dentro del objeto RandomAccessFile.
- [skipBytes\(int desplazamiento\)](#)
Permite saltar un número específico de bytes desde la posición actual del puntero.
- [length\(\)](#)
Devuelve la longitud total del archivo en bytes.



ACCESO ALEATORIO (RandomAccessFile)



EL PUNTERO SE COLOCA EN CUALQUIER POSICIÓN CON `seek()`,
SIN LEER LO ANTERIOR

DOS NOVEDADES

1. EL CURSOR SALTA A CUALQUIER POSICIÓN CON `seek()`
2. LECTURA Y ESCRITURA SOBRE EL MISMO FICHERO

MÉTODOS CLAVE

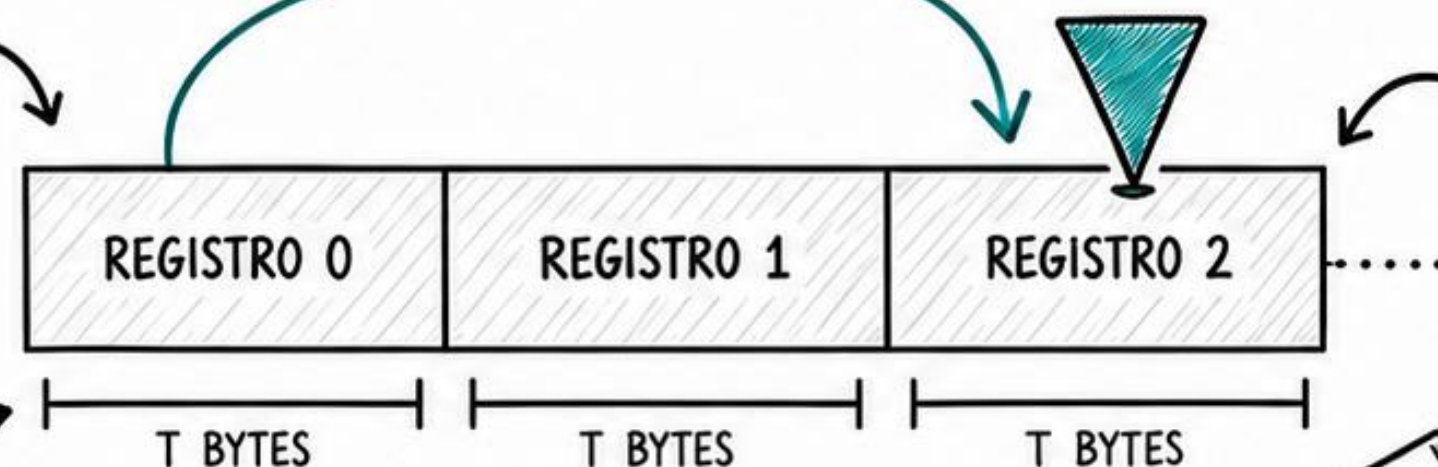
- `seek(long pos)`: MUEVE EL PUNTERO
- `getFilePointer()`: POSICIÓN ACTUAL
- `skipBytes(int n)`: AVANZA N BYTES
- `length()`: TAMAÑO DEL FICHERO

LÍMITES

- NO SE PUEDE INSERTAR NI BORRAR EN MEDIO
- SOLO REEMPLAZAR SI OCUPA LO MISMO
- PARA LO DEMÁS: FICHEROS AUXILIARES
- ES UNA LIMITACIÓN DEL SISTEMA DE FICHEROS, NO DE JAVA



`seek()`



MODOS DE APERTURA

- "r" = SOLO LECTURA
- "rw" = LECTURA Y ESCRITURA
- "rwd" / "rws" = LECTURA/ESCRITURA CON ESCRITURA SÍNCRONA

POSICIÓN DEL REGISTRO $i = i * T$

LA CLAVE: REGISTROS DE LONGITUD FIJA

- TODOS LOS REGISTROS OCUPAN LO MISMO (T BYTES)
- ASÍ SE PUEDE CALCULAR DÓNDE EMPIEZA CADA UNO



SECUENCIAL = DE PRINCIPIO A FIN / ALEATORIO = SALTO DIRECTO



IDEAL PARA MODIFICAR UN DATO CONCRETO EN FICHEROS GRANDES
(COMO UNA BASE DE DATOS SENCILLA)



ACTIVIDAD PRÁCTICA



Actividad 5 RA1_A5

Operaciones comunes sobre
ficheros de acceso aleatorio

Realizar una serie de operaciones de lectura y escritura en un archivo de datos, usando la clase `RandomAccessFile`. Se practican los conceptos de acceso aleatorio a través de la clase, moviéndose a posiciones específicas dentro del archivo y actualizando información

[Ver Actividad 5 en Aula Virtual]



Otros tipos de acceso

Según la bibliografía consultada... binarios, dinámicos...

Acceso binario

Definición

Permite acceder directamente a datos en formato de bytes.

Uso

Útil para ficheros no legibles como imágenes, música o vídeos.

Ventaja

Permite leer o escribir datos en bloques de bytes.

Locklas

CESS

online stots
fator
or stofs
ge -4virg
ch'ilire.

Losuer (Rememerd)

● Flatl ridont

● Oigirol rrom necek.

● Empedation

● Aliren faiture

● Exill tident

ne
e jofig
ne locdice
re rating
irie!

Bvili lux 21x havide a rimint
Eaili lux tlarde:lerl i
Exill docros wirls
Flax Tlo de
1.41-vens in feles

S

vply
r leedfing
ct avits
ng. oirty
jem durts wite

Pimbrackding carbe
Lecinte citler tepited
Emp fides concemients
En inowativit oiemitiron a (metning)

JES

s
r pase
o seins
r leets duditists-
isieg foreidole
erby vise on escepting)

Plack mont
Spinie.d arilime
Exinnial processonc
Dite rclus es ccces

Diferencias entre los tipos de acceso

Tipo de acceso	Características principales	Ejemplo de uso
Secuencial	Lee o escribe datos en un flujo continuo, desde el principio hasta el final	Ficheros de texto (logs, registros)
Aleatorio	Permite saltar a posición específica	Bases de datos en formato de fichero
Binario	Trabaja con bloques de bytes en lugar de caracteres, ideal para datos no legibles	Ficheros multimedia (imágenes, vídeos)

4.3. Aplicaciones reales



Archivos de registro (logs)

En muchas aplicaciones, los archivos de registro son utilizados para almacenar datos de eventos de forma secuencial. Cada nuevo registro se añade al final del fichero.



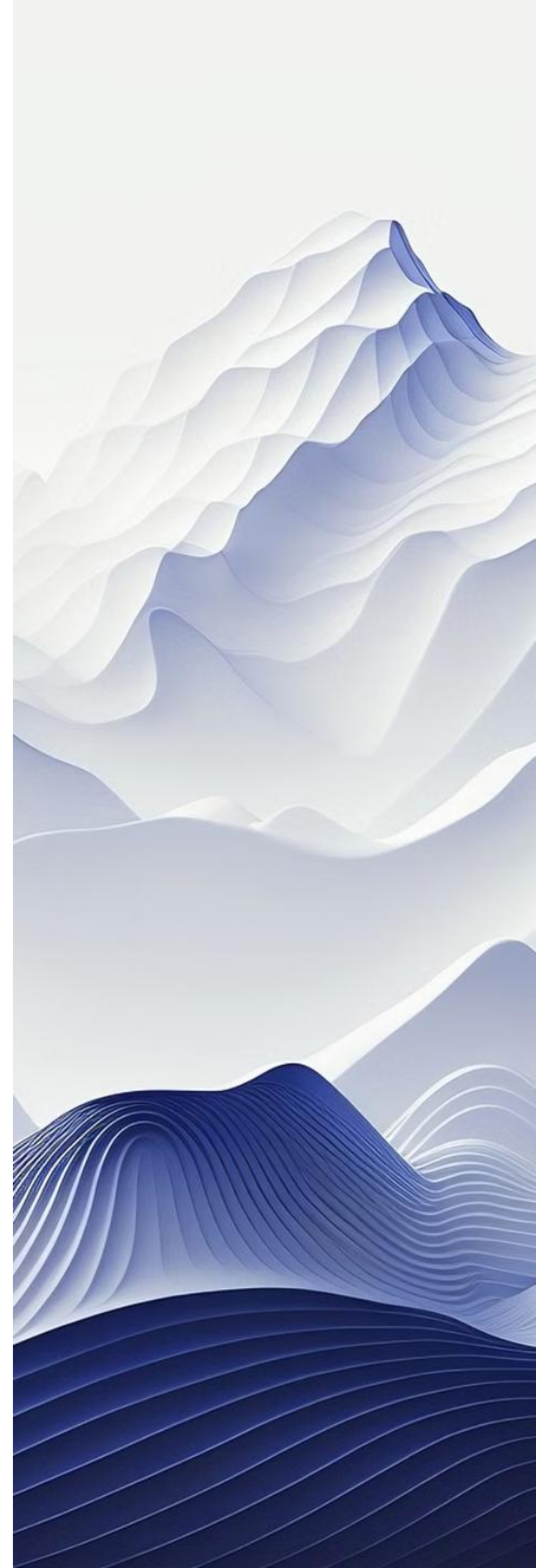
Bases de datos simples

Algunas aplicaciones utilizan ficheros binarios para implementar una base de datos simple donde los registros están organizados en bloques de tamaño fijo y el acceso aleatorio facilita las operaciones de consulta o modificación.



Archivos multimedia

Los ficheros binarios son comúnmente usados para almacenar datos multimedia como imágenes, vídeos y música. Estos ficheros necesitan ser accedidos aleatoriamente para poder modificar, extraer o reproducir secciones específicas del contenido.



Importancia de la gestión correcta

1

Errores de gestión

Pueden provocar pérdida de datos al final del archivo.

2

Impacto

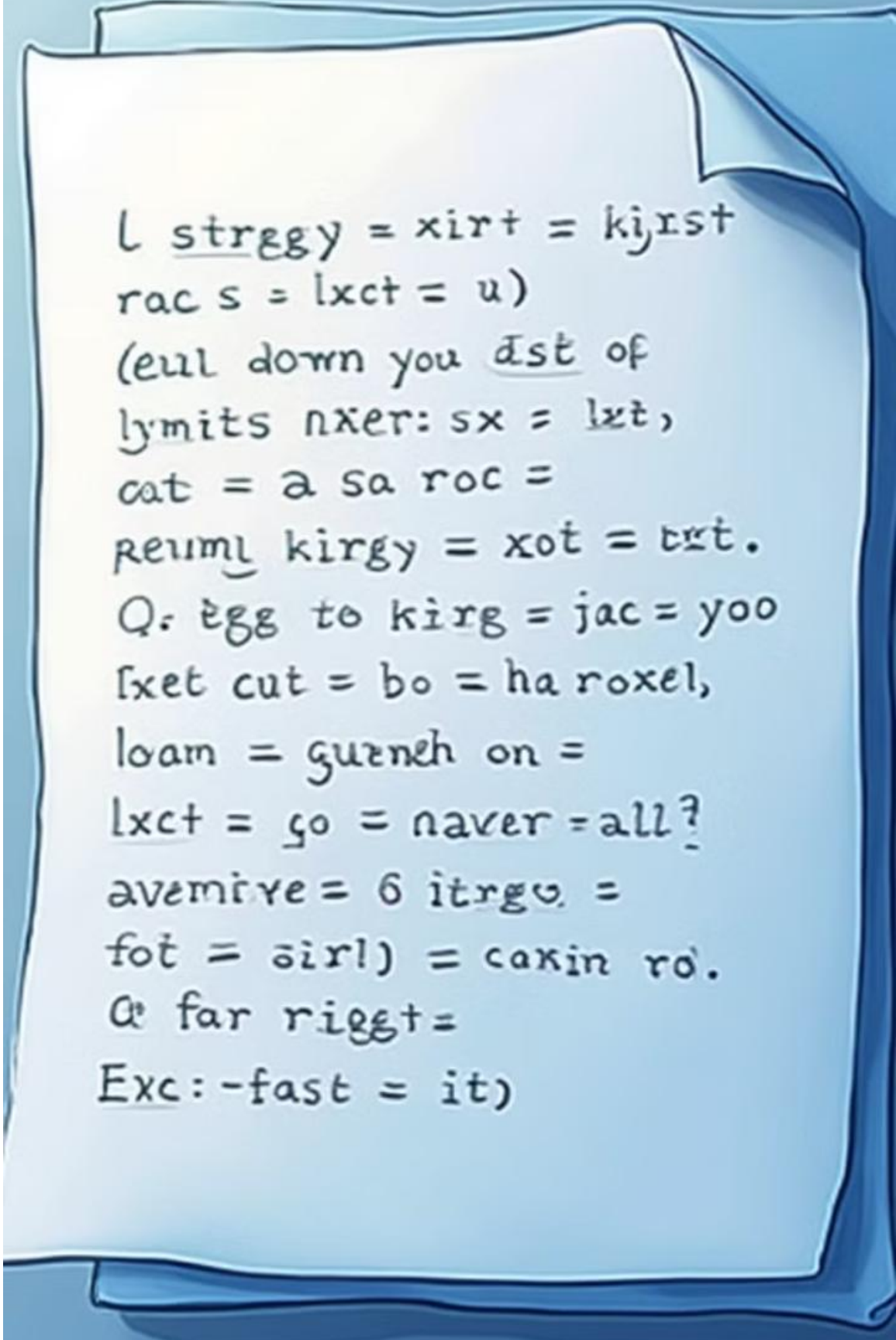
Afecta a usuarios con grandes cantidades de datos en ficheros secuenciales.

3

Solución

Lanzamiento rápido de parches para corregir fallos.

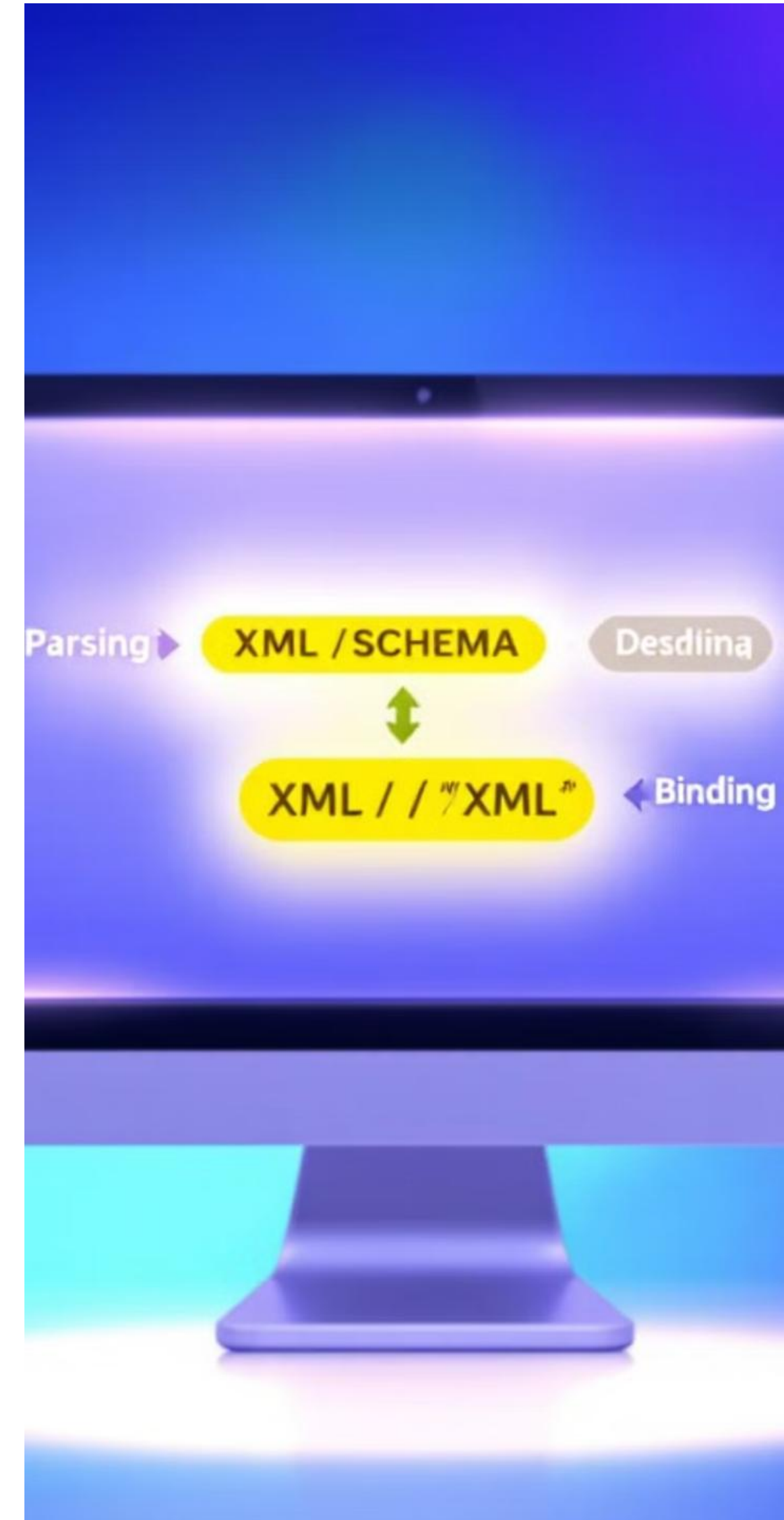
Errores en la gestión de ficheros con el sistema operativo, concretamente en la gestión de los accesos secuenciales a ficheros, puede provocar la pérdida de datos al final del archivo. En ese caso se tendría que lanzar un parche rápidamente, ya que este fallo afectaría a los usuarios que almacenaban grandes cantidades de datos en ficheros de acceso secuencial. Esta posible situación pone de manifiesto la importancia de gestionar correctamente el acceso a ficheros, especialmente en sistemas operativos con millones de usuarios.



```
l strggy = xirt = kirst  
rac s = lxct = u)  
(eul down you dst of  
lymits nxer: sx = lxt,  
cat = a sa roc =  
reuml kirgy = xot = txt.  
Q: egg to kirg = jac = yoo  
[xet cut = bo = ha roxel,  
loam = guzneh on =  
lxct = go = naver = all?  
avemive = 6 itrgo =  
fot = sirl) = caxin ro.  
Q: far riggt=  
Exc: -fast = it)
```

6. Ficheros XML

- 1 — 6.1. Introducción
 - Conceptos clave, ventajas, etc...
- 2 — 6.2. Parser o analizador XML
 - Tipos de parsers, etc
- 3 — 6.3. Parser SAX
- 4 — 6.4. Parser DOM **(RA1_A6)**
- 5 — 6.5. Binding (JAXB)
- 6 — 6.6. Librerías para conversion XML a otros formatos



5.1. Introducción

¿Qué es XML?

Es un lenguaje de marcas (metalenguaje) creado para estructurar, almacenar e intercambiar datos.

¿Por qué se crea XML?

- **Interoperabilidad:** Permite la comunicación entre plataformas y redes distintas.
- **Estandarización:** Facilita un soporte estándar para el intercambio de datos.
- **Accesibilidad:** Asegura un formato y acceso estándar, útil en aplicaciones y sistemas heterogéneos.





Ventajas del uso de archivos XML

Facilita el intercambio de información

Utiliza estándares aceptados para el intercambio entre aplicaciones.

Estructuración clara de los datos

La estructura permite un procesamiento sencillo y la reutilización de datos.

Versatilidad y compatibilidad

Se puede utilizar en una amplia variedad de aplicaciones y sectores.

¿Qué puede hacer un desarrollador con XML?

Leer archivos XML



Cargar el archivo en memoria desde un formato de texto estándar.

Procesar y analizar datos



Obtener información estructurada y transformar o manipular datos.

Almacenar datos en formato XML



Guardar los resultados de las operaciones en archivos XML para su uso posterior.

```
<?xml version="1.0"?>
<quiz>
  <qanda seq="1">
    <question>
      Who was the forty-second
      president of the U.S.A.?
    </question>
    <answer>
      William Jefferson Clinton
    </answer>
  </qanda>
  <!-- Note: We need to add
  more questions later.-->
</quiz>
```

XML

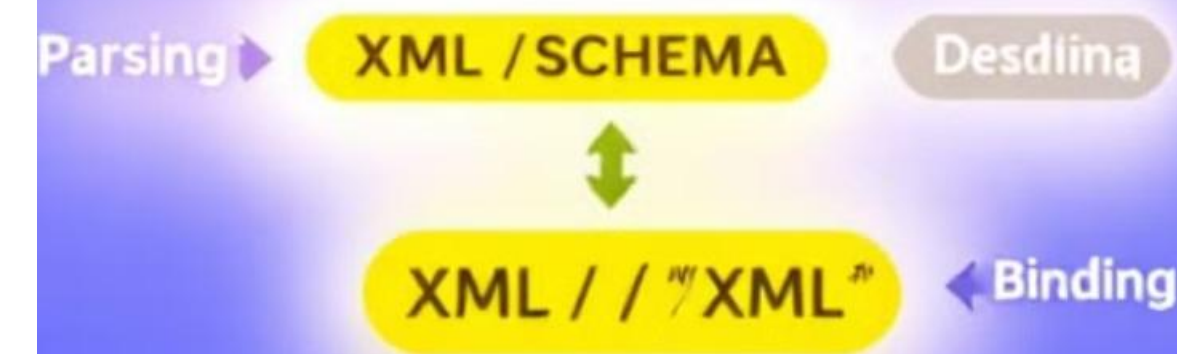
5.1. Conceptos clave

PARSING:

- ¿Qué es parsing (análisis léxico sintáctico)?
- ¿Que es un parser XML?
- ¿Para qué se utilizan los parser XML?

¿Schema (esquema)?

¿Binding (vinculación)?



5.1. Conceptos clave

☐ Parsing (análisis léxico sintáctico)

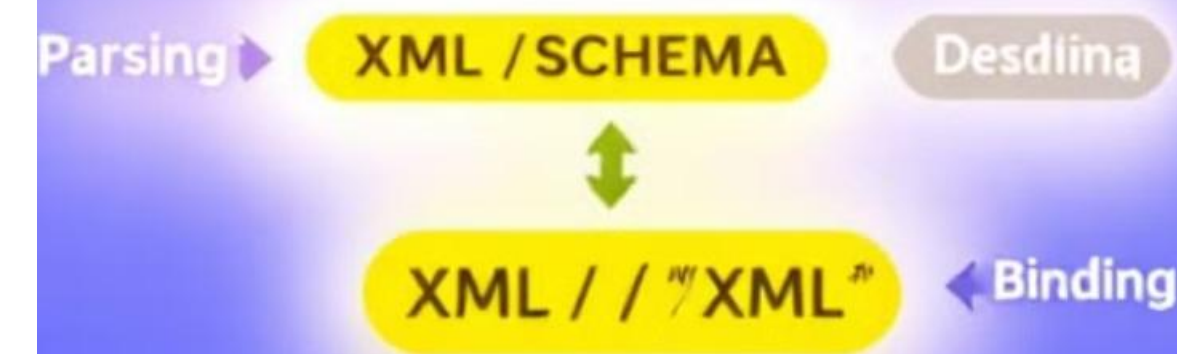
Lectura y validación de un documento XML para asegurar que esté bien formado y estructurado. Para, posteriormente, pasar el contenido de ese documento a una aplicación cliente que necesite consumir dicha información.

☐ Schema (esquema)

Especifica los elementos permitidos en un archivo XML, su orden y sus relaciones. Permite que el documento XML sea válido.

☐ Binding (vinculación)

Proceso de generar clases de Java que representan el esquema XML, facilitando su manipulación y uso en aplicaciones.



5.2. PARSER O ANALIZADOR XML

¿Qué es un Parser XML?

- ❑ Un módulo, biblioteca o programa que lee y analiza un archivo XML.
- ❑ Transforma el archivo XML en una representación interna para facilitar su uso en aplicaciones.

¿Por qué usar Parsers XML?

- ❑ Aseguran que el documento sea sintácticamente válido.
- ❑ Facilitan la extracción y organización de datos.



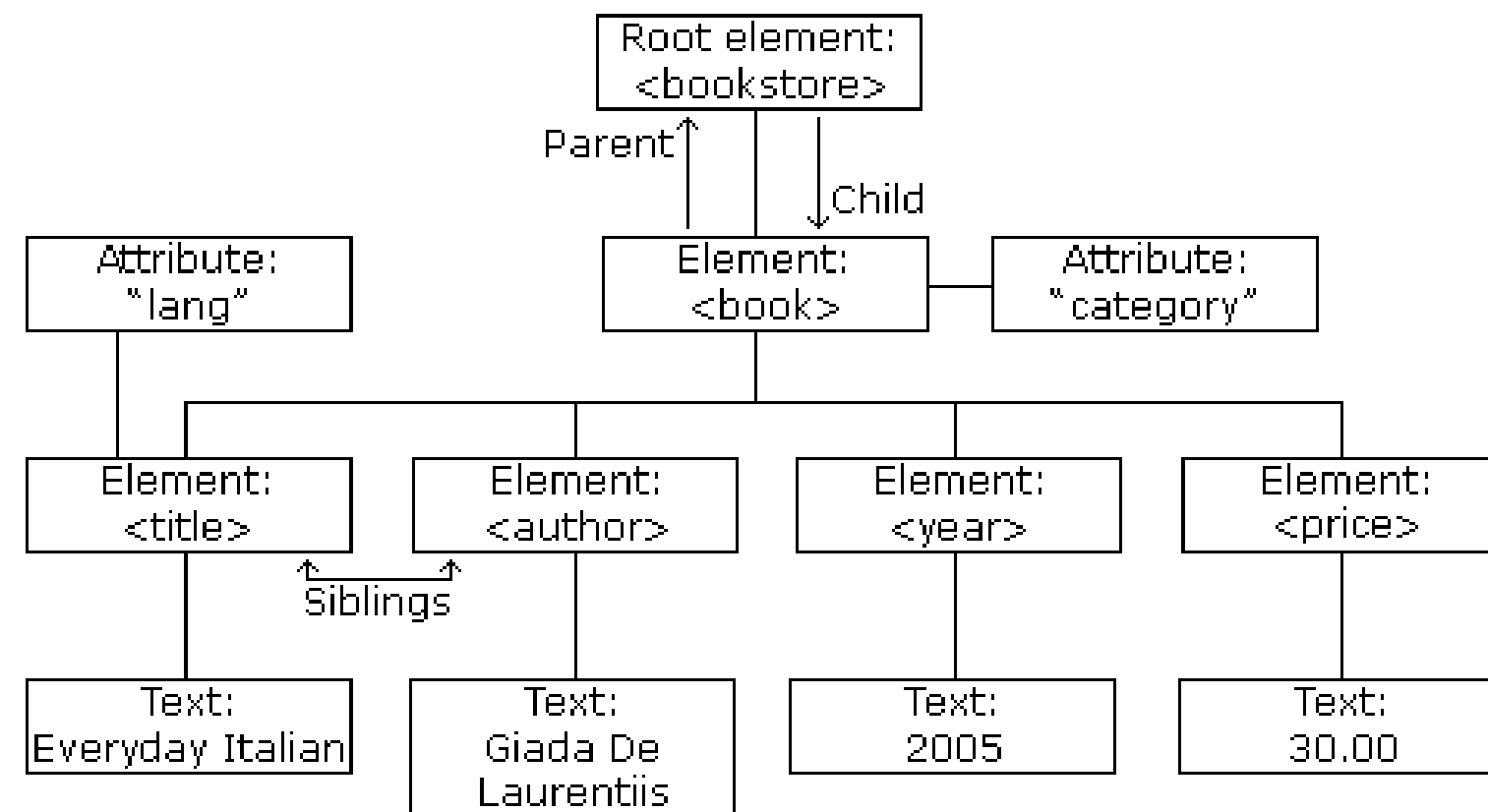
Objetivo de XML y su estructura

Formato Estandarizado:

- XML es ideal para el intercambio de datos en plataformas heterogéneas.
- Estructura los datos en un formato basado en texto y etiquetas.

Estructura Jerárquica:

- XML almacena datos en una jerarquía similar a la de los objetos en programación orientada a objetos.
- Etiquetas y atributos ayudan a clasificar la información.



5.2.1. Tipos de Parsers XML

Parsers SECUENCIALES

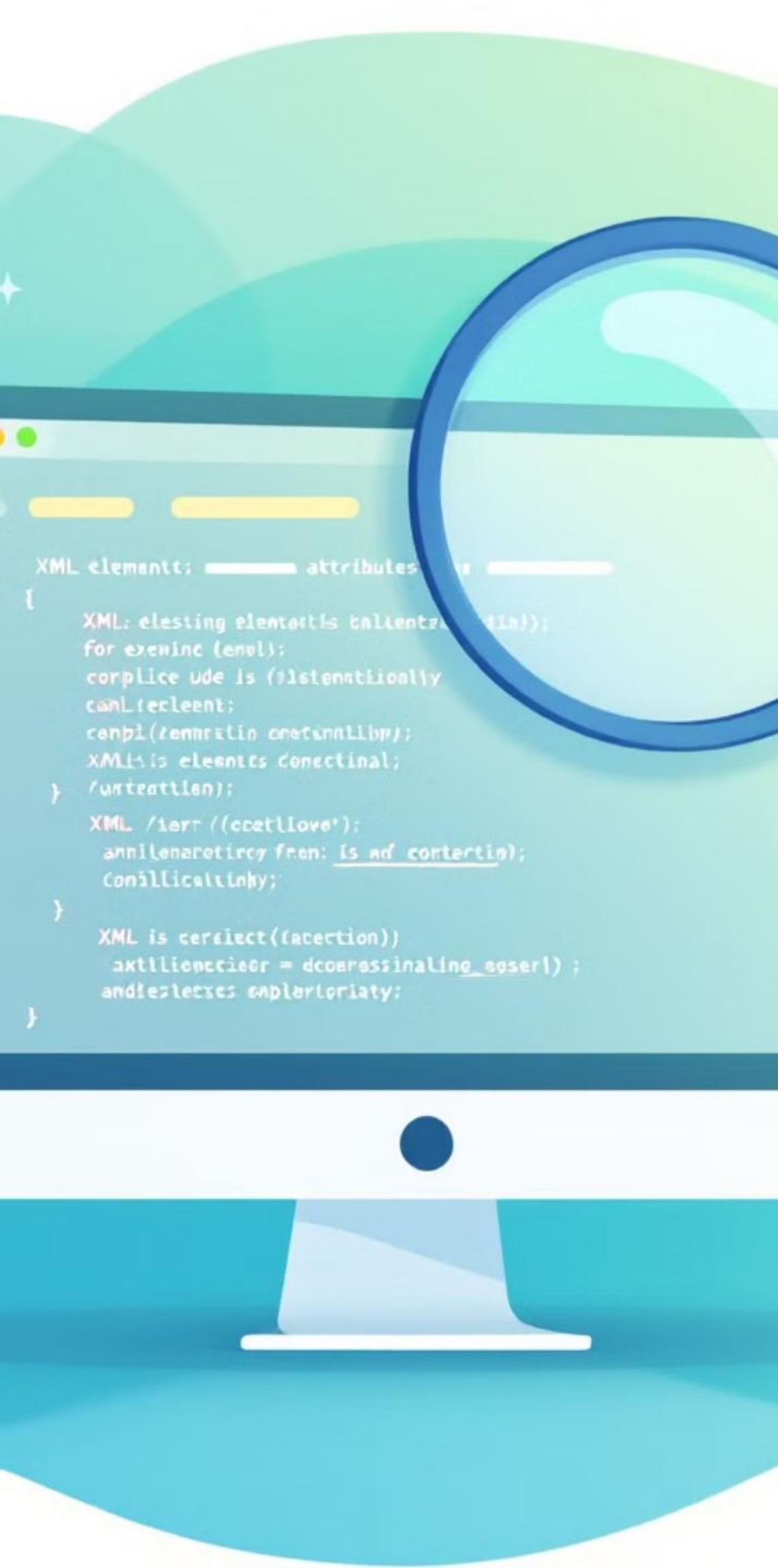
Parsers JERÁRQUICOS

-  **Parsers secuenciales:**
¿qué son? ¿como funcionan? ¿algún ejemplo?
-  **Parsers jerárquicos:**
¿qué son? ¿como funcionan? ¿algún ejemplo?

5.2.1. Tipos de Parsers XML

Parsers Secuenciales:

- Analizan el documento de principio a fin. A medida que se encuentran las etiquetas de apertura y cierre, extraen el contenido.
- Se denominan analizadores sintácticos y son muy rápidos, pero requieren leer el documento completo para encontrar información específica.
- Por ejemplo SAX y STAX en Java.



5.2.1. Tipos de Parsers XML

Parsers Jerárquicos:

- Guardan toda la estructura en memoria RAM, siguiendo la jerarquía del documento.
- Más eficientes para consultas repetitivas y para aplicaciones que requieren acceso continuo a los datos.
- Las aplicaciones finales que necesitan trabajar con datos XML suelen usar analizadores jerárquicos.
- El formato de la estructura donde se almacena la información en la memoria RAM ha sido especificado por el organismo internacional W3C (World Wide Web Consortium) y se suele conocer como

DOM (Document Object Model).



Parser SAX (API Simple para XML)

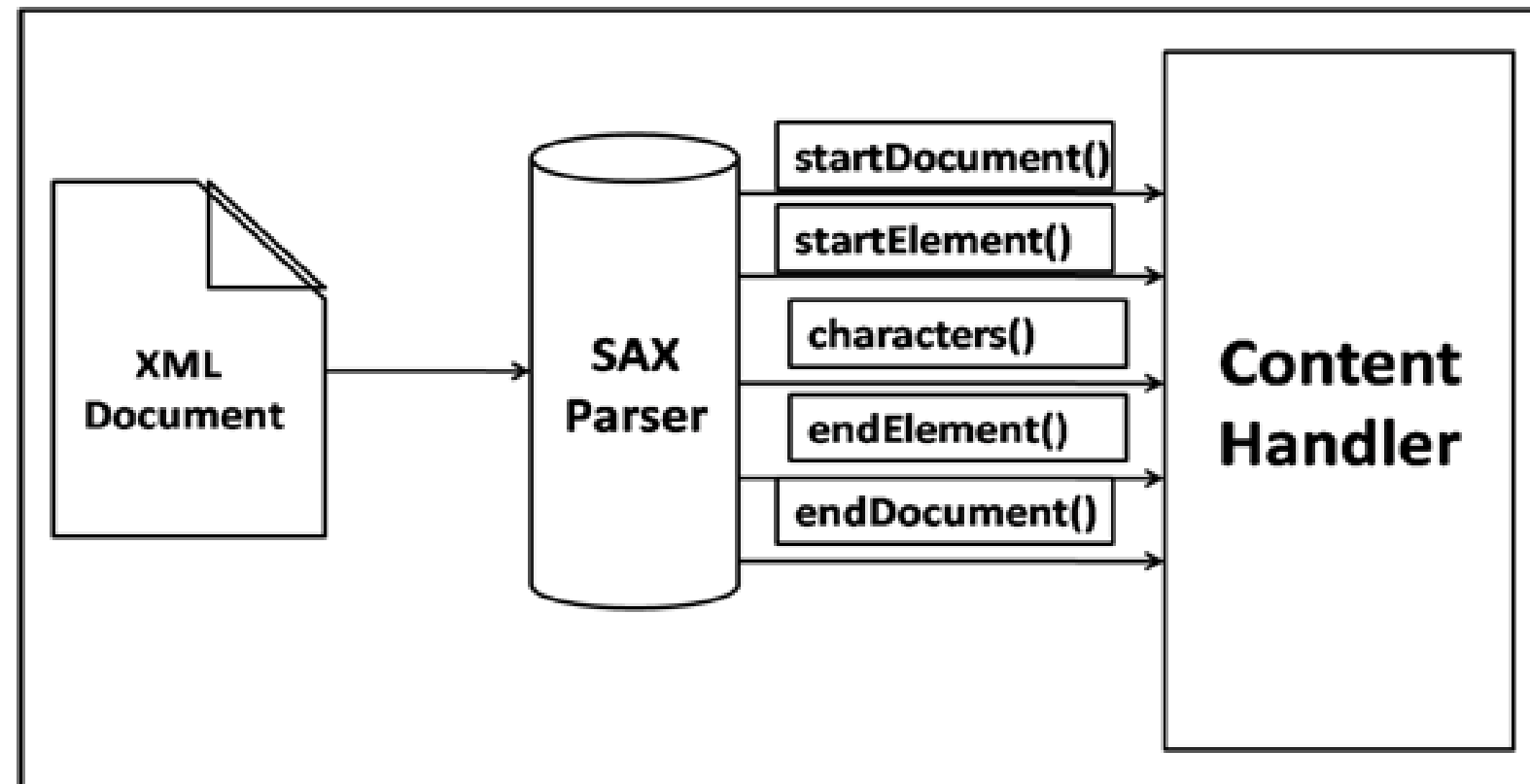


¿Qué es SAX?

- Es una interfaz simple de aplicaciones XML, fácil e intuitiva.

Uso de SAX en aplicaciones

- Los parsers SAX leen el fichero XML de manera secuencial, y producen eventos secuencialmente en función de los resultados de lectura.

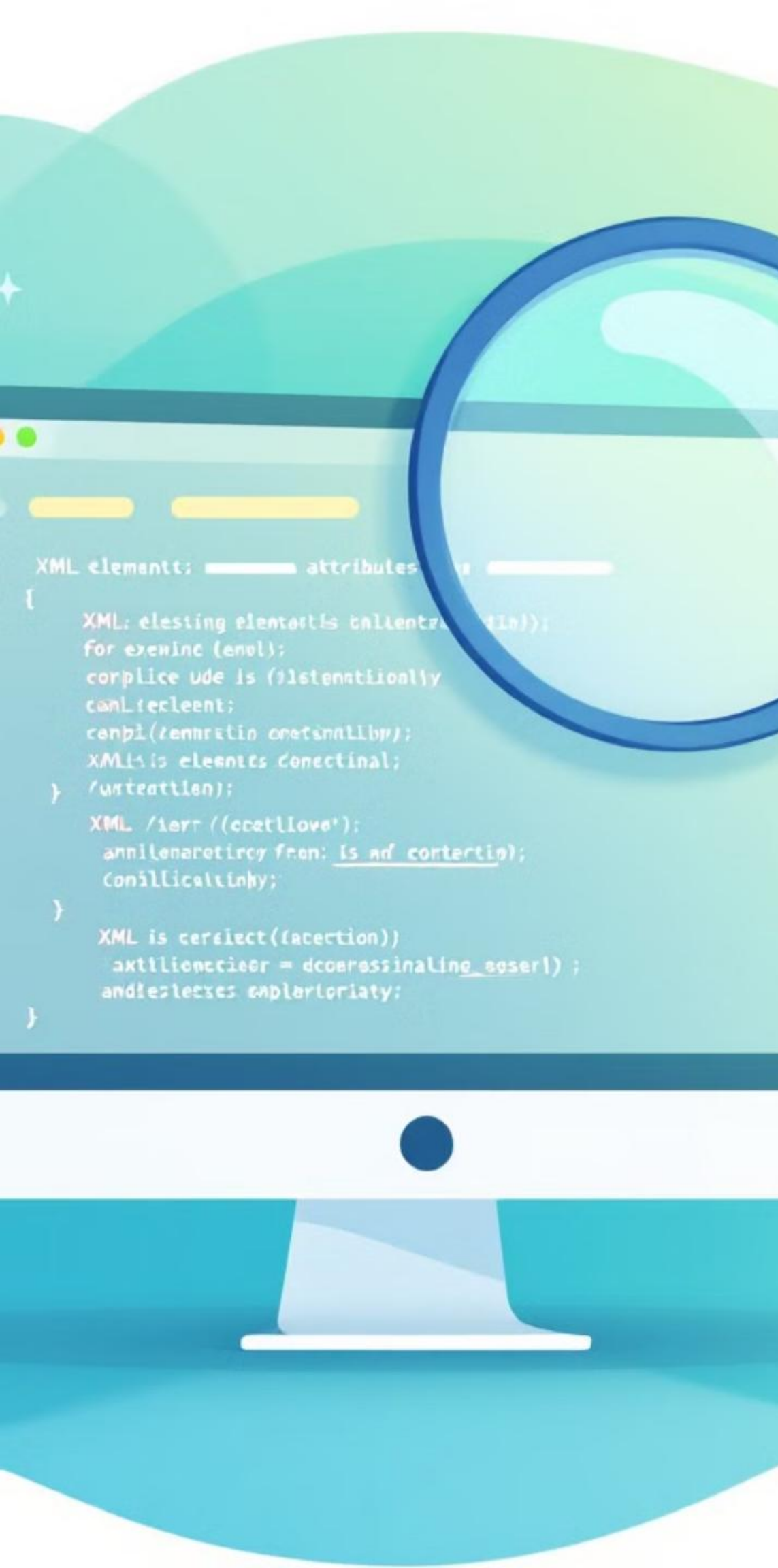


Ventajas e inconvenientes

- Procesa el documento poco a poco y consume muy poca memoria.
- Impide tener una vision general del documento XML

¿Cuándo conviene utilizar SAX?

- ☐ Cuando el documento XML es considerablemente grande.
- ☐ Cuando no se requiere modificación estructural
- ☐ Cuando se quiere parsear diferentes partes del documento

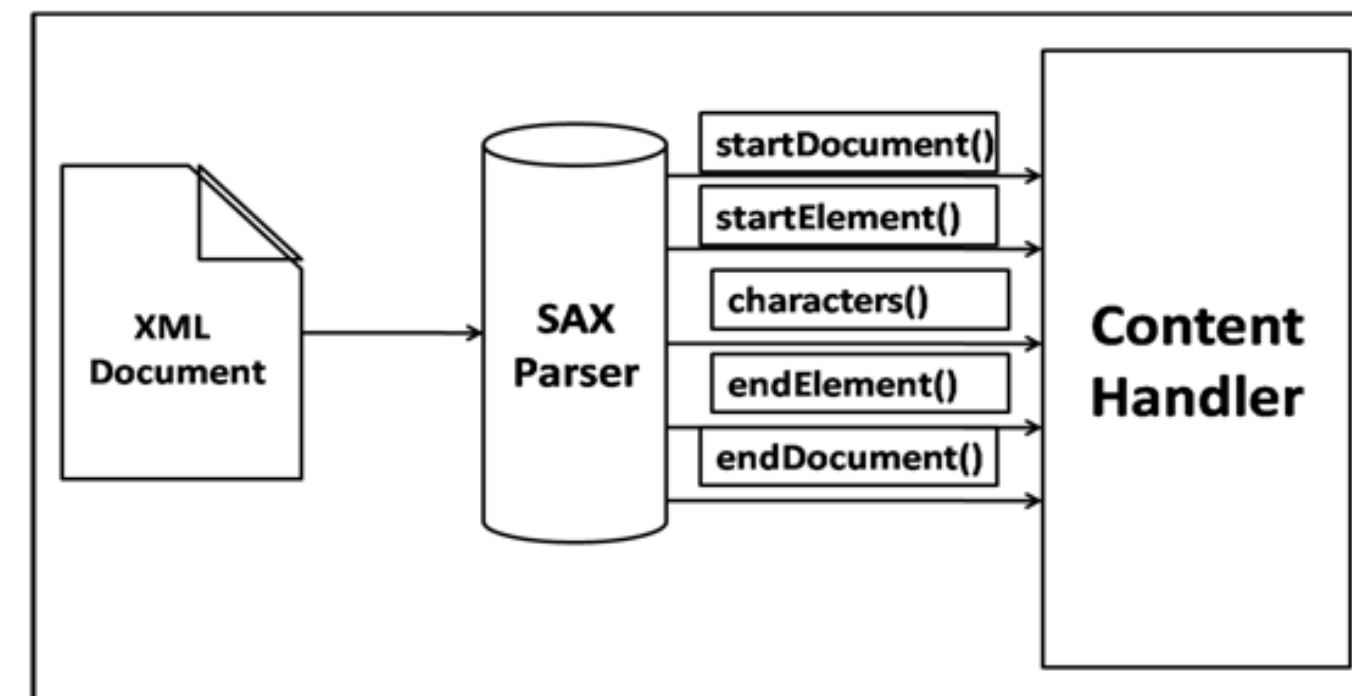


Eventos y métodos de SAX

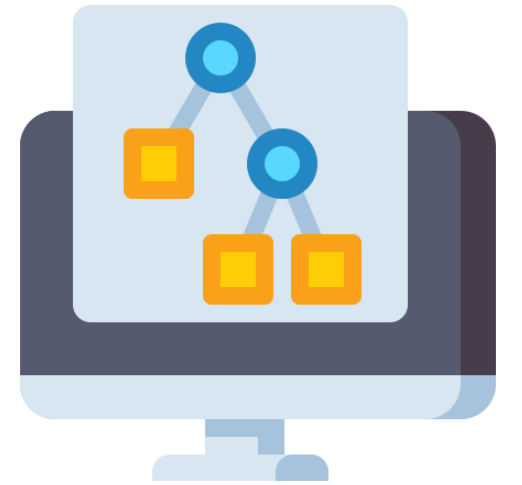
La lectura de un documento XML produce eventos los cuales invocan a métodos.

Eventos:

- Inicio de un archivo XML (startDocument())
- Inicio y fin de un elemento (startElement() y endElement())
- Información que llevan las etiquetas (Character())
- Fin de un archivo XML (endDocument()).



Parser DOM (Document Object Model)

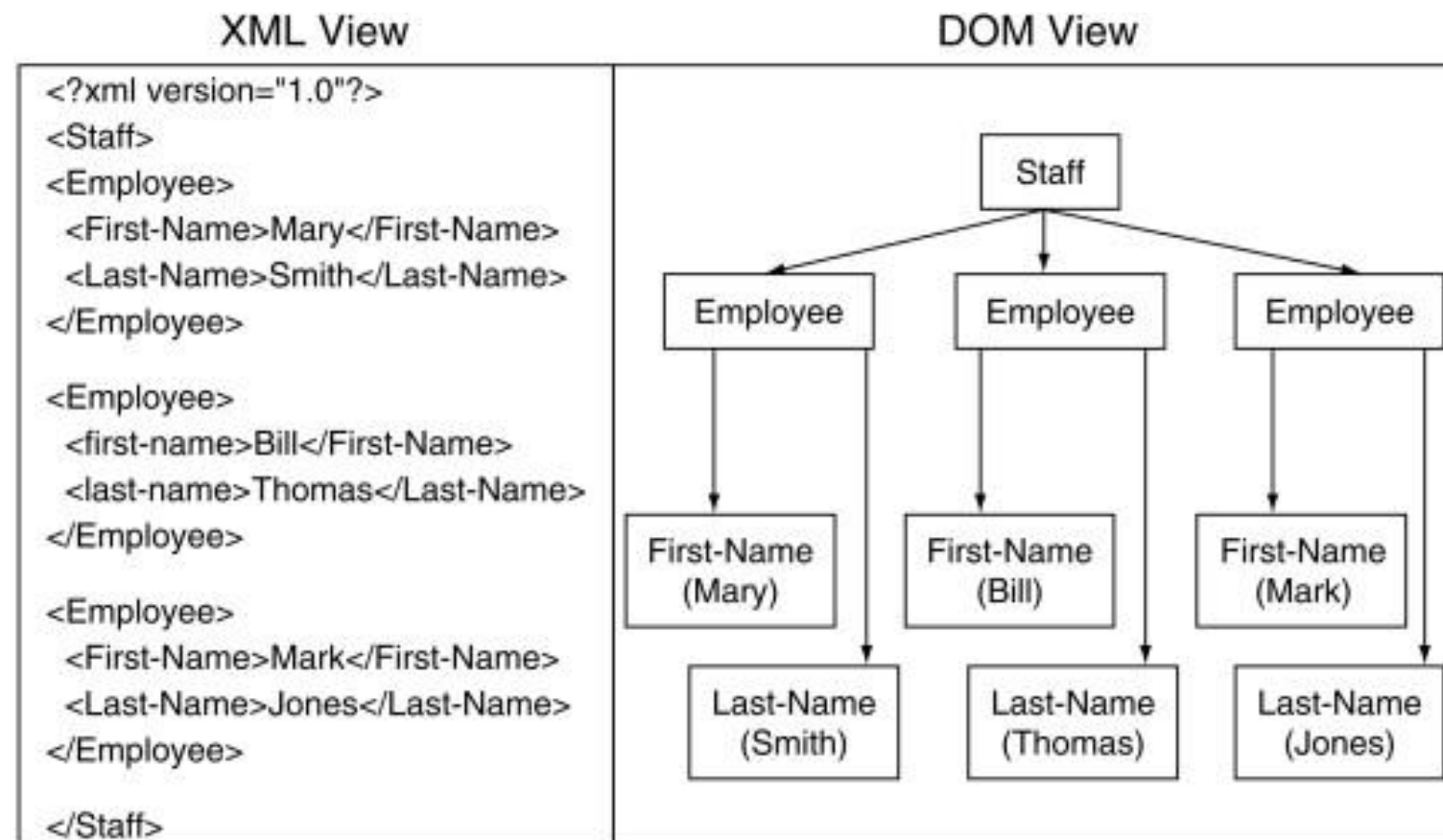


¿Qué es DOM?

- Es una recomendación del W3C para manipular documentos XML y HTML.
- Define una interfaz para acceder y actualizar el estilo, la estructura y el contenido de los documentos XML.

Uso de DOM en aplicaciones

- Los parsers DOM leen el XML y crean un modelo en memoria, permitiendo la manipulación fácil de su contenido.



Paquetes XML para JAVA

```
import org.w3c.dom.*;
import javax.xml.parsers.*;
import java.io.*;
```



Cómo funciona el parser DOM

Proceso básico -> instanciar estructuras DOM a partir de un XML:

- Carga todo el archivo XML en memoria como un árbol de nodos.
- Permite el acceso y manipulación de cualquier parte del documento.

Clases clave:

- **DocumentBuilder** (clase abstracta que no se puede instanciar de forma directa), por ello especifican la clase **DocumentBuilderFactory**.
- **DocumentBuilderFactory** para crear y configurar el parser DOM.
- **Document** representa el archivo XML como un árbol de nodos.

Instrucciones para cargar un documento XML con DOM

Pasos para crear un DOM:

1

Instanciar un `DocumentBuilderFactory`.

2

Crear un `DocumentBuilder` desde la Factory.

3

Parsear el archivo XML usando `DocumentBuilder` para obtener un `Document`.

```
...
File file = new File("empleados.xml");
...
try {
    DocumentBuilderFactory dbFactory = DocumentBuilderFactory.newInstance();
    DocumentBuilder dBuilder = dbFactory.newDocumentBuilder();
    Document doc = dBuilder.parse(file);
} catch (Exception e) {
    e.printStackTrace();
}
```

La estructura DOM

Toma la forma de un árbol, donde cada parte del XML se encontrará representada en forma de nodo.

Diferentes tipos de nodo:

- **El nodo principal (todo el XML entero) -> Documento**
- **Etiquetas (incluida la etiqueta raíz) -> nodos elemento.**
- **El contenido textual -> nodo de tipo TextElement**
- **Atributos -> nodos de tipo Attribute**

Principales interfaces de DOM

Node:

Representa cualquier nodo del documento.

Attr:

Representa un atributo de un elemento.

Document:

El árbol DOM completo, con métodos para crear y manipular nodos.

Element:

expone propiedades y métodos para manipular atributos y otros elementos del documento.

Text:

Contenido de texto de un elemento o atributo.

NodeList:

Colección de nodos, similar a una lista, que facilita el acceso secuencial a los nodos.

Métodos comunes en DOM

Document.getDocumentElement()

- Devuelve el elemento raíz.

Node.getLastChild()

- Devuelve el último nodo hijo.

Node.getPreviousSibling()

- Devuelve el nodo hermano anterior.

Node.getFirstChild()

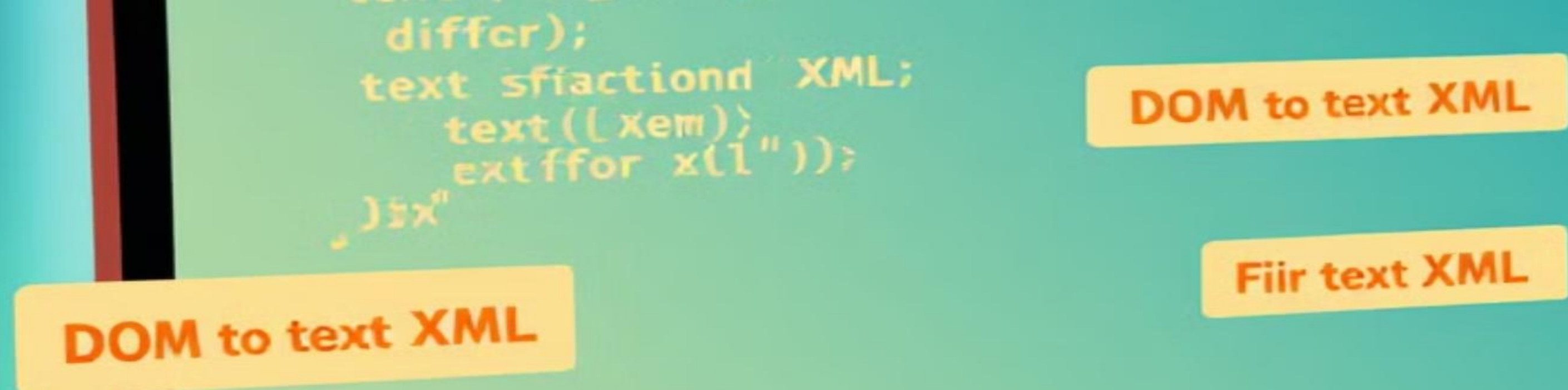
- Devuelve el primer nodo hijo.

Node.getNextSibling()

- Devuelve el siguiente nodo hermano.

Node.getAttribute(attrName)

- Devuelve el valor de un atributo del nodo.



Convertir el DOM a texto XML

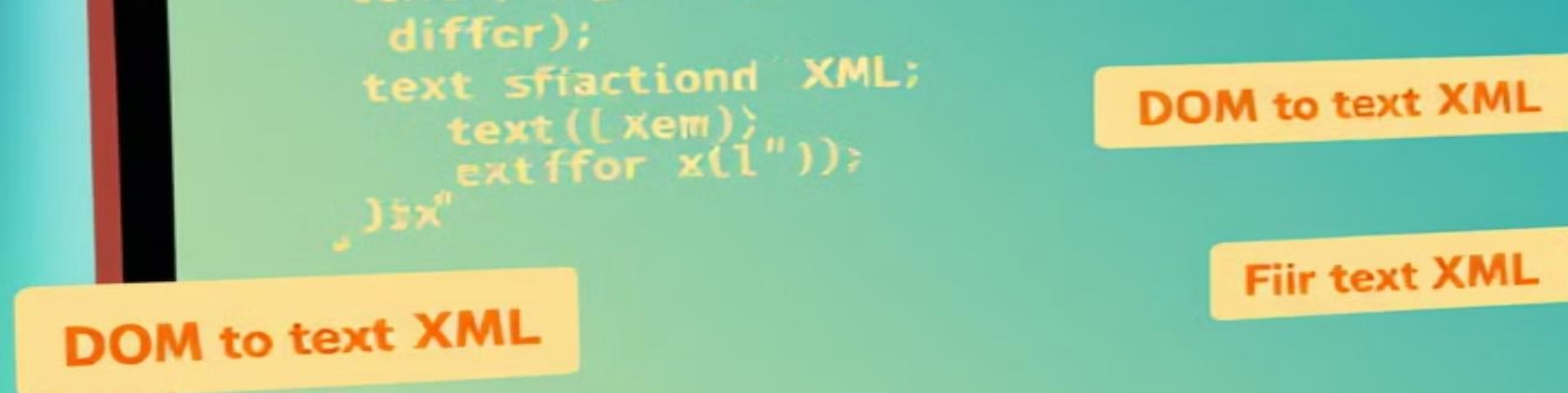
Clase Transformer:

- Convierte la estructura DOM de nuevo a un formato de texto XML.
- Utiliza adaptadores (DOMSource y StreamResult) para manipular diferentes formatos de entrada y salida.

Pasos para escribir un archivo XML:



1. Crear una instancia de Transformer.
2. Usar DOMSource para leer el árbol DOM y StreamResult para escribir en un archivo XML.



Busca ejemplos de conversion de DOM a archivo de texto XML e identifica las principales partes del código

1. Importar paquetes necesarios.
2. Instanciar un DocumentBuilderFactory.
3. Crear un DocumentBuilder desde la Factory.
4. Crear elemento raiz.
5. Añadir elementos.
6. Escribir el contenido de DOM en un archivo XML (Transformer)

Ejemplo

1 ¿QUÉ ES UN PARSER?



MÓDULO O BIBLIOTECA QUE LEE Y ANALIZA UN DOCUMENTO XML PARA CONVERTIRLO EN ALGO MANEJABLE POR LA APLICACIÓN

2 ¿PARA QUÉ SIRVE?



- ✓ COMPRUEBA QUE EL XML ESTÉ BIEN FORMADO
- ✓ VALIDA CONTRA UN ESQUEMA (DTD / XSD)
- ✓ FACILITA EXTRAER Y ORGANIZAR DATOS
- ✓ PERMITE LA INTEROPERABILIDAD ENTRE PLATAFORMAS

3 EN JAVA SE USA JAXP

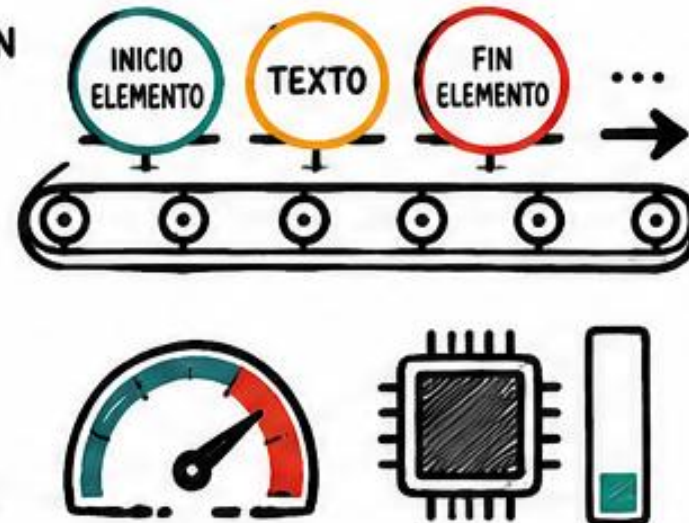
OFRECE DOS ENFOQUES: SAX Y DOM



ANALIZADORES XML (PARSERS)

4 SAX (POR EVENTOS)

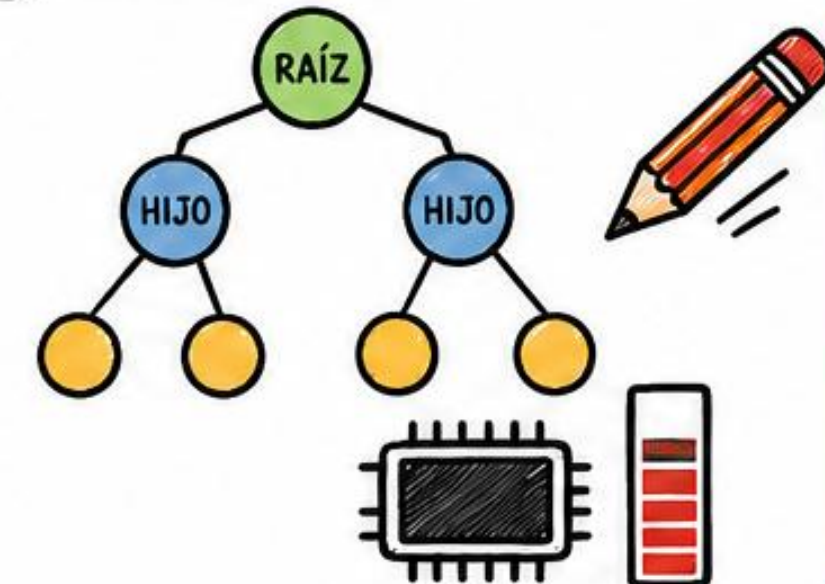
- RECORRE EL XML DE PRINCIPIO A FIN
- LANZA EVENTOS: INICIO Y FIN DE ELEMENTO, TEXTO...
- BAJO CONSUMO DE MEMORIA
- MUY RÁPIDO
- SOLO LECTURA HACIA DELANTE
- IDEAL PARA FICHEROS MUY GRANDES



VS

5 DOM (ÁRBOL EN MEMORIA)

- CARGA TODO EL XML COMO UN ÁRBOL DE NODOS
- PERMITE NAVEGAR Y MODIFICAR EL DOCUMENTO
- MÁS CONSUMO DE MEMORIA
- ACCESO ALEATORIO A CUALQUIER NODO
- IDEAL PARA MODIFICAR Y GUARDAR EL XML



SAX = RÁPIDO Y LIGERO

DOM = FLEXIBLE Y EDITABLE

ELIGE SEGÚN EL TAMAÑO DEL XML Y SI NECESITAS MODIFICARLO

ACTIVIDAD PRÁCTICA



Actividad 6 RA1_A6

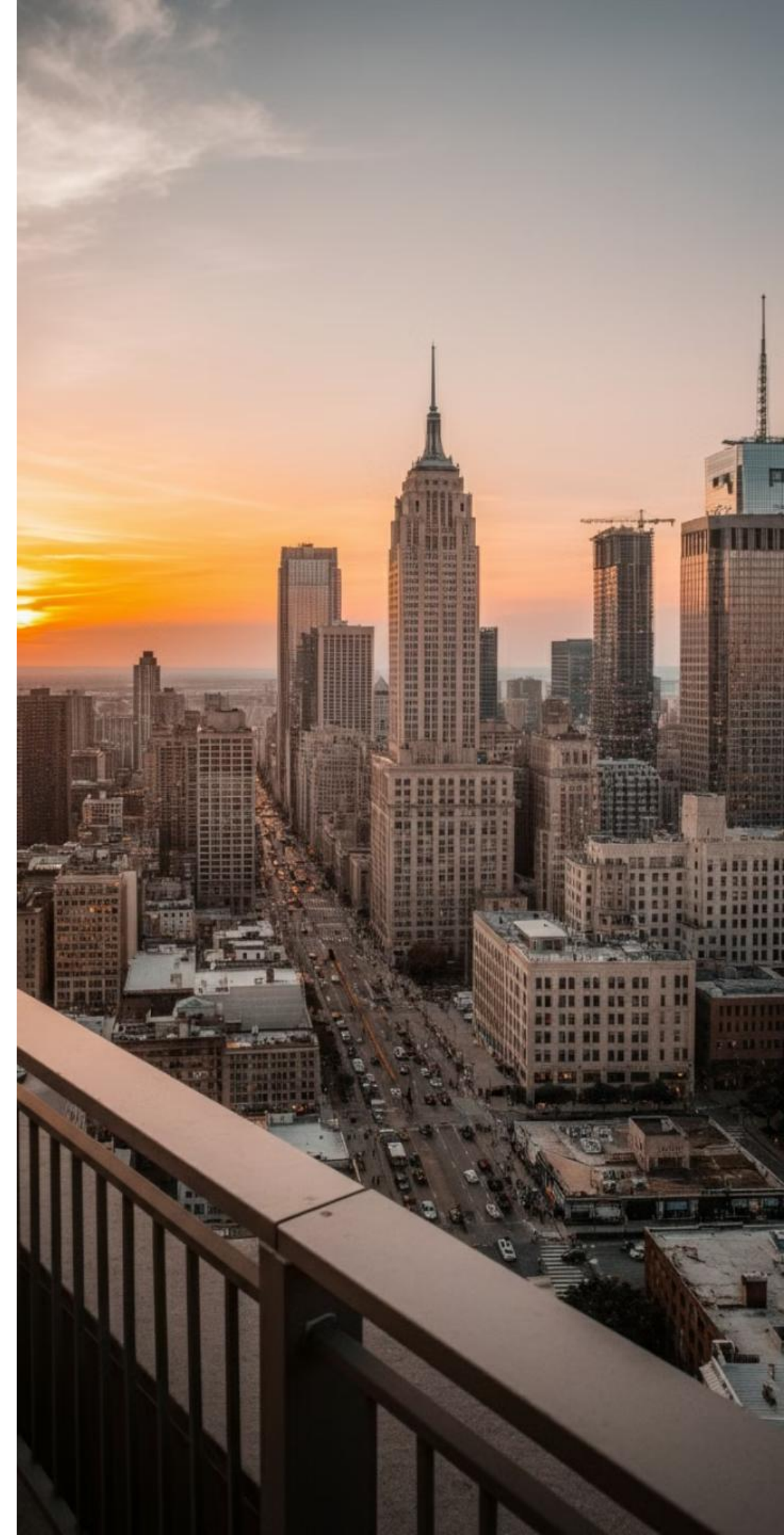
Lectura de XML con DOM

[Ver Actividad 6 en Aula Virtual]

5.3. Binding (JAXB) Vinculando Java y XML

JAXB (Java Architecture for XML Binding) es una API estándar que facilita la vinculación entre clases de Java y documentos XML.

Esta técnica permite convertir automáticamente datos desde un formato a otro, simplificando el trabajo del programador y mejorando la eficiencia en el desarrollo de aplicaciones que manejan datos XML.





- ¿Qué es **Binding**?
- ¿Cual es la diferencia entre **binding** y **parser**?

DATA
STRUCTURES

PROGRAMMING
LANGUAGES

5.3.1. Introducción

¿Qué es el binding y su utilidad en Java?

El binding es una técnica que facilita la vinculación entre estructuras de datos, en este caso, entre clases de Java y documentos XML.

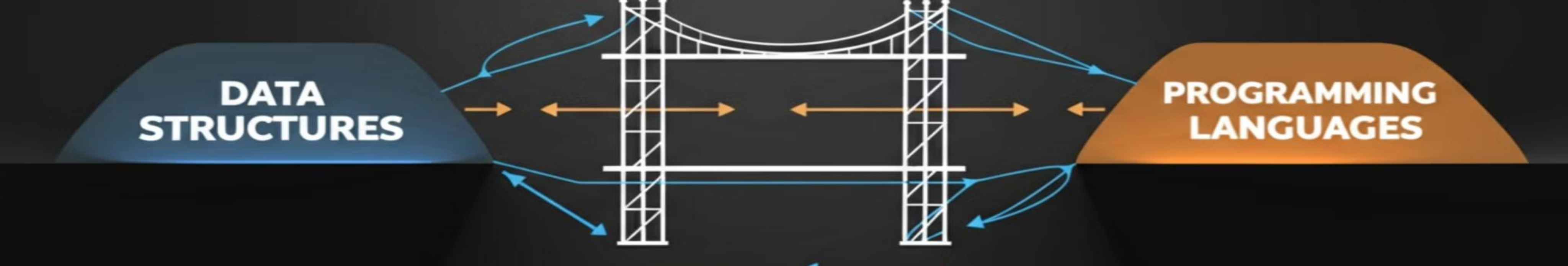
En términos de desarrollo de software

El binding permite que un programa pueda convertir automáticamente datos desde un formato a otro, simplificando así el trabajo del programador.

En el contexto de Java, el binding se utiliza para vincular clases con formatos de almacenamiento específicos, como XML, de modo que un objeto Java puede ser serializado (**marshalled**) a XML o deserializado (**unmarshalled**) desde XML. Este proceso permite almacenar, recuperar y manipular datos de una forma más intuitiva y sencilla.



Figura 6.3

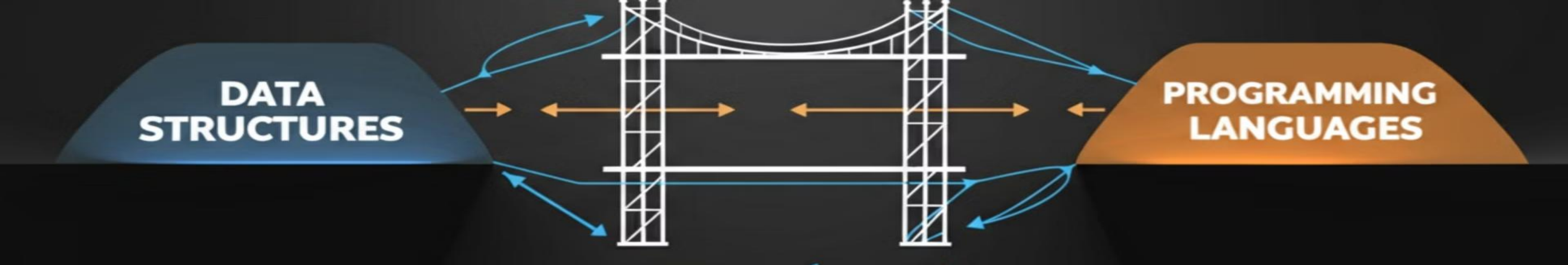


5.3.1. Introducción

■ Diferencia Binding -Parser

PARSER: Un parser, o analizador, es una herramienta o proceso que lee y procesa datos estructurados, como un archivo XML o JSON, y los convierte en un formato que un programa puede manipular. -> Objetivo: interpretar el documento y crear una representación interna.

BINDING: es el proceso de enlazar datos de un documento estructurado (como XML o JSON) directamente a objetos en el código, de manera que puedas manipular estos datos como si fueran clases Java u otros objetos nativos. -> Objetivo: mapear los datos a estructuras de datos en el lenguaje de programación, eliminando la necesidad de trabajar directamente con los nodos de un árbol o los elementos individuales del documento.



5.3.1. Introducción

Diferencia Binding -Parser

	Parser	Binding
Propósito	Leer y analizar un documento estructurado y crear una representación interna.	Mapear datos a objetos nativos del lenguaje, para que se puedan manipular directamente como clases.
Resultado	Una estructura de árbol o una serie de eventos (en caso de SAX) que representa el documento.	Objetos que reflejan la estructura de los datos, con los valores mapeados a campos y atributos de la clase.
Nivel de abstracción	Trabaja a nivel de nodos y elementos; debes recorrer y manipular el documento explícitamente.	Ofrece un nivel más alto de abstracción, permitiendo trabajar con datos directamente como objetos.
Ejemplo de Uso	DOM, SAX, y StAX para XML, JSON Parsers para JSON.	JAXB para XML, Jackson/GSON para JSON.



5.3.1. Introducción

- ¿Qué hace el compilador xjc?
- ¿Qué es XML Schema?
- ¿Qué es Marshalling?
- ¿Qué es Unmarshalling?

5.3.1. Introducción

Conceptos clave

El compilador de binding llamado xjc

Convierte esquemas XML (**.xsd**) en clases Java que representan los elementos y relaciones definidos en el XML. También permite realizar operaciones de marshalling y unmarshalling mediante las clases generadas.

XML Schema

JAXB facilita la creación de clases Java directamente a partir de un esquema XML (**.xsd**) mediante el uso del compilador de binding **xjc**.



¿Cómo se gestiona el binding en Java? JAXB

En Java

la gestión del binding se realiza mediante JAXB (**Java Architecture for XML Binding**), una API estándar que hay que añadirlo como dependencia externa (Jakarta XML Binding y JAXB Runtime)..

JAXB

permite a los desarrolladores mapear clases Java a representaciones XML y viceversa.

Objetivo:

En lugar de escribir manualmente el código para manejar los datos XML, JAXB automatiza el proceso, proporcionando herramientas y anotaciones que definen cómo deben serializarse y deserializarse los objetos.



RESUMEN

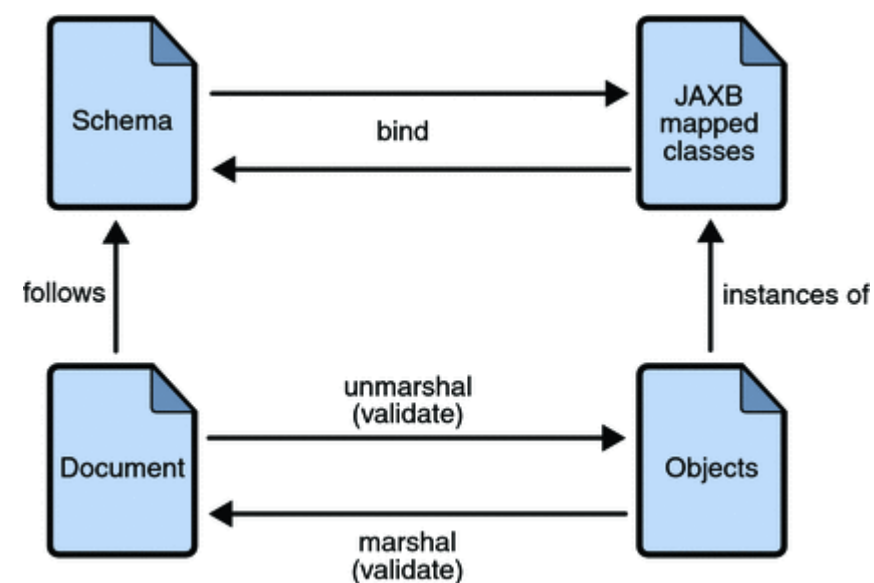
JAXB: Facilita el trabajo con datos XML

Marshalling

La conversión de objetos Java a documentos XML.

■ Unmarshalling

La conversión de documentos XML a objetos Java.



Beneficios de usar JAXB para persistencia de datos en XML

Automatización del mapeo entre objetos Java y XML

JAXB permite al programador centrarse en la lógica de la aplicación sin preocuparse por el formato de almacenamiento.

Compatibilidad con estándares XML

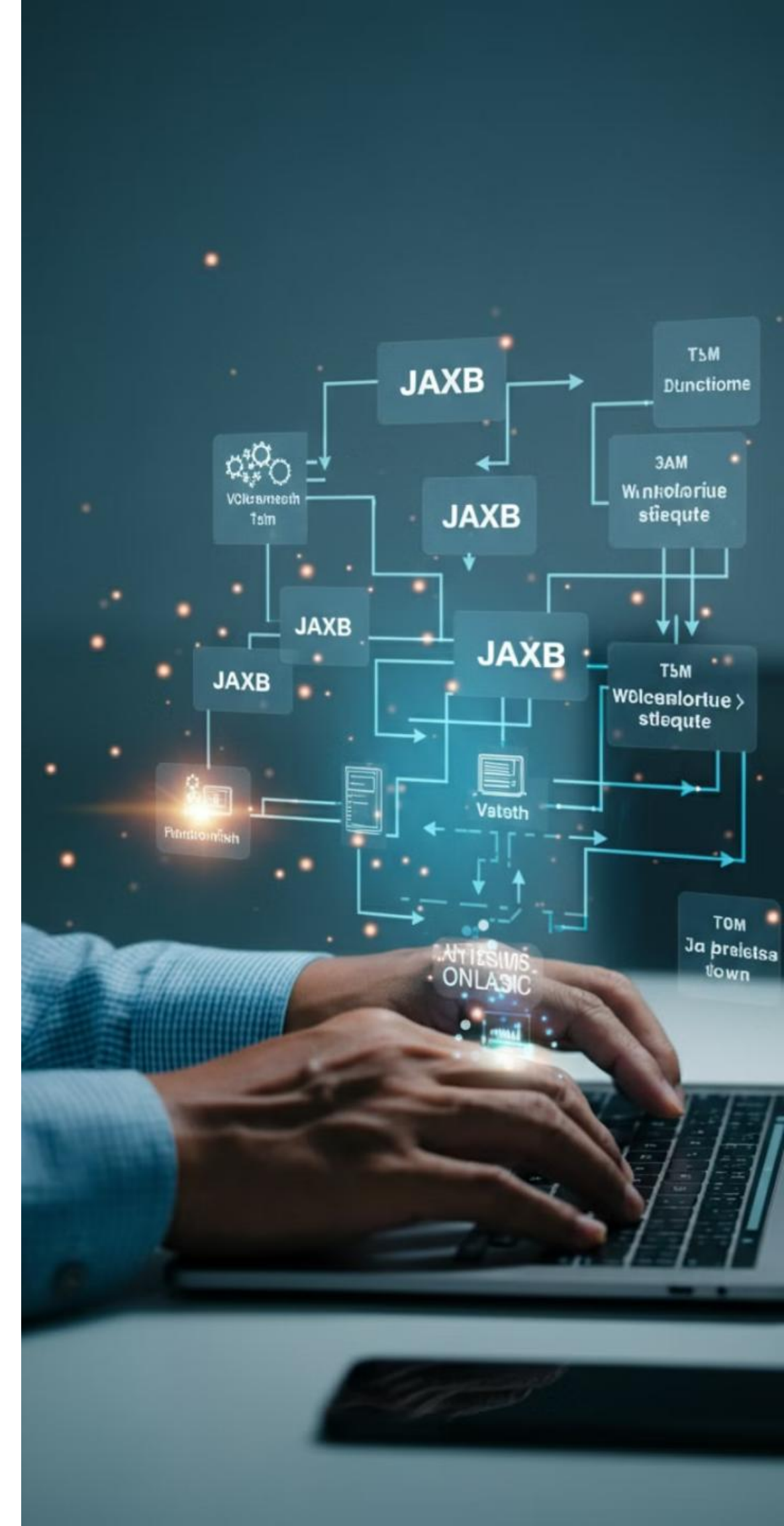
Al usar JAXB, los datos XML generados cumplen con los estándares, facilitando la interoperabilidad con otros sistemas y tecnologías.

Simplicidad en la gestión de datos

JAXB reduce la complejidad en la manipulación de datos XML al eliminar la necesidad de parsers o analizadores XML complejos.

Validación de datos

JAXB ofrece soporte para validar los datos XML contra esquemas, asegurando que los datos procesados cumplan con requisitos específicos.



Resumen funcionamiento JAXB

-  **De manera resumida**
JAXB convierte el contenido de un documento XML en una estructura de clases Java (**Unmarshalling**), o convierte objetos Java a documentos XML (**marshalling**).
-  **El JAXB crea la estructura de clases que albergará el contenido del XML en base a su esquema**
El esquema es la referencia de JAXB para saber la estructura de las clases que contendrán el documento XML.
-  **El documento XML debe tener un esquema XML asociado (fichero.xsd)**
por tanto el contenido del XML debe ser válido con respecto a ese esquema.
-  **Cuando JAXB crea en tiempo de diseño (no durante la ejecución) la estructura de clases**
el proceso de unmarshalling (creación de objetos de las clases creadas con el contenido del XML) y marshalling (almacenaje de los objetos como un documento XML) es sencillo y rápido, y se puede hacer en tiempo de ejecución.

Ejemplos ficheros xsd

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema version="1.0"
  xmlns:xs="http://www.w3.org/2001/XMLSchema"
  elementFormDefault="qualified">
  <xs:element name="clientes">
    <xs:complexType>
      <xs:sequence>
        <xs:element ref="cliente" minOccurs="0" maxOccurs="unbounded" />
      </xs:sequence>
    </xs:complexType>
  </xs:element>
  <xs:element name="cliente">
    <xs:complexType>
      <xs:sequence>
        (...)
      </xs:sequence>
      <xs:attribute name="DNI" use="required" type="tipo_DNI"/>
    </xs:complexType>
  </xs:element>
  (...)
</xs:schema>
```

```
▼<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema" version="1.0">
  ▼<xsd:element name="Libros">
    ▼<xsd:complexType>
      ▼<xsd:choice maxOccurs="unbounded">
        ▼<xsd:element name="Libro" minOccurs="0" maxOccurs="unbounded">
          ▼<xsd:complexType>
            ▼<xsd:sequence>
              <xsd:element name="Titulo" type="xsd:string"/>
              <xsd:element name="Autor" type="xsd:string"/>
            </xsd:sequence>
            <xsd:attribute name="publicado_en" type="xsd:string"/>
          </xsd:complexType>
        </xsd:element>
      </xsd:choice>
    </xsd:complexType>
  </xsd:element>
</xsd:schema>
```

Configuración de JAXB

⚙️ Herramientas y bibliotecas necesarias

Para configurar JAXB en un proyecto de Java, necesitas la API que permite realizar la conversión entre documentos XML y clases de objetos Java, que incluye varias herramientas y bibliotecas.

🔧 Integración en el JDK

- En Java SE 6, 7 y 8 JAXB se integró directamente en el JDK.
- En Java 9 y 10, JAXB estaba en un módulo separado.
- A partir de Java 11, JAXB fue eliminado del JDK. Hay que añadirlo como dependencia externa (**Jakarta XML Binding y JAXB Runtime**).





Configuración de JAXB

La API JAXB está activa fuera del JDK. Bajo el nombre de **Jakarta XML Binding**. Además, también es necesario la implementación (o motor real **JAXB-Runtime**)



Generación de clases Java

Específicamente, para manejar la generación automática de clases Java a partir de un esquema XML, es útil el compilador de binding llamado **xjc**, que convierte esquemas XML (**.xsd**) en clases Java que representan los elementos y relaciones definidos en el XML.



Operaciones de Marshalling y Unmarshalling

También permite realizar operaciones de marshalling y unmarshalling mediante las clases generadas.



Configuración de JAXB

Conceptos fundamentales

API JAXB

API para convertir XML < --- > objetos Java.
Define como se enlazan los datos XML con objetos Java
(Binding).

XJC

Generador automático de las clases que JAXB necesita.

Ejemplo

1º Tenemos el esquema usuario.xsd:

```
<xs:element name="usuario">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="nombre" type="xs:string"/>
      <xs:element name="email" type="xs:string"/>
    </xs:sequence>
    <xs:attribute name="id" type="xs:int" use="required"/>
  </xs:complexType>
</xs:element>
```



2º Ejecutamos:

xjc usuario.xsd -d src -p 'proyectoJava'



3º Genera una clase Java como:

```
@XmlRootElement(name = "usuario")
public class Usuario {
    private int id;
    private String nombre;
    private String email;

    @XmlAttribute
    public int getId() { return id; }
    @XmlElement
    public String getNombre() { return nombre; }
    @XmlElement
    public String getEmail() { return email; }
}
```

¡¡ Esta clase está
lista para usarse
con JAXB

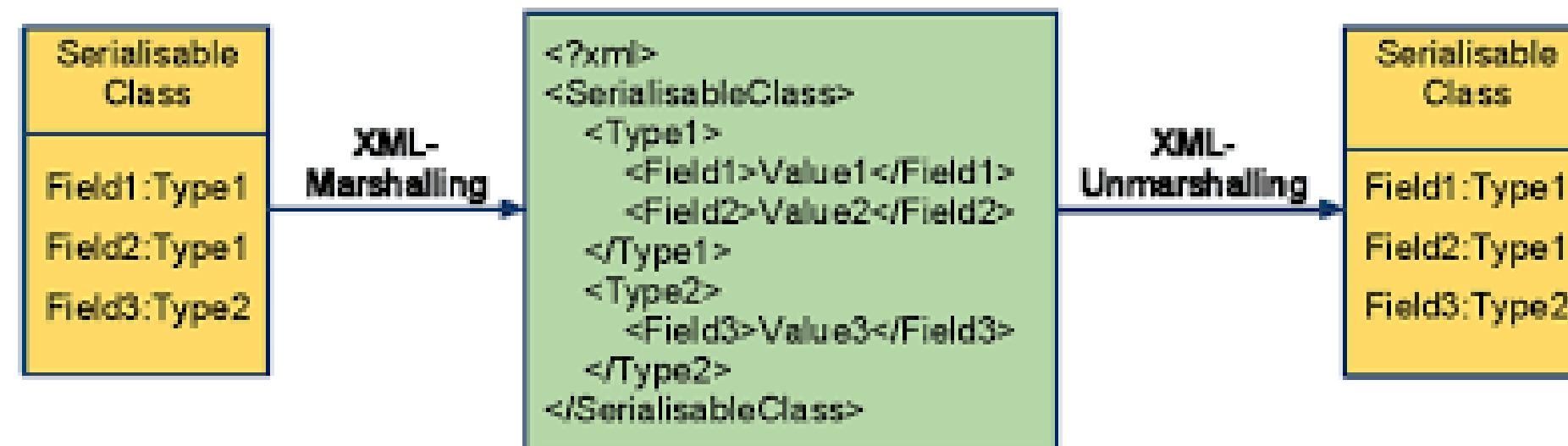
Deserialización y serialización con JAXB

Marshalling (serialización)

La conversión de objetos Java en documentos XML.

Unmarshalling (deserialización)

La conversión de documentos XML a objetos Java.



JAXB permite convertir objetos Java en documentos XML y viceversa, lo que se conoce como marshalling (**serialización**) y unmarshalling (**deserialización**). Estos procesos facilitan la persistencia y recuperación de datos en formato XML.

Generación automática de Clases Java a partir de Esquemas XML (.xsd)

- JAXB facilita la creación de clases Java directamente a partir de un esquema XML (.xsd) mediante el uso del compilador de binding xjc.
- Este proceso permite generar automáticamente las clases necesarias para representar los elementos y tipos definidos en el esquema XML, lo cual es útil cuando trabajamos con documentos XML estructurados y queremos manipularlos en Java.



Uso de la herramienta xjc para generar clases Java (manual)



1. Preparar el esquema XML (.xsd)

Coloca el archivo .xsd en el proyecto, en la ubicación de tu preferencia (por ejemplo, en la carpeta src o una subcarpeta dentro del proyecto en IntelliJ).

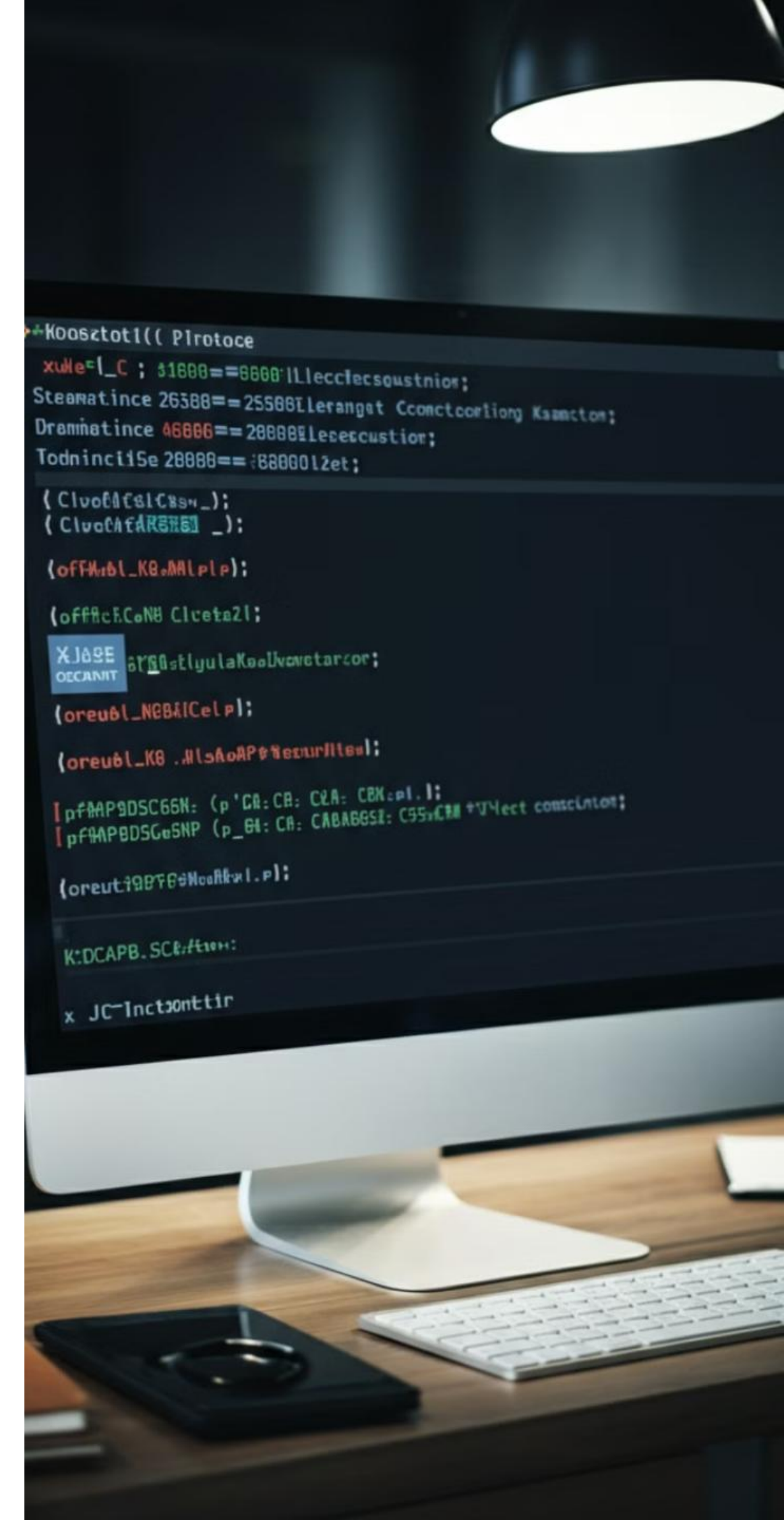


2. Ejecutar xjc desde la línea de comandos

Abre una terminal y navega a la carpeta donde tienes el archivo .xsd. Usa el siguiente comando para generar las clases Java:

```
xjc -d src -p com.ejemplo.paquete archivo.xsd
```

- **-d src**: Especifica el directorio de destino para las clases generadas (en este caso, la carpeta src).
- **-p com.ejemplo.paquete**: Especifica el nombre del paquete para las clases generadas.
- **- archivo.xsd**: Es el nombre del archivo de esquema XML.

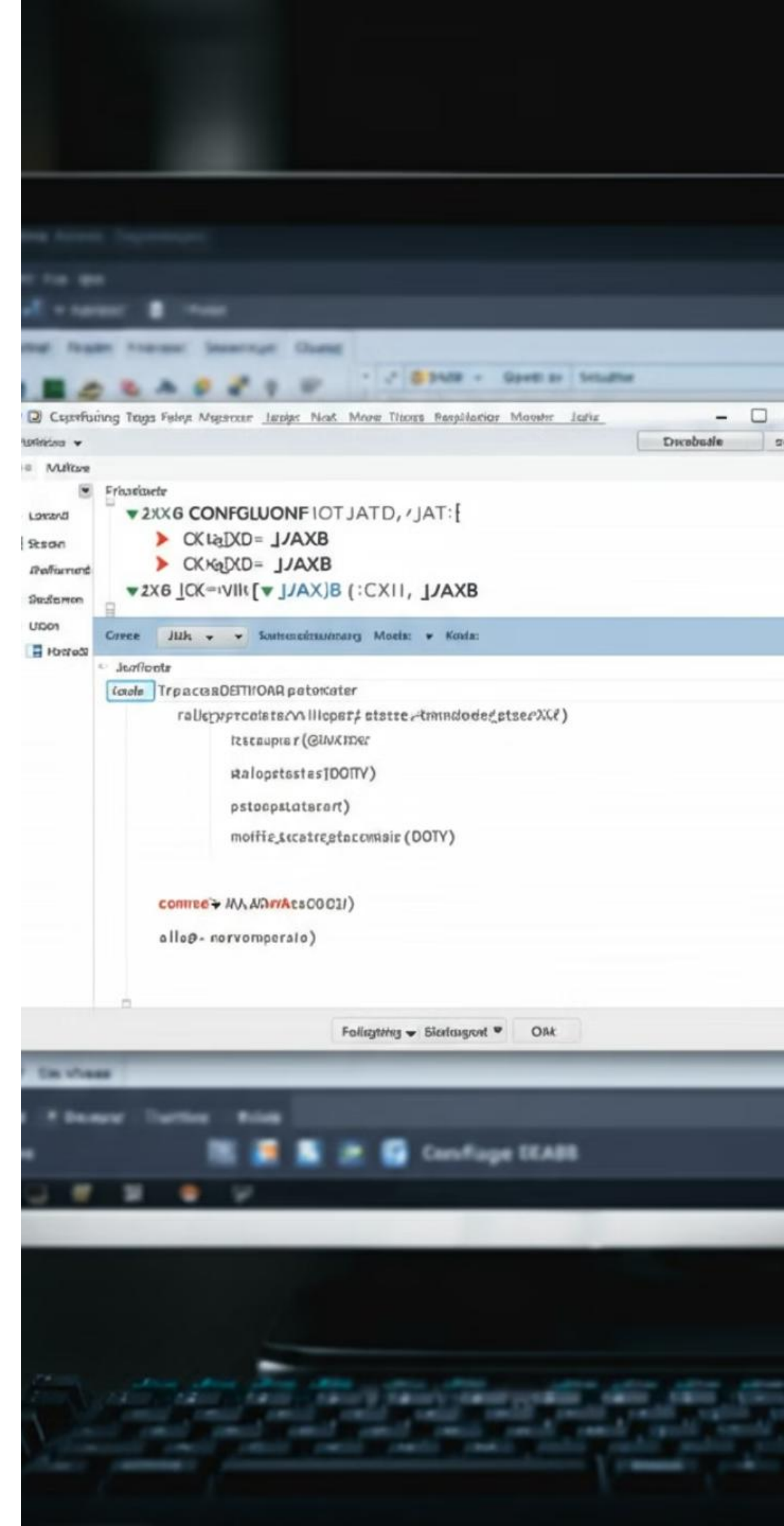


Uso de JAXB para generar clases Java con IntelliJ

1. Instala el plugin Jakarta EE: Web Services (JAX-WS)
2. Crea un proyecto Maven e incluye las dependencias en el POM
3. Abre en el editor de IntelliJ, el Schema (archivo .xsd).
4. En el menu principal, ve a Tools | XML Actions | Generate Java Code From XML Schema Using JAXB.

IntelliJ ejecutará el proceso y generará las clases Java automáticamente en el directorio especificado.

Las clases generadas tendrán métodos get y set para cada uno de los elementos y atributos definidos en el esquema XML.



Resumen de xjc y generación de clases

El compilador xjc

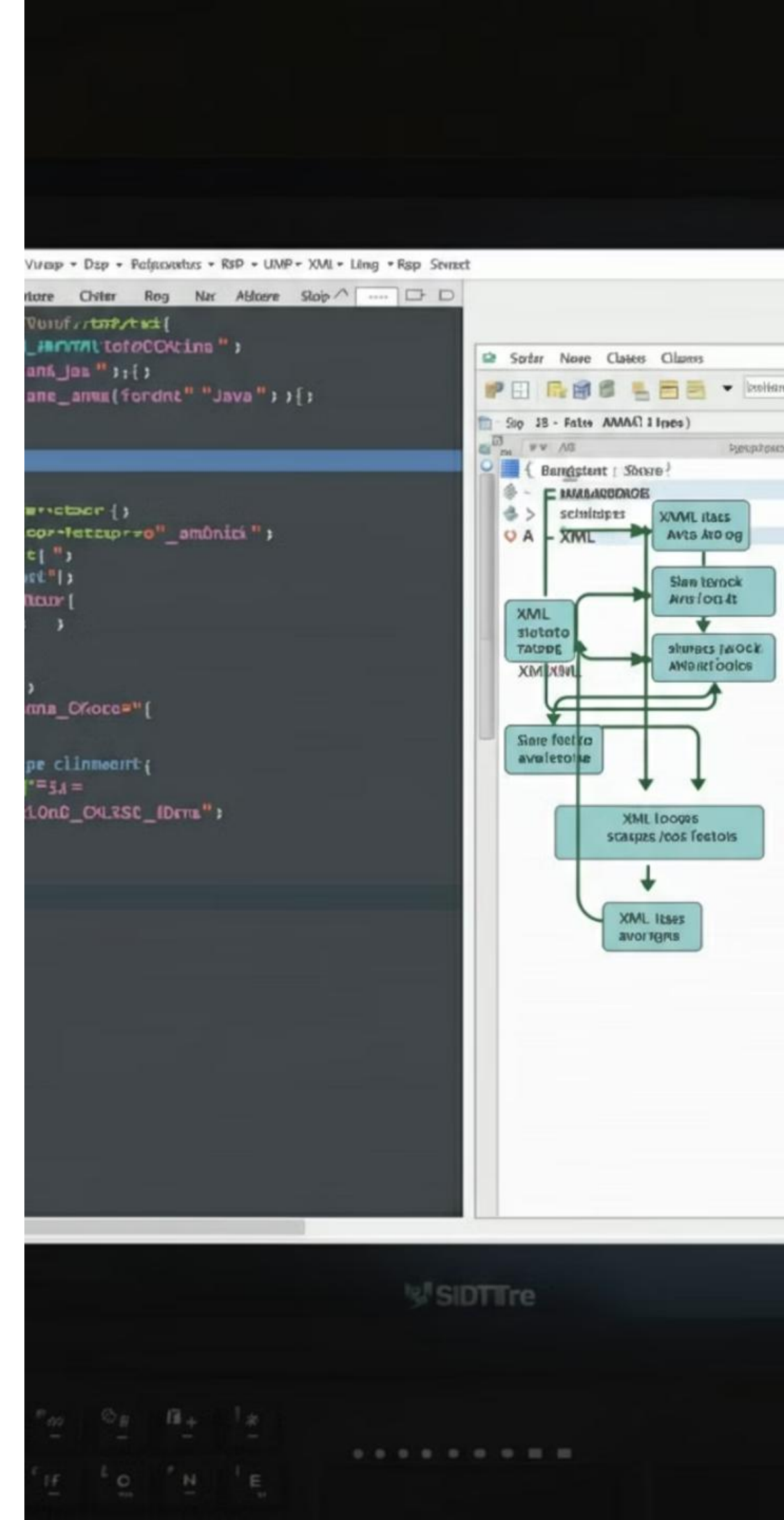
proporciona una forma rápida y automatizada de convertir esquemas XML en clases Java.

Mejora la productividad

al evitar la creación manual de las clases.

Asegura que la estructura de los datos en Java

siga el esquema XML original.



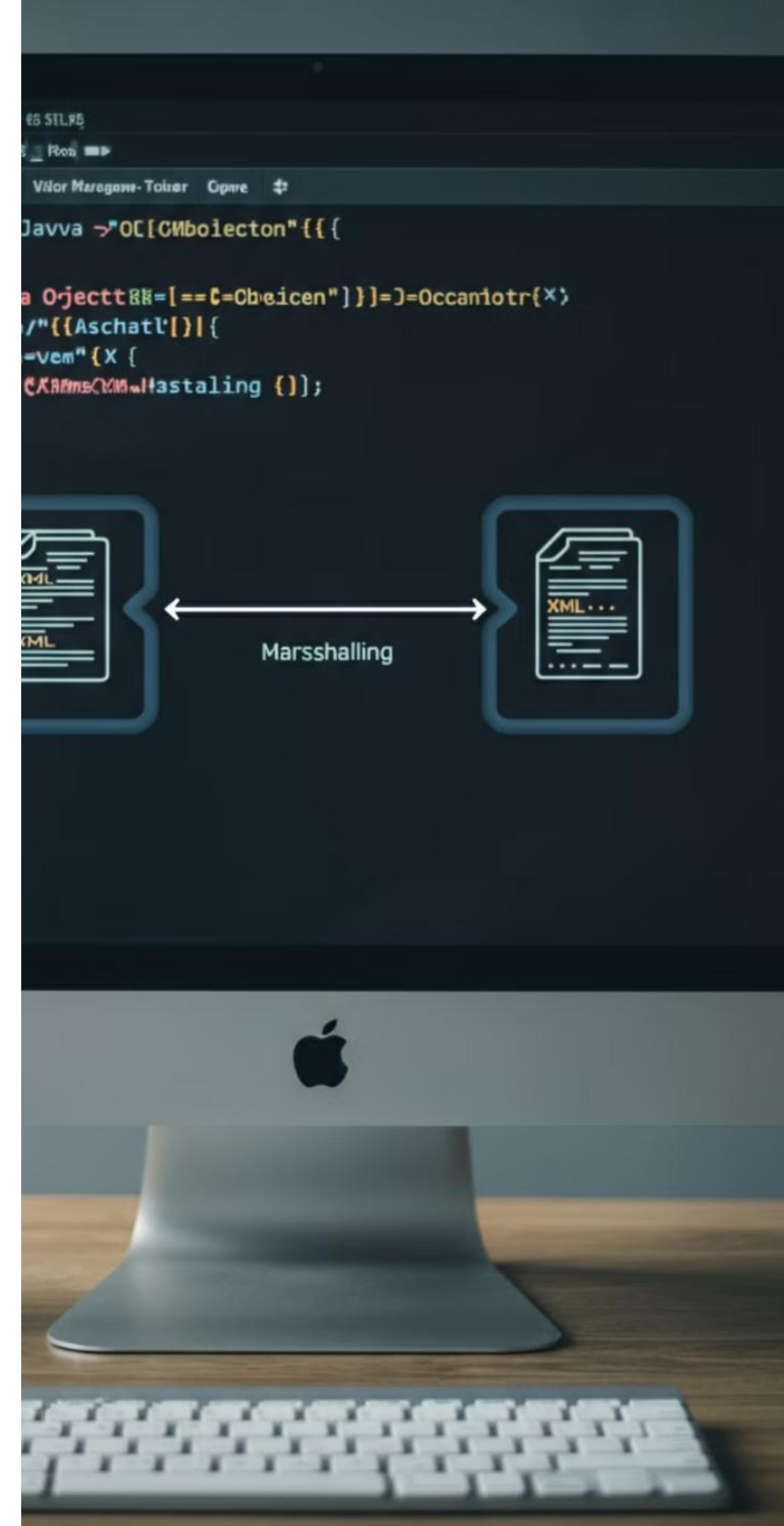
5.3.2. Marshalling: Conversión de objetos Java a XML

El proceso de marshalling

consiste en serializar un objeto Java, es decir, convertirlo en un documento XML.

Con JAXB

el proceso de marshalling es sencillo gracias a la API que proporciona.



Pasos para realizar marshalling

1

1. Crear un contexto JAXB

Se utiliza la clase **JAXBContext**, que representa el entorno de binding para el conjunto de clases que se desea procesar.

```
JAXBContext context = JAXBContext.newInstance(Persona.class);
```

2

2. Crear un marshaller

Este objeto gestiona el proceso de convertir un objeto Java a XML.

```
Marshaller marshaller = context.createMarshaller();
```

3

3. Configurar el marshaller (opcional)

Se puede establecer la propiedad de formato de salida para que el XML se genere de forma legible.

```
marshaller.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);
```

4

4. Serializar el objeto

Utiliza el método **marshal** para escribir el objeto en un archivo, en la consola o en un OutputStream.

```
marshaller.marshal(persona, System.out);
```



Actividad 7

RA1_A7

Ejemplo de marshalling

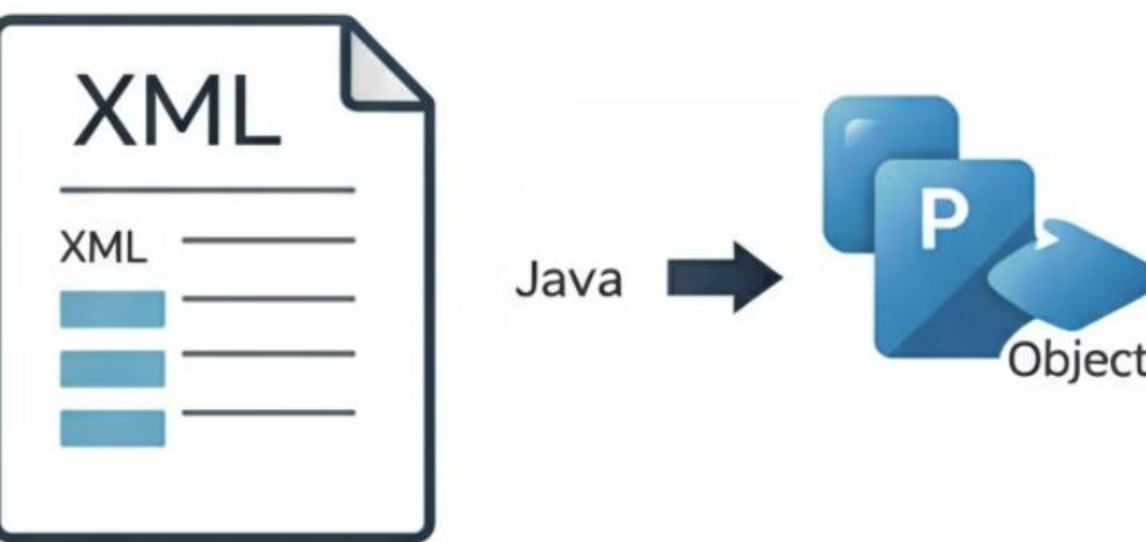
Ver actividad 7 en el aula virtual



5.3.3. Unmarshalling: Conversión de XML a objetos Java

Unmarshalling

psoum Denatent



Unmarshalling

El unmarshalling

es el proceso inverso al marshalling. Consiste en deserializar un documento XML y convertirlo en un objeto Java, lo cual permite trabajar con datos XML de manera nativa en Java.

Pasos para realizar unmarshalling

1. **Crear un contexto JAXB:** Similar al marshalling, primero se configura el JAXBContext.

1

```
JAXBContext context = JAXBContext.newInstance(Persona.class);
```

2. Crear un unmarshaller

Este objeto gestiona el proceso de convertir XML a objetos Java.

2

```
Unmarshaller unmarshaller = context.createUnmarshaller();
```



3

3. Deserializar el archivo XML

Se utiliza el método `unmarshal` para leer el XML y convertirlo en una instancia de la clase Java.

```
Persona persona = (Persona) unmarshaller.unmarshal(new  
File("persona.xml"));
```

Actividad 8

RA1_A8

Ejemplo de unmarshalling

Ver actividad 8 en el aula virtual



Resumen de marshalling y unmarshalling



- ☐ **Marshalling**
convierte un objeto Java en un archivo XML.
- ☐ **Unmarshaller**
convierte un archivo XML en un objeto Java.
- ☐ Estos procesos son útiles para aplicaciones que necesitan persistir datos o intercambiar información en formato XML.
- ☐ Facilitan la integración con sistemas externos que utilizan XML para la transferencia de datos.

Resumen de marshalling y unmarshalling

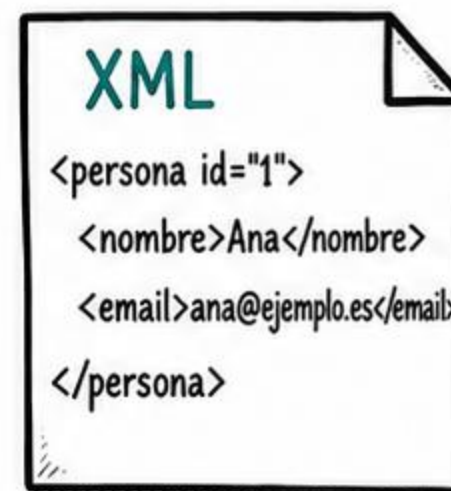
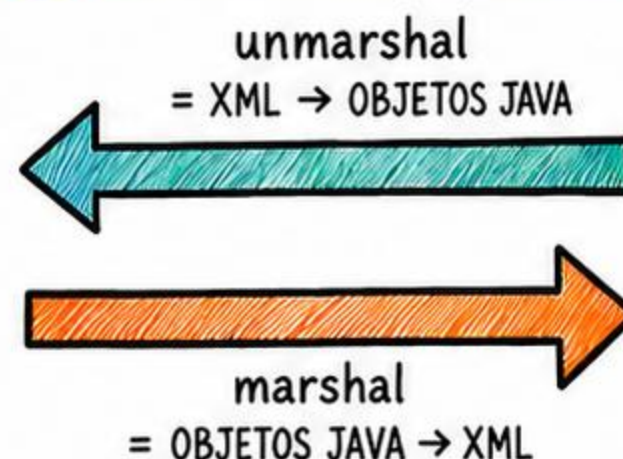


- **Estos procesos**
son útiles para aplicaciones que necesitan persistir datos o intercambiar información en formato XML.
- **Facilitan la integración**
con sistemas externos que utilizan XML para la transferencia de datos.

JAXB: VINCULANDO JAVA Y XML

BINDING = MAPEAR DATOS XML A OBJETOS JAVA (Y VICEVERSA)

1 LA IDEA CLAVE



2 ¿PARSER O BINDING?



EL PARSER INTERPRETA EL DOCUMENTO

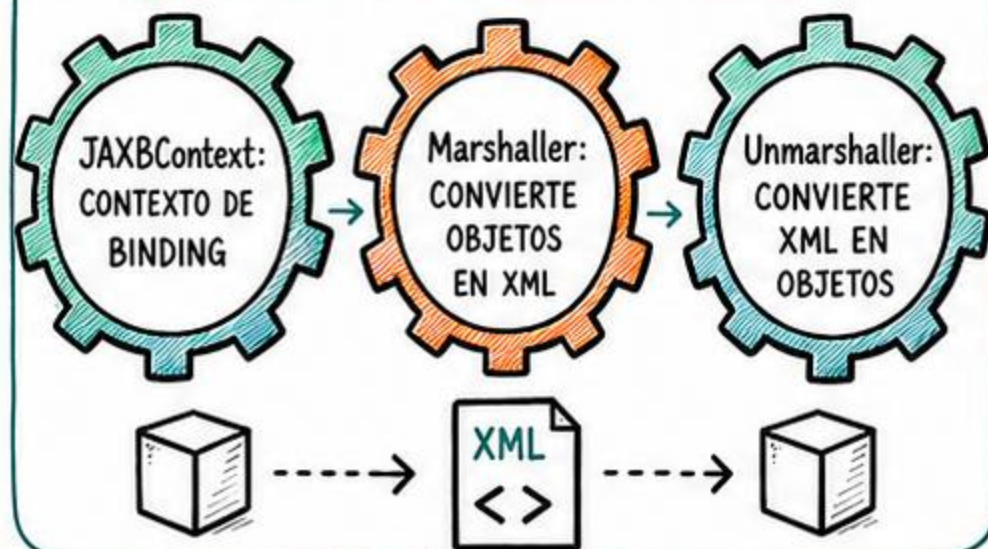


EL BINDING MAPEA SU CONTENIDO A CLASES

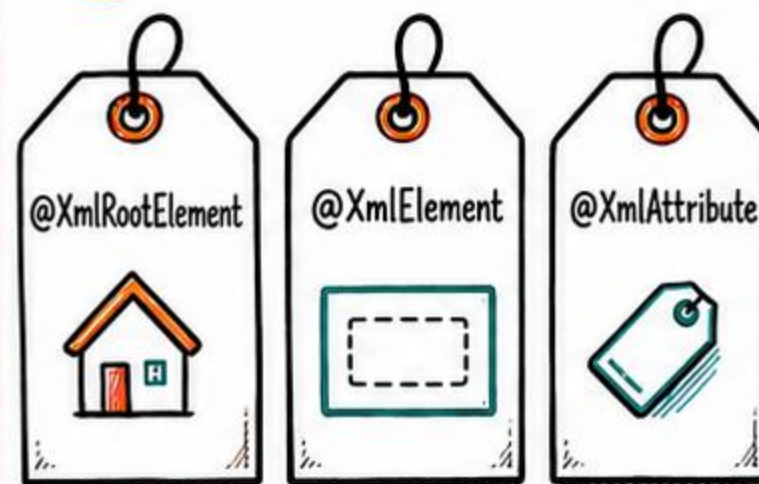


MÁS ABSTRCCIÓN: TRABAJAS CON OBJETOS, NO CON NODOS

3 PIEZAS DE LA API



4 ANOTACIONES ÚTILES



5 GENERAR CLASES DESDE UN XSD



DESDE JAVA 11 JAXB NO VIENE EN EL JDK:
SE AÑADE COMO DEPENDENCIA (JAKARTA XML BINDING)



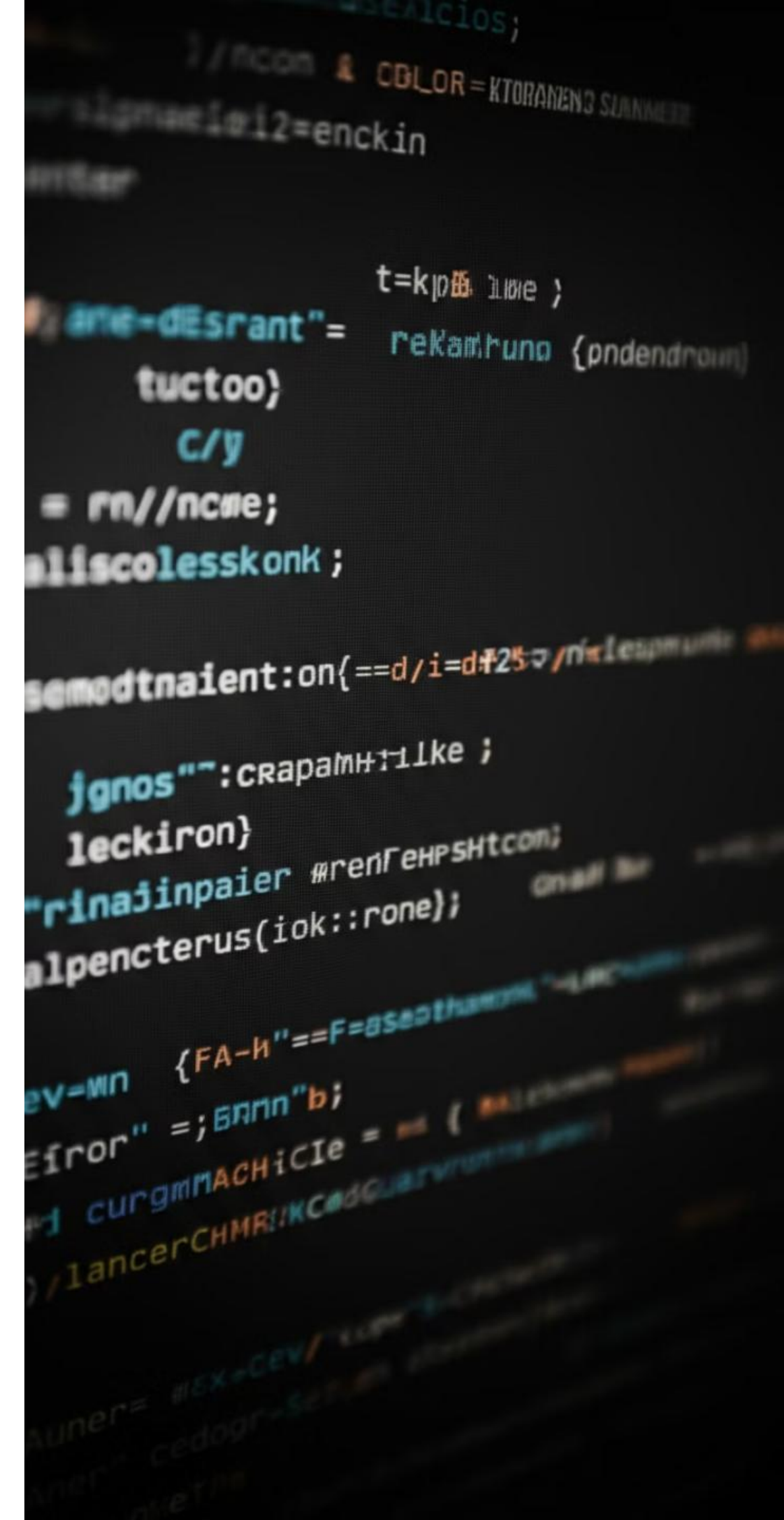
MANIPULA EL XML COMO SI FUERAN
OBJETOS NORMALES DE TU APLICACIÓN

Manejo de Excepciones y Validación de Datos en JAXB

Al trabajar con JAXB

en los procesos de marshalling (**serialización**) y unmarshalling (**deserialización**), es fundamental prever y gestionar correctamente las posibles excepciones y errores que pueden surgir.

Asimismo, es importante realizar validaciones para asegurar que los datos XML cumplen con los esquemas y los requisitos definidos en el sistema.



Principales excepciones en procesos de marshalling y unmarshalling

Los procesos de marshalling y unmarshalling

pueden generar diferentes tipos de excepciones. A continuación, se describen las más comunes:

1. JAXBException

- Es la excepción genérica en JAXB, la cual puede ser lanzada en diversas situaciones, como errores durante el proceso de conversión de objetos Java a XML o viceversa.

- Ejemplo de manejo:

```
try {  
    JAXBContext context = JAXBContext.newInstance(Persona.class);  
    Marshaller marshaller = context.createMarshaller();  
    marshaller.marshal(persona, System.out);  
} catch (JAXBException e) {  
    e.printStackTrace();  
}
```

2. MarshalException

Esta excepción específica ocurre cuando hay un problema durante el proceso de **marshalling** (conversión de objeto Java a XML). Puede ser causada por datos inválidos o incompatibles en el objeto que se intenta serializar.

Manejo típico

```
try {  
    marshaller.marshal(persona, System.out);  
} catch (MarshalException e) {  
    System.out.println("Error durante la serialización: " + e.getMessage());  
}
```



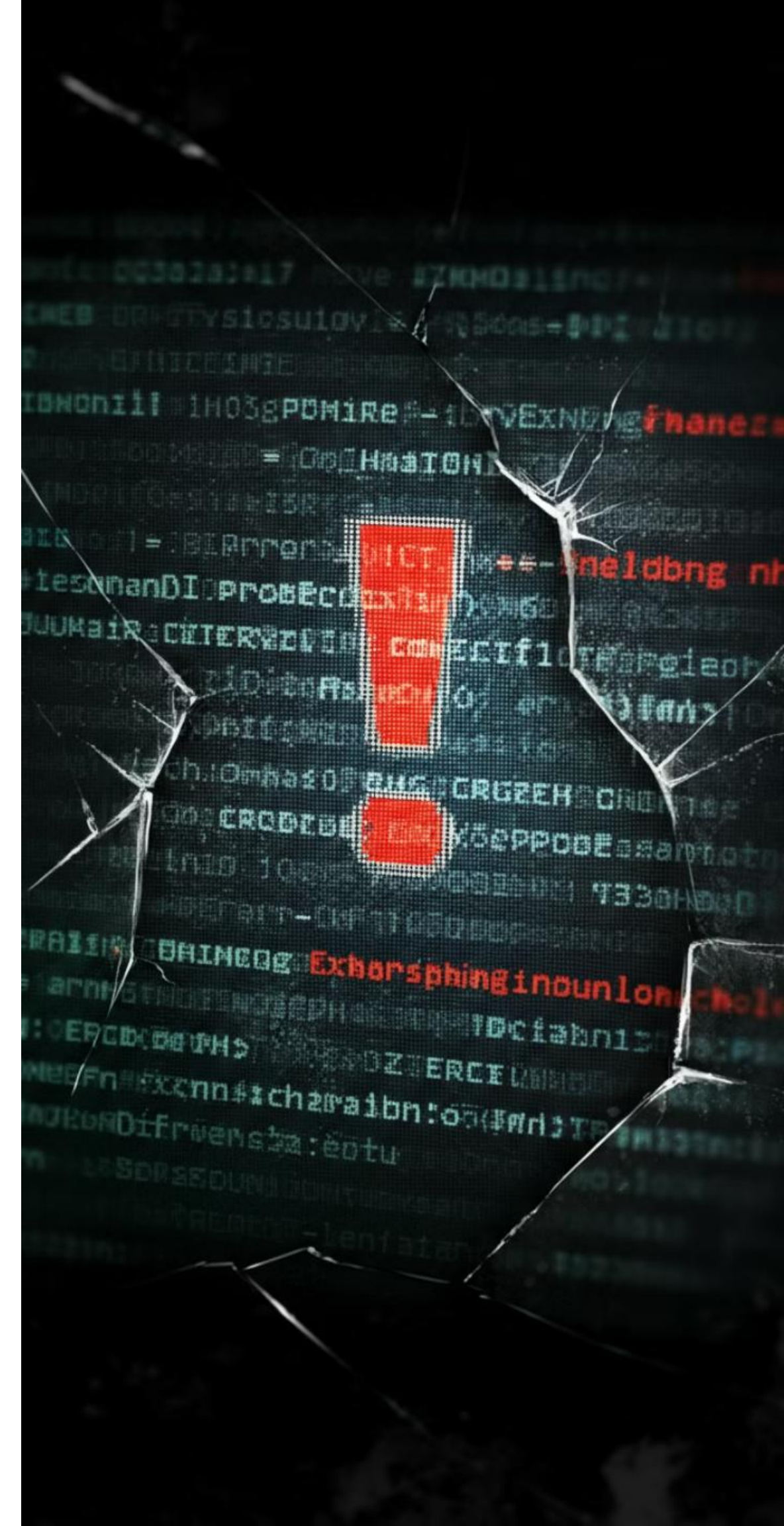
3. UnmarshalException

Ocurre durante el proceso de unmarshalling (conversión de XML a objeto Java)

Esta excepción se lanza si el documento XML tiene problemas de formato o si no cumple con el esquema definido.

Ejemplo de manejo

```
try {  
    Unmarshaller unmarshaller = context.createUnmarshaller();  
    Persona persona = (Persona) unmarshaller.unmarshal(new File("persona.xml"));  
} catch (UnmarshalException e) {  
    System.out.println("Error durante la deserialización: " + e.getMessage());  
}
```



Validación de datos con JAXB

Para asegurarse de que los datos en el archivo XML son válidos y cumplen con un esquema, JAXB permite realizar validaciones al deserializar (**unmarshal**) documentos XML. Esta validación garantiza que el XML no solo tiene la estructura correcta sino que también cumple con las reglas de datos especificadas en el esquema.

Crear un SchemaFactory

1. Crear un SchemaFactory para el tipo de esquema XML.

```
SchemaFactory sf =  
    SchemaFactory.newInstance(XMLConstants.W3C_XML_SCHEMA_NS_URI);
```

Cargar el esquema

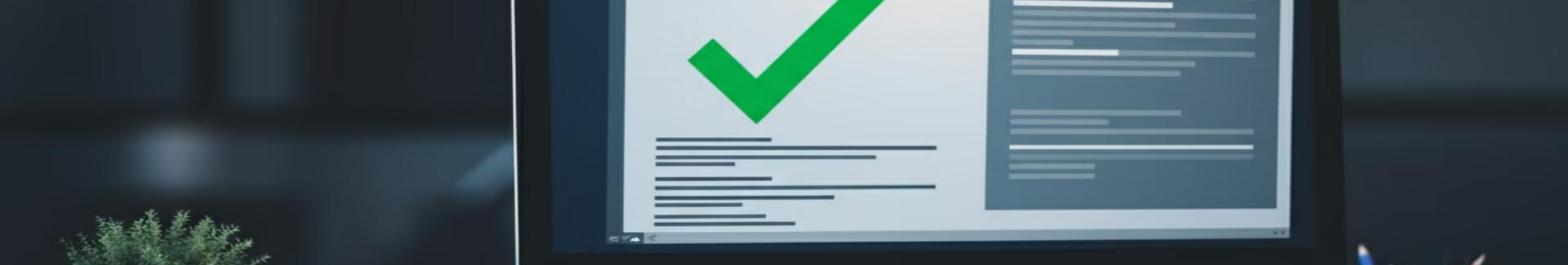
2. Cargar el esquema desde un archivo .xsd y aplicarlo al unmarshaller:

```
Schema schema = sf.newSchema(new File("persona.xsd"));  
  
Unmarshaller unmarshaller = context.createUnmarshaller();  
  
unmarshaller.setSchema(schema);
```

Establecer un manejador de errores

3. Establecer un manejador de errores: Permite personalizar la respuesta ante errores o advertencias en la validación del esquema.

```
unmarshaller.setEventHandler(new ValidationEventHandler() {  
  
    public boolean handleEvent(ValidationEvent event) {  
  
        System.out.println("Evento de validación: " + event.getMessage());  
  
        return true;  
  
    }  
  
});
```



Resumen excepciones

Gestionar excepciones

y realizar validaciones con JAXB permite que las aplicaciones manejen correctamente datos XML incluso cuando surgen problemas de formato o datos inválidos.

Estas prácticas aseguran que los datos se manejen de manera robusta y conforme a las expectativas de la aplicación.

An abstract graphic on the left side of the slide. It features a dark blue background with numerous thin, glowing lines in various colors (blue, green, yellow, orange, red, purple) that curve and intersect. Small, bright dots of corresponding colors are scattered along these lines, creating a sense of dynamic movement and data flow.

5.4. Conversión de Ficheros XML a Otros Formatos

La conversión de XML a otros formatos es esencial en el desarrollo de aplicaciones, ya que permite **adaptar los datos a diferentes sistemas y necesidades**.

XML es un formato muy utilizado para la estructura y almacenamiento de datos, pero otros formatos, como **JSON, CSV o HTML**, son más adecuados para tareas específicas, como el intercambio de datos entre aplicaciones web, el análisis en herramientas de hoja de cálculo, o la presentación en un navegador.



¿Qué es XSL y XSLT?

¿Para qué se utilizan?



XSL

XSL (eXtensible Stylesheet Language)

XSL (eXtensible Stylesheet Language) es un **lenguaje de transformación que permite generar nuevos documentos a partir de un documento XML.**

Originalmente, XSL se diseñó para separar el contenido de la presentación, permitiendo que un mismo conjunto de datos en XML pueda transformarse y visualizarse de diferentes maneras según el formato de salida deseado.



XSLT (XSL Transformations)

Función de XSLT

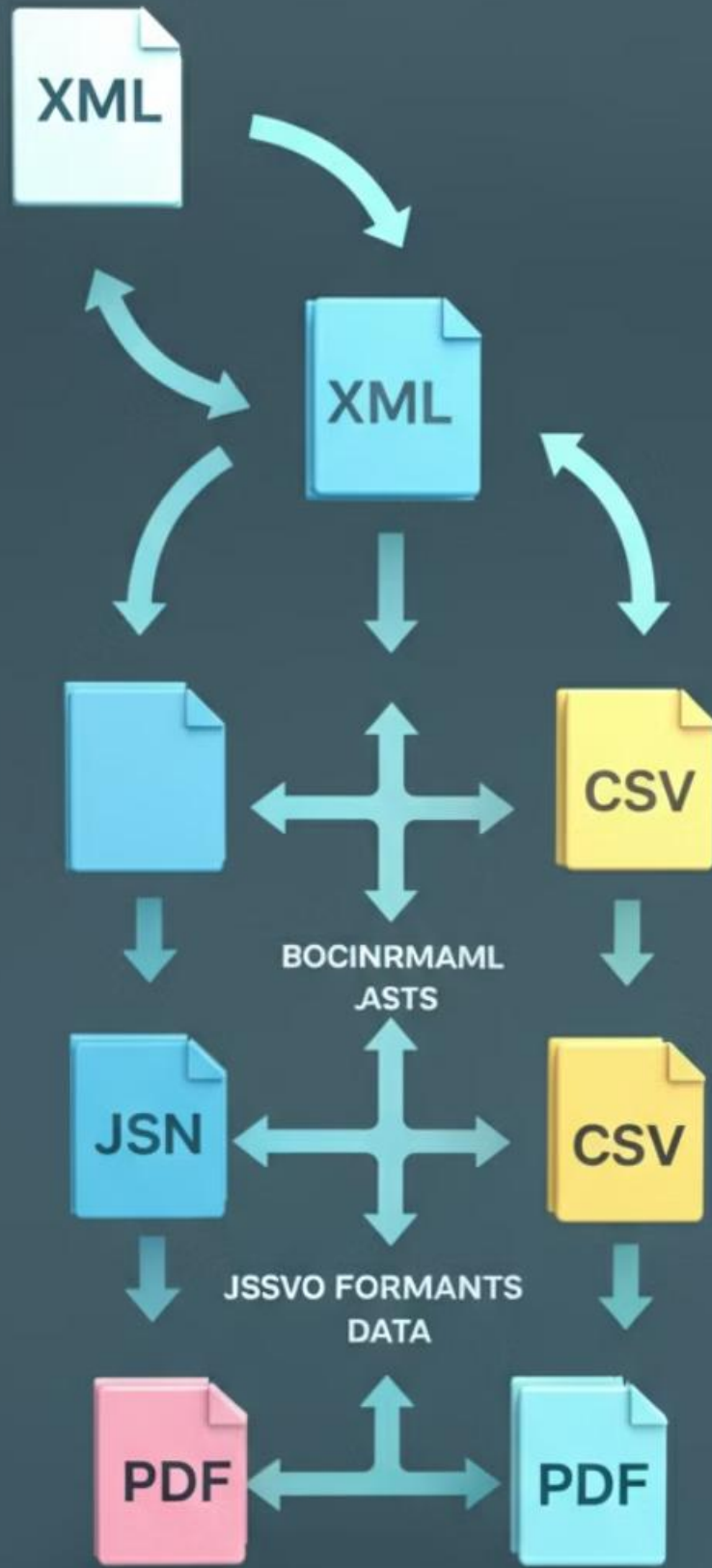
XSLT (XSL Transformations) es la parte de XSL encargada de realizar la transformación de un documento XML a otros formatos, como HTML, CSV o incluso otro XML.

Plantillas de Transformación

XSLT utiliza un conjunto de **reglas de transformación**, llamadas **plantillas**, para definir cómo los elementos y atributos del XML deben ser mapeados y organizados en el documento de salida.

Este proceso facilita que los datos XML puedan ser reutilizados y adaptados a diferentes dispositivos o contextos, desde aplicaciones web hasta sistemas de gestión de datos.

¿POR QUÉ SE UTILIZA XSLT PARA TRANSFORMAR FICHEROS XML?



Versatilidad

XSLT puede generar documentos en distintos formatos de texto, como HTML para páginas web o CSV para análisis de datos -> los datos XML se adaptan a múltiples usos sin necesidad de reestructurar el contenido original.

Control Detallado

XSLT proporciona funcionalidades avanzadas, como la posibilidad de definir variables, aplicar condiciones, crear bucles, y controlar el formato del documento de salida. Esto es útil para generar archivos con estructuras de datos complejas o personalizadas.

TRANSFORMACIÓN XSLT

¿Que implica la transformación de un archivo XML con XSLT? A continuación, se describen los pasos principales del proceso:

1

1. Definir la plantilla XSLT

Crear un archivo XSLT que contenga las reglas de transformación. Estas reglas, llamadas plantillas, especifican cómo cada elemento y atributo del XML debe ser procesado y estructurado en el archivo de salida. La plantilla XSLT también incluye instrucciones sobre el formato de salida (por ejemplo, html, text o xml).

2

2. Aplicar la plantilla al documento XML

Se utiliza un procesador XSLT (como el integrado en Java o en navegadores web) para aplicar la plantilla al archivo XML. Este procesador lee las instrucciones XSLT, selecciona los nodos XML relevantes usando XPath y aplica las transformaciones definidas.

3

3. Generar el archivo de salida

El procesador XSLT produce un nuevo documento en el formato especificado por la plantilla. Este archivo de salida puede ser HTML para una página web, CSV para análisis de datos o cualquier otro formato de texto según la configuración de la transformación.

APLICACIÓN EN JAVA

Clases principales

Java proporciona las clases necesarias para realizar transformaciones XSLT dentro del paquete **javax.xml.transform**:

TransformerFactory

Crea una instancia de un transformador XSLT.

Transformer

Representa el transformador en sí, que aplica una plantilla XSLT a un archivo XML.

Source y Result

Estas interfaces representan el archivo XML de entrada y el archivo de salida, respectivamente. Java utiliza subclases como `StreamSource` para XML y XSLT, y `StreamResult` para la salida (que puede ser un archivo, un `OutputStream`, etc.).



Configuración de la transformación

El proceso para realizar una transformación XSLT en Java generalmente sigue estos pasos:

1

Cargar el archivo XSLT

Utiliza TransformerFactory para cargar la plantilla XSLT desde un archivo o recurso.

2

Configurar el transformador

Crea un objeto Transformer y configura cualquier propiedad adicional si es necesario (por ejemplo, ajustes de salida).

3

Aplicar la transformación

Usa el método `transform()` del objeto `Transformer`, pasando como parámetros el XML de entrada y el resultado de salida.

[illegible]

6. Manejo de Excepciones en Java



¿Qué son las excepciones?

Eventos que interrumpen el flujo normal de un programa y requieren un manejo adecuado.

Gestión de excepciones

Java proporciona un mecanismo estructurado basado en la clase **Exception** y sus subclases.

Bloques try-catch

Permiten capturar y manejar excepciones para evitar la interrupción del programa.



6.1. Introducción

¿Qué son las Excepciones?

Definición de Excepción

Una excepción es **un evento** que ocurre durante la ejecución de un programa y que **interrumpe su flujo normal**.

Origen de las Excepciones

Las excepciones surgen cuando el programa se encuentra con **situaciones inesperadas**, como intentar dividir por cero, acceder a un archivo que no existe o manipular un objeto nulo.

¿Cómo gestiona Java las excepciones?

Mecanismo estructurado

Java proporciona un mecanismo estructurado para el manejo de excepciones, basado en la clase **Exception** y su jerarquía de subclases.

Creación de objetos de excepción

Cuando ocurre un error en tiempo de ejecución, se crea un objeto de excepción que contiene detalles sobre el error.

Propagación de excepciones

El objeto de excepción se lanza hacia la pila de métodos hasta que alguien lo maneje adecuadamente.



Ejemplo básico de excepción

Excepción `ArithmeticException`

Supongamos que se intenta dividir un número por cero, lo cual generará una excepción de tipo **`ArithmeticException`**.

```
public class EjemploExcepcion {  
    public static void main(String[] args) {  
        int resultado = 10 / 0; // Esto provocará una ArithmeticException  
        System.out.println("El resultado es: " + resultado);  
    }  
}
```

Interrupción del programa

En este caso, el programa se detendría inmediatamente y mostraría un mensaje de error indicando que ha ocurrido una **`ArithmeticException`**.

Excepciones: objetos en Java

Excepciones como Objetos

Las excepciones en Java son objetos que pertenecen a una clase que hereda de **Exception** o de alguna de sus subclases.

Bloques try-catch

Java utiliza bloques especiales como **try-catch** (y **finally**) para permitir la captura, el tratamiento y la liberación de recursos al manejar excepciones.

Captura y gestion (try-catch)

Estos bloques permiten a los programadores **controlar el flujo** del programa cuando ocurre un error y reaccionar de manera adecuada **sin interrumpir la ejecución**.



Importancia de la gestión de excepciones

Prevenir errores críticos

Permite evitar que errores interrumpan la ejecución normal del programa.

Mejorar la robustez

Maneja fallos inesperados de forma controlada, aumentando la fiabilidad.

Facilitar la depuración

Captura y maneja excepciones, proporcionando mensajes útiles para la resolución de problemas.





6.2. Captura y gestión de excepciones

Cuando una excepción no se gestiona adecuadamente, el programa se detiene bruscamente y muestra un mensaje de error.

Detención Abrupta del Programa

Java permite a los programadores capturar y gestionar excepciones utilizando **bloques try-catch**, controlando el flujo del programa cuando ocurre un error.

Bloques try-catch

Los bloques try-catch permiten reaccionar de manera apropiada a las excepciones sin interrumpir la ejecución del programa.

Reacción Adecuada

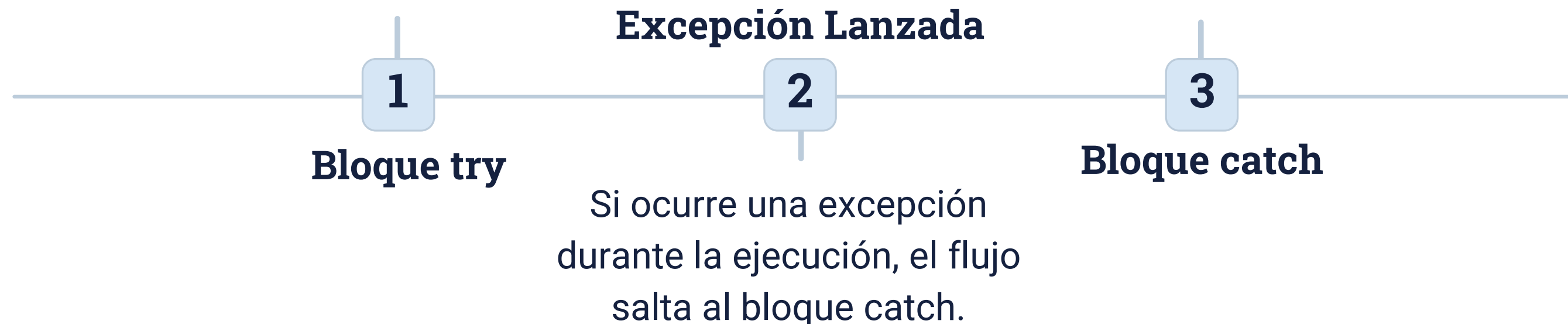


Estructura del bloque “try-catch”

```
try {  
    // Código que puede lanzar una excepción  
} catch (TipoDeExcepcion e) {  
    // Código para gestionar la excepción  
}
```

Contiene el código que puede generar una excepción.

Captura y gestiona la excepción especificada entre paréntesis.



Ejemplo de captura de excepciones

Captura de `ArithmeticException`

El siguiente ejemplo muestra cómo capturar una **`ArithmeticException`** cuando se intenta dividir por cero. El programa continúa su ejecución normal después de mostrar un mensaje de error.

```
public class EjemploTryCatch {  
    public static void main(String[] args) {  
        try {  
            int resultado = 10 / 0; // Esto provocará una ArithmeticException  
            System.out.println("El resultado es: " + resultado);  
        } catch (ArithmeticException e) {  
            System.err.println("Error: No se puede dividir por cero.");  
        }  
        System.out.println("Fin del programa.");  
    }  
}
```

Salida del programa

```
Error: No se puede dividir por cero.  
Fin del programa.
```

La excepción se captura dentro del bloque `catch` y se muestra un mensaje de error, evitando que el programa se detenga.

Recomendación:

Utilizar **`System.err`** en lugar de **`System.out`** para mostrar los mensajes de error, ya que **`System.err`** está específicamente destinado para este propósito.

Ventajas de capturar excepciones

Evita interrupciones

Al capturar una excepción, el programa puede continuar su ejecución después de gestionar el error, evitando que se detenga abruptamente.

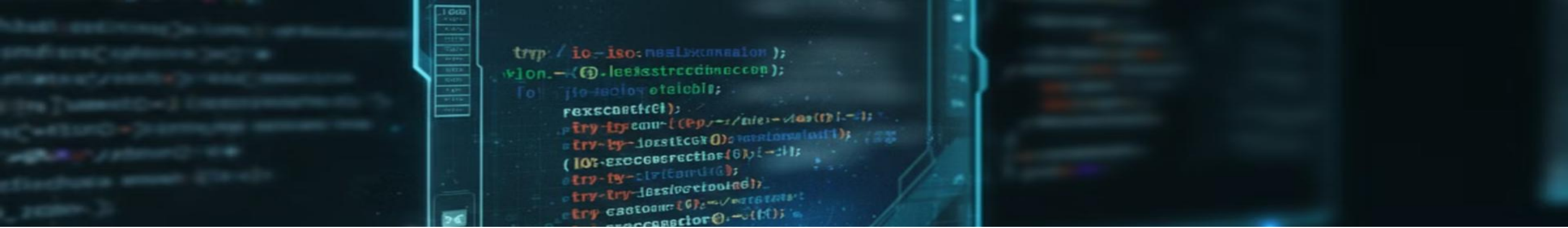
Mejora la depuración

Los bloques `try-catch` permiten identificar y manejar errores en secciones específicas del código, facilitando la resolución de problemas.

Proporciona información detallada

Java ofrece métodos como `getMessage()` y `printStackTrace()` en los objetos de excepción para obtener información valiosa sobre la causa del error.





6.3. Gestión de múltiples excepciones

Múltiples bloques catch

Permite **capturar y manejar diferentes tipos de excepciones** de manera específica, mejorando el control sobre el flujo del programa.

Jerarquía de excepciones

Es importante tener en cuenta la jerarquía de excepciones para evitar que un bloque catch general intercepte una excepción antes de que se maneje por bloques más específicos.



Captura específica

Cada bloque catch se especializa en capturar un tipo particular de excepción.



Gestión selectiva

Si una excepción de un tipo específico se lanza, solo el bloque catch correspondiente la gestionará.

Ejemplo: Gestión de múltiples excepciones

```
public class EjemploMultiplesCatch {  
    public static void main(String[] args) {  
        try {  
            int[] numeros = {1, 2, 3};  
            System.out.println(numeros[3]); // Esto provocará ArrayIndexOutOfBoundsException  
            int resultado = 10 / 0; // Esto provocará ArithmeticException  
        } catch (ArrayIndexOutOfBoundsException e) {  
            System.err.println("Error: Acceso fuera de los límites del array.");  
        } catch (ArithmeticException e) {  
            System.err.println("Error: No se puede dividir por cero.");  
        }  
        System.out.println("Fin del programa.");  
    }  
}
```

ArithmeticException

Si el error ocurre en la operación aritmética, se captura por el segundo bloque catch.

ArrayIndexOutOfBoundsException

Si el error ocurre en la primera línea del bloque try (acceso fuera del array), se captura por el primer bloque catch.

Bloque catch general

Seguridad adicional

Además de manejar excepciones específicas, es recomendable incluir un bloque catch para manejar excepciones de tipo **Exception**.

Red de seguridad

Este bloque catch general actúa como una "red de seguridad" para cualquier excepción no capturada por bloques catch más específicos.

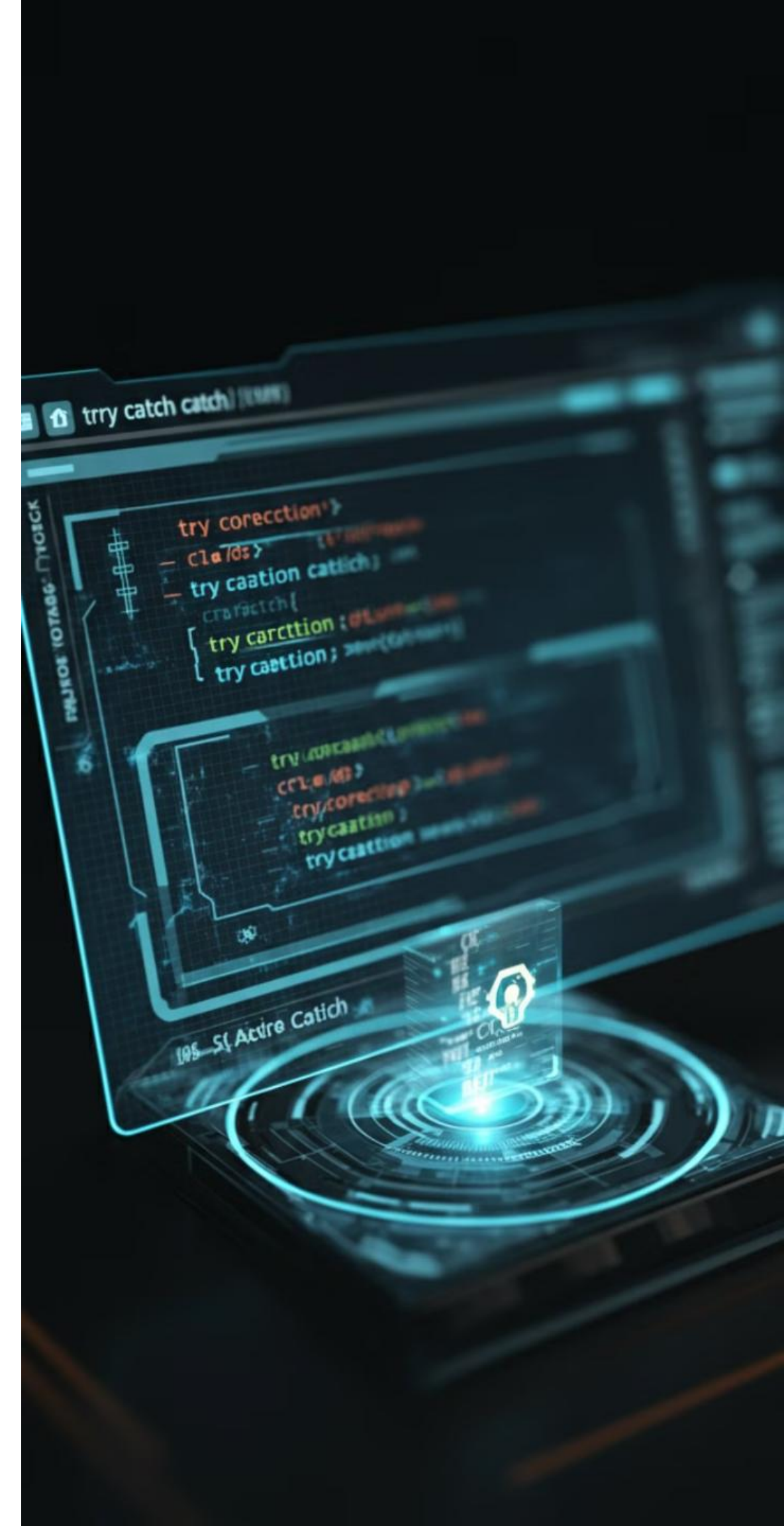
```
try {  
    // Código que puede lanzar excepciones  
} catch (TipoDeExcepcion1 e1) {  
    // Gestión específica  
} catch (Exception e) {  
    // Gestión general para cualquier otra excepción  
}
```

Jerarquía de excepciones en Java

Exception

Clase base de la que heredan todas las demás excepciones.

Los bloques catch deben estar organizados de la manera adecuada, colocando primero los tipos más específicos y dejando al final los más generales.



6.4. Declaración de excepciones en métodos

1

Excepciones no gestionadas

Cuando un método puede generar excepciones que no se manejan internamente, es necesario declararlas utilizando la cláusula **throws**.

2

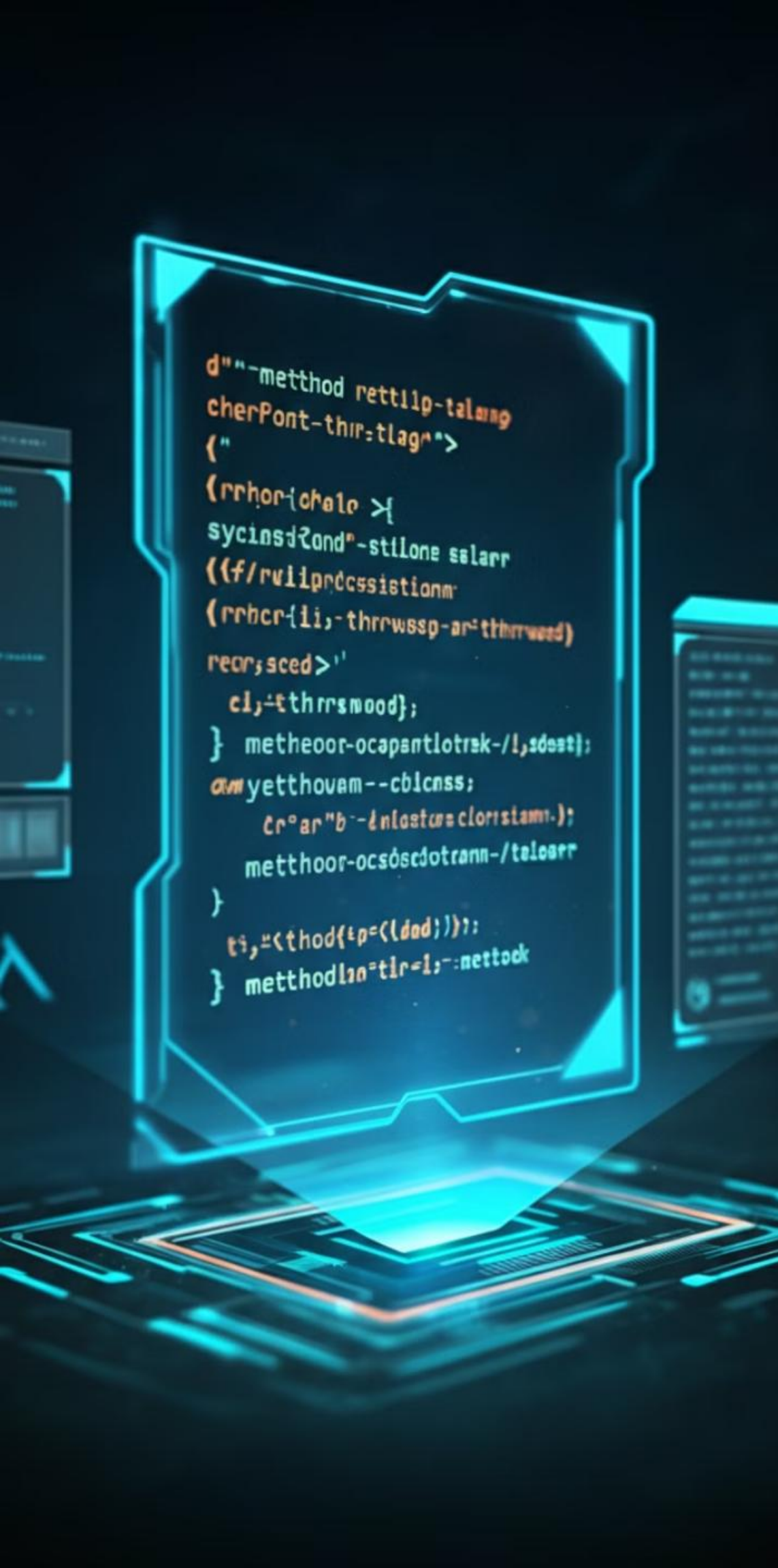
Informar a métodos

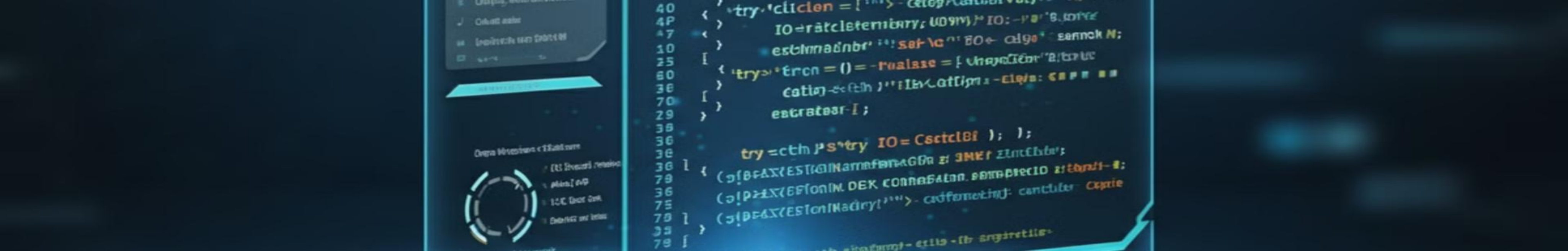
Esto indica a los métodos que llaman a este método que podrían enfrentarse a excepciones y deben estar preparados para gestionarlas.

3

Delegar responsabilidad

Al declarar excepciones con **throws**, el método no las gestiona internamente, sino que delega esa responsabilidad al código que lo invoca.





Ejemplo: Lanzar una IOException

1 Método que lanza IOException

El método **escribirArchivo** declara que puede lanzar una **IOException** utilizando **throws**.

2 Excepción no gestionada

Esto significa que el método no gestiona internamente la excepción, sino que la delega al código que lo invoca.

3 Bloque try-catch

El método que llama a **escribirArchivo** captura la **IOException** en un bloque try-catch.

```
import java.io.FileWriter;
import java.io.IOException;

public class EjemploThrows {
    public static void escribirArchivo(String contenido) throws IOException {
        FileWriter escritor = new FileWriter("archivo.txt");
        escritor.write(contenido);
        escritor.close(); // Puede lanzar IOException
    }
}

public class EjemploPrincipal {
    public static void main(String[] args) {
        try {
            EjemploThrows.escribirArchivo("Hola, mundo");
        } catch (IOException e) {
            System.err.println("Error al escribir en el archivo: " + e.getMessage());
        }
    }
}
```



Excepciones comprobadas vs. Excepciones no comprobadas

Excepciones comprobadas



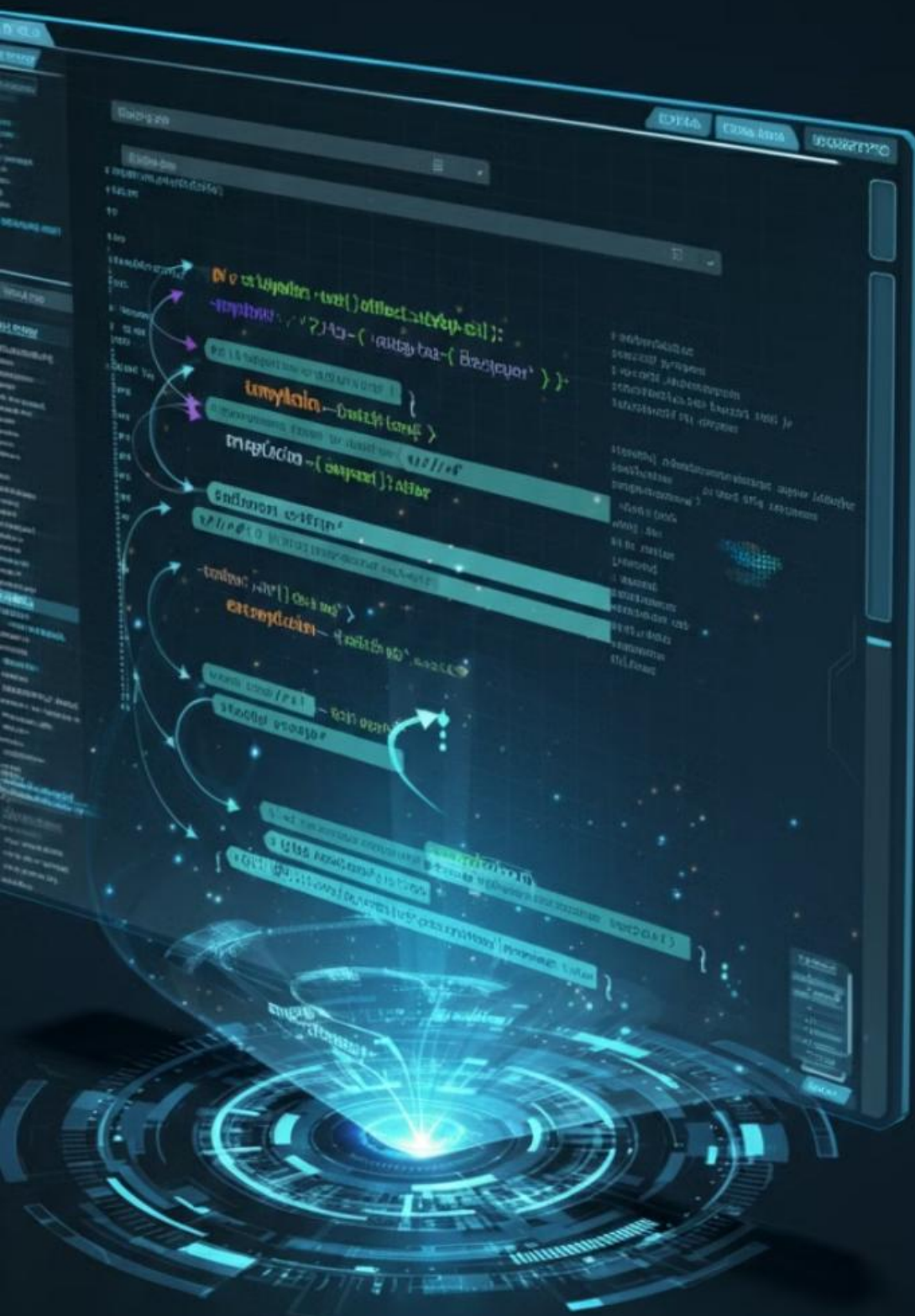
Son excepciones que el compilador obliga a declarar con **throws** o manejar con **try-catch**, como **IOException** y **SQLException**. Su manejo es esencial para operaciones críticas como E/S de archivos y acceso a bases de datos.

Excepciones no comprobadas



Estas excepciones, como **NullPointerException** y **ArrayIndexOutOfBoundsException**, no requieren declaración o manejo explícito, ya que heredan de **RuntimeException**. Su gestión no es obligatoria, pero puede mejorar la robustez del programa.

Resumen: delegar manejo de excepciones



1

Delegar responsabilidad

El uso de **throws** permite a los desarrolladores delegar el manejo de excepciones a los métodos que invocan funciones que pueden fallar.

2

Enfoque útil

Este enfoque es útil cuando el método en sí no puede o no debe encargarse de gestionar el error.

3

Manejo en niveles superiores

Permite que el manejo de excepciones se realice en niveles más altos del programa.



¿Para qué sirve **try/catch**?

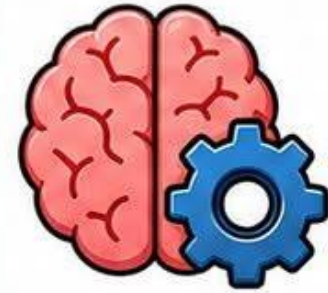
Manejo de errores y excepciones

🛡️ Cómo evitar que tu aplicación **explote**

```
try {  
  // código riesgoso  
  ejecutar();  
} catch (error) {  
  // manejar error  
}
```



1 Concepto



Estructura usada para **capturar** y **manejar errores** durante la ejecución.

2 Ejemplo

```
try {  
  // Código que puede fallar  
  let resultado = 10 / 0;  
  console.log(resultado);  
} catch (error) {  
  // Manejo del error  
  console.error("Ocurrió un error:", error.message);  
}
```

try

Intenta ejecutar el código

catch

Captura y maneja el error

error

Objeto con info del error

3 Claves



Captura errores



Evita caídas

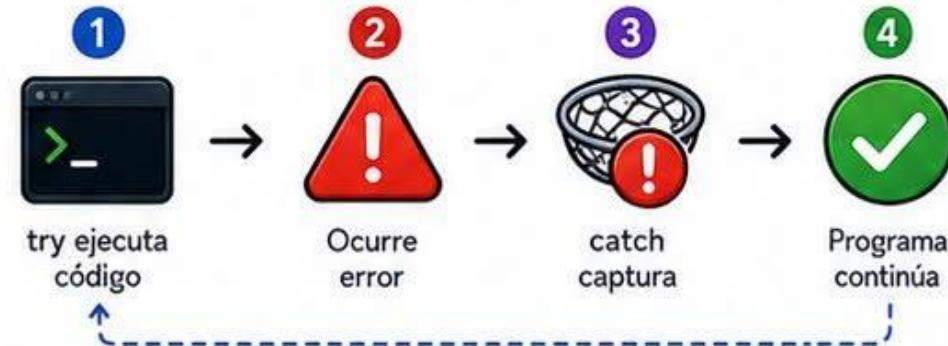


Manejo excepciones



Mayor estabilidad

4 Cómo funciona



5 Qué maneja



Errores API

Fallas en peticiones o respuestas

÷ 0

Divisiones inválidas

Operaciones matemáticas que fallan



Archivos faltantes

No encontrar archivos o recursos



Datos incorrectos

Información no válida o mal formateada

6 Ventajas



Apps más seguras



Menos crashes



Debugging fácil



Mejor experiencia

7 Uso real



Frontend

Manejo de errores en UI



Backend

Control de excepciones



APIs

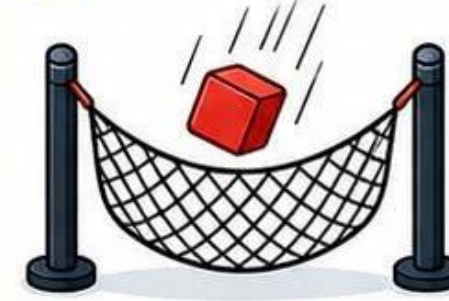
Respuesta a fallos externos



Bases de datos

Consultas y conexiones seguras

8 Analogía



Como una **red de seguridad** que evita que el programa se caiga por completo.



Clave

try/catch permite controlar errores **sin detener** toda la aplicación.

```
try {  
  código...  
}
```

¿Error?

Sí

```
catch (error) {  
  manejar error  
}
```

Programa continúa normalmente

Programa continúa normalmente



try/catch = Seguridad + Errores + Estabilidad



7. DOCUMENTACIÓN DE LAS APLICACIONES



[INTELIJ: VIDEO](#)
[ECLIPSE: VIDEO](#)

